

하네스 엔지니어링 — 회사 정책을 코드로 자동 적용한다

1강 settings.json·권한 + 2강 Hooks + 3강 Skills·Slash·MCP (총 120분)

코스	Course 02 · 사내 AI 구축·운영 종합 가이드
강의	1강(60분) + 2강(60분) + 3강(60분) = 총 180분
대상	IT 담당자 + 시니어 개발자
URL	visionc.co.kr/ai-solution/academy/build-ai
비밀번호	visioncDown (대소문자 구분)
형식	사내 출강 / 워크숍

INTRO · TITLE

하네스 엔지니어링

1강 — settings.json으로 회사 정책 코드화

 3분 · 시작 · 환영

SAY

“반갑습니다. 사내 AI 구축·운영 종합 가이드 6편 ‘하네스 엔지니어링’을 시작합니다. 본 강좌 11편 중 ★ 표시가 붙은 본격 운영 편입니다. 1편부터 5편까지는 ‘무엇을 도입할지’, ‘어떤 도구를 쓸지’를 결정하는 단계였다면 6편부터는 ‘그 도구를 우리 회사에서 안전하게 운영하려면 어떻게 정책을 코드화해야 하는가’를 본격적으로 다룹니다.”

SAY

“‘하네스(harness)’라는 단어가 낯설 수 있습니다. 원래 ‘말의 마구’를 뜻하는 영단어예요. 야생의 말은 강력하지만 통제되지 않으면 위험합니다. 그래서 옛 사람들은 마구·고삐·재갈을 만들어 말이 인간의 의도 안에서 일하도록 묶었습니다. 그게 바로 ‘하네스’입니다. 우리가 5편에서 본 에이전트는 강력합니다. 코드를 수정하고, 메일을 보내고, DB를 바꿀 수 있습니다. 하지만 그대로 두면 사고가 납니다. ‘하네스 엔지니어링’은 그 강력한 에이전트에 회사 정책이라는 마구를 채워서, 회사의 의도 안에서만 일하게 만드는 작업입니다.”

SAY

“6편 3강 구성을 미리 알려드립니다. 1강은 ‘settings.json’이라는 파일 하나로 권한·환경·모델을 표준화하는 방법입니다. 2강은 ‘Hooks’라는 메커니즘으로 에이전트의 모든 동작 전후에 자동 검증·로그·차단을 끼워 넣는 방법입니다. 3강은 ‘Skills + Slash Commands + MCP’ 세 가지를 결합해서 우리 회사의 도메인 지식을 코드 패키지로 묶고, 자주 쓰는 명령을 단축으로 만들고, 외부 시스템을 연결하는 방법입니다. 세 강이 끝나면 ‘우리 회사 하네스 폴스택 1페이지’라는 결과물이 손에 남습니다.”

SAY

“6편이 약속드리는 한 가지는 ‘초보자가 들어도 이해할 수 있는 깊이’입니다. settings.json의 모든 키를 외우거나, Hooks 함수를 직접 코딩하지 않으셔도 됩니다. 그 작업은 사내 IT가 합니다. 강의를 듣는 의사결정자가 해야 하는 일은 ‘우리 회사가 어떤 정책을 코드화할지’, ‘어떤 권한을 직원에게 줄지’, ‘어떤 사고를 사전에 막을지’를 결정하는 것입니다. 결정자가 방향을 알면 IT가 그것을 구현합니다.”

Q

도입 질문 1을 청중에게 던집니다. ‘하네스 엔지니어링이라는 표현을 이전에 들어보신 분 손 들어주세요.’ — 거의 없으리라 예상됩니다. 손이 안 올라가도 자연스러운 반응이에요. 이 용어는 Anthropic·OpenAI 같은 AI 회사들이 2024년 후반부터 본격적으로 사용하기 시작한 신조어입니다. 1년 안에 업계 표준 용어가 될 가능성이 큼니다. 강사는 ‘오늘 처음 들으셨다면 정상’이라며 청중을 안심시킵니다.

Q

도입 질문 2: ‘5편 끝에 ‘Claude Code 시작’을 결정하신 분?’ — 대부분 손을 듭니다. 5편에서 추천한 시작 셋 (Qwen 7B + Ollama + Q4 + Open WebUI 또는 Claude Code + Pro 구독)을 결정한 분들이 다수입니다. 강사는 ‘ 좋습니다. 6편은 그 결정을 본격 운영하기 위한 마구 만들기입니다. 결정만으로는 사고가 납니다. 마구를 채워야 안전합니다’라고 메시지를 연결합니다.

NOTE

★ 강사 핵심 배경지식 — 하네스가 ‘설정’과 다른 이유: ‘설정(configuration)’이라는 단어로 부르지 않는 이유가 있습니다. 일반 소프트웨어 설정은 ‘어떻게 작동할지’를 정합니다. 하네스는 ‘작동의 정책 + 검증 + 자동화’를 모두 포함합니다. 회사 정책 매뉴얼 100페이지를 직원 모두에게 읽히는 대신, 그 정책을 코드로 만들어 시스템이 자동으로 적용합니다. 직원의 실수 가능성이 0이 됩니다. 사내 AI 운영의 핵심 패러다임 전환입니다.

NOTE

강사 배경지식 — 6편 인용 출처: 본 강좌의 모든 settings.json·Hooks·Skills 문법은 Anthropic Claude Code 공식 문서(docs.claude.com/claude-code) 사양 2026년 6월 기준입니다. 사내 사례는 ‘How Anthropic teams use Claude Code’(claude.com/blog) 블로그를 1차 인용했고요. Skills는 2025년 출시 ‘Introducing Agent Skills’ 발표 자료를 기준으로 했습니다. 청중이 ‘출처가 어디예요’라고 물으면 위 URL을 안내합니다.

NOTE

강사 배경지식 — 6편 60분 시간 배분 (1강): 0~3분 인사·여는 메시지. 3~6분 학습목표. 6~10분 5편 복습. 10~16분 하네스라는 개념의 본질. 16~24분 settings.json의 구조와 핵심 3배열. 24~32분 Permission Modes 4가지. 32~40분 env·hooks·기타 키. 40~46분 한국 회사 권장 settings 5종 패턴. 46~52분 우선순위와 git 공유. 52~58분 실습. 58~60분 마무리·다음 강 예고. 시간 압박이 크므로 ‘각 개념 핵심만 + 디테일은 사내 IT가 추후 학습’ 메시지로 진행합니다.

TIP

강사 함정 — settings.json 문법을 자세히 외우라고 하지 마세요. JSON 키 이름·따옴표 위치를 외우는 건 사내 IT의 일입니다. 청중에게는 ‘구조와 의도’만 전달합니다. 청중이 ‘아 이런 게 가능하구나, 우리도 해보자’ 결정만 하면 6편의 목표는 달성됩니다.

“그럼 학습 목표를 빠르게 보고 본격 시작하겠습니다.”

OBJECTIVES

60분 후, 이 3가지를 한다

- ① settings.json 구조(allow / ask / deny)를 설명한다
- ② 한국 회사 권장 settings 5종 패턴을 안다
- ③ ‘우리 회사 settings.json 표준 1페이지’를 완성한다

🕒 3분 · 학습목표

SAY

“오늘 1강 60분이 끝나면 다음 세 가지를 할 수 있게 됩니다. 첫째, settings.json 파일의 구조 — allow(자동 허용)·ask(매번 승인)·deny(절대 차단) 세 개의 배열로 모든 권한이 표현된다는 것 — 을 동료에게 설명할 수 있습니다. ‘우리 회사는 이런 명령은 자동, 저런 명령은 매번 확인, 위험 명령은 차단’이라고 분석적으로 토론할 수 있게 되는 거예요.”

SAY

“둘째, 한국 회사가 단계별로 적용하는 settings 5종 패턴을 알게 됩니다. 첫 도입 1주에는 ‘최소 안전 패턴’, 1~3개월에는 ‘일반 운영 패턴’, 3~6개월에는 ‘본격 자율 패턴’ 식이에요. 우리 회사가 어느 단계에 있는지 파악하고, 다음 단계로 진화하는 시점을 결정할 수 있게 됩니다.”

SAY

“셋째, 가장 중요한 결과물 — ‘우리 회사 settings.json 표준 1페이지’ 워크시트를 완성합니다. 이 한 장이 사내 IT에게 ‘이렇게 작성해 주세요’ 발주서 역할을 합니다. 5편의 ‘에이전트 도구 결정서’와 합쳐서 사장님께 보고하는 ‘에이전트 운영 도면’이 만들어집니다.”

SAY

“1강 핵심 메시지를 미리 드릴게요. ‘settings.json은 단순 파일이 아닙니다. 우리 회사의 권한 매트릭스를 코드화한 결과물입니다. 한 번 작성해서 git에 커밋하면 전 직원이 같은 정책 아래에서 일하게 됩니다. 사장님 결재를 받은 회사 정책이 코드로 자동 적용되는 거예요.’ 이 메시지가 1강의 본질입니다.”

NOTE

★ 강사 핵심 배경지식 — 1강의 성공 기준: 청중이 ‘우리 회사 settings.json 시작 + 핵심 권한 결정’을 자신 있게 할 수 있어야 합니다. ‘완벽한 settings’를 작성하는 게 목표가 아니에요. ‘시작 + 점진 개선’ 마인드. 강사는 ‘60% 답으로 시작 → 운영하면서 보완’이라는 1편부터 이어진 메시지를 다시 강조합니다.

NOTE

강사 배경지식 — settings.json의 세 가지 레벨: 본 강좌에서 settings.json은 세 가지 레벨로 존재합니다. (1) 사용자 레벨 — ~/.claude/settings.json — 개인 컴퓨터의 개인 설정. (2) 프로젝트 레벨 — .claude/settings.json — 프로젝트 폴더 안에 git으로 공유되는 팀 설정. (3) Enterprise 레벨 — Anthropic 이 회사 전체에 강제 통제하는 설정. 우선순위는 Enterprise > 사용자 > 프로젝트 > 기본값. 사내 IT가 ‘회사 정책은 Enterprise 또는 프로젝트로 강제, 개인 취향만 사용자’로 분리합니다. 이 슬라이드에서는 ‘3레벨 존재한다’만 인지시키고, 디테일은 후반 슬라이드에서 자세히 다룹니다.

NOTE

강사 배경지식 — 학습 목표 슬라이드의 시간 압박: 3분 안에 끝내야 합니다. 학습 목표는 ‘오늘 60분 후 무엇을 할 수 있는지 약속’이지 ‘본문’이 아닙니다. 청중에게 ‘약속받았으니 60분 동안 듣자’는 동기 부여만 주고 빠르게 다음 슬라이드로 넘어가세요. 시간이 5분을 넘기면 본문 시간이 부족해집니다.

TIP

강사 진행 팁 — 학습 목표 3분 안에 끝내기 위한 진행. 0~1분 첫째 목표. 1~2분 둘째·셋째 목표. 2~3분 1강 핵심 메시지. 청중 발언 받지 않습니다. 본문에서 받습니다.

BRIDGE

“그럼 5편을 빠르게 복습하고 6편 위치를 명확히 한 다음 본문에 들어가겠습니다.”

RECAP · PART 5

5편 복습 — 에이전트 시작 결정

■ 불변층 — 5편의 5개 핵심

- 1강 — 에이전트 4요소(LLM·메모리·도구·플래너) + 5패턴 + 4단계 + ROI
- 2강 — Claude Code(공식 에이전트, 권한·Skills·Hooks·MCP)
- 3강 — 오픈 4종(OpenCode·Cline·Aider·Roo Code)
- ★ 추천 시작 — 개발팀 + Claude Code + 시니어 1명 + Pro 구독 \$20
- ★ 6편 위치 — 5편 ‘권한·Skills·Hooks·MCP’ 4가지의 본격 디테일

SAY

“5편을 안 들으신 분도 따라갈 수 있도록 5개 핵심을 30초씩 빠르게 복습하겠습니다. 5편은 ‘에이전트가 무엇이고 우리 회사 어디부터 시작할지’를 결정하는 강의였습니다.”

SAY

“첫째 — 에이전트의 4요소. 챗봇과 에이전트의 본질 차이를 풀이했습니다. 에이전트는 LLM(두뇌)·메모리(기억)·도구(손)·플래너(설계자) 네 가지가 결합된 시스템입니다. 챗봇은 ‘1회 답’만 하는데 에이전트는 ‘목표 인식 → 단계 계획 → 도구 실행 → 결과 평가 → 다음 단계’ 루프를 자율적으로 돌립니다. ‘답하기’가 아니라 ‘일하기’예요.”

SAY

“둘째 — 5가지 사용 패턴. 분류·요약·생성·검색·실행. 가장 안전한 패턴은 분류와 요약, 가장 위험한 패턴은 실행이에요. 첫 도입은 분류·요약부터 시작합니다.”

SAY

“셋째 — 4단계 도입 일정. 관찰(0~3개월) → 파일럿(3~6개월) → 확장(6~12개월) → 자율(12~24개월). 한국 회사 평균 12~24개월. 빠른 회사 6~12개월. 단계 건너뛰기는 사고로 이어집니다.”

SAY

“넷째 — Claude Code. Anthropic의 공식 에이전트입니다. CLI 인터페이스 + 권한(settings.json) + Skills(도메인 지식) + Hooks(자동 동작) + MCP(외부 통합) 네 가지 기능을 갖췄습니다. 본 강좌의 5편 2강에서 다뤘고, 오늘 6편이 그 ‘권한·Skills·Hooks·MCP’ 네 가지를 본격적으로 디테일하게 다룹니다.”

SAY

“다섯째 — 오픈 4종. Claude Code 대안으로 OpenCode·Cline·Aider·Roo Code. 자체 호스팅 + 로컬 LLM 옵션이 강점입니다. 보안 강박 회사 또는 클라우드 비용 줄이려는 회사에 적합해요.”

SAY

“5편 추천 시작 셋을 다시 — ‘개발팀 + Claude Code + 시니어 1명 + Pro 구독 월 \$20’입니다. 가장 안전하고 빠른 도입 패턴이에요. 오늘 6편이 그 추천 셋의 ‘운영 본격화’를 다룹니다.”

NOTE

★ 강사 핵심 배경지식 — 5편 vs 6편의 본질 차이: 5편은 ‘무엇을 쓸지 결정’입니다. Claude Code 또는 Cline 또는 OpenCode 중에서 우리 회사에 맞는 도구를 선택하는 단계예요. 6편은 ‘선택한 도구를 어떻게 운영할지 표준화’입니다. 도구는 같지만 운영 디테일이 회사마다 달라야 합니다. 같은 Claude Code를 카카오와 우리 회사가 똑같이 운영하면 안 됩니다. 카카오는 5,000명+ + 보안 강박 + 코딩 표준 풍부, 우리 회사는 50명 + IT 5명 + 표준 모호. 운영 디테일이 다릅니다. 6편이 그 ‘우리 회사 맞춤 운영’을 다룹니다.

NOTE

강사 배경지식 — 5편 안 듣고 6편만 듣는 청중 대응: ‘부서장·IT 담당자’ 중에 5편 안 들으신 분이 있을 수 있습니다. 이 슬라이드(4분 복습)가 그분들을 위한 다리 역할입니다. 단 ‘5편 디테일은 visionc.co.kr 강좌 페이지에서 자료 다운로드 후 학습 권장’이라고 안내합니다. 4분 복습으로 5편 전부 압축은 불가능 — ‘메인 메시지만’ 전달합니다.

NOTE

강사 배경지식 — 5편 워크시트와 6편 연결: 5편이 끝나면 청중 손에 ‘에이전트 도구 결정서 + Claude Code 도입 결정서 + 오픈 에이전트 결정서’ 3장이 남습니다. 6편이 끝나면 거기에 ‘settings.json 표준 + 첫 Hook 설계서 + 하네스 풀스택’ 3장이 추가됩니다. 5편 3장 + 6편 3장 = 에이전트 본격 운영 6장. 사장님께 보고하는 마스터 자료가 됩니다.

Q

강사가 청중에게 질문을 하나 던집니다. ‘5편 끝의 추천 시작 셋이 무엇이었나요?’ — ‘개발팀 + Claude Code + 시니어 1명 + Pro 구독 \$20’이라는 답이 나오면 좋습니다. 안 나오면 강사가 답합니다. 청중이 5편 내용을 기억하는지 빠르게 체크하는 용도입니다.

TIP

강사 진행 팁 — 4분 시간 배분: 0~2분 5편 5핵심 30초씩. 2~3분 추천 시작 셋 + 6편 위치 메시지. 3~4분 청중 질문. 복습이 5분을 넘으면 본문 시간이 부족해집니다. 강사는 시계를 보며 진행 속도를 조절하세요.

BRIDGE

“복습 끝났습니다. 이제 본격적으로 — 1번 본문, ‘하네스가 무엇인가’부터 시작하겠습니다.”

WHAT IS HARNESS

하네스 = 회사 정책 + 코드 + 자동

■ 불변증 — 본질

- 정책 매뉴얼 + 직원 실수 (하네스 없이) — 회사 정책 매뉴얼(종이) — ‘민감 데이터 외부 송신 금지’. 직원이 모르거나 잊어서 위반 → 사고 발생. 사후 처벌. 같은 사고 반복.
- 코드 + 자동 차단 (하네스 사용) — settings.json·Hooks가 ‘민감 데이터 자동 감지 + 차단’. 직원 실수 가능성 0. 사전 방지. 사고 발생 자체가 줄어들.

🕒 6분 · 하네스 풀이

SAY

“1강 본문 첫 번째 — 하네스가 정확히 무엇인지 풀이합니다. ‘회사 정책 + 코드 + 자동’이라는 세 단어로 정리됩니다.”

SAY

“하네스가 없는 상태를 먼저 봅시다. 일반 회사는 ‘정책 매뉴얼’이라는 종이 문서를 갖고 있습니다. 100페이지 분량. ‘민감 데이터 외부 송신 금지’, ‘.env 파일 git 커밋 금지’, ‘프로덕션 DB 직접 수정 금지’ 같은 조항이 적혀 있어요. 신입사원 입사 시 매뉴얼 보고 서명합니다. 그런데 — 6개월 후 직원이 그 100페이지를 기억할까요? 거의 못 합니다. 어느 날 급한 마음에 .env 파일을 그대로 git push해버립니다. 비밀 키가 외부에 노출됩니다. 사고가 납니다. 사후 처벌과 보안 교육 강화로 이어지지만, 같은 사고가 6개월 후 다른 직원에게서 또 일어납니다. 매뉴얼 기반 정책의 한계입니다.”

SAY

“하네스를 사용하면 그림이 바뀝니다. settings.json에 ‘.env 파일 Edit 금지’가 적혀 있으면, 직원이 Claude Code에게 ‘.env 파일 수정해줘’라고 요청해도 시스템이 거부합니다. ‘이 파일은 회사 보안 정책에 의해 수정 불가’ 메시지가 나오고 자동으로 차단됩니다. 직원의 의도가 무엇이든, 매뉴얼을 기억하든 못하든, 사고는 발생하지 않습니다. 시스템이 차단하기 때문이에요. 직원 실수 가능성이 0이 됩니다.”

SAY

“하네스의 본질 원칙은 ‘인간 신뢰가 아닌 시스템 차단’입니다. 인간은 누구나 실수합니다. 신뢰가 부족해서가 아니라 인간의 본성이 그래요. 그런데 시스템은 실수하지 않습니다. 코드는 정직합니다. ‘이 작업은 차단’이라고 적혀 있으면 100% 차단합니다. 회사 정책을 인간의 기억과 의지에 맡기지 않고 시스템에 맡기는 것 — 이게 본 강좌 9편 보안의 본질입니다. 6편은 그 보안 본질을 ‘에이전트 운영’에 적용한 결과물이에요.”

SAY

“하네스는 네 가지 도구로 구성됩니다. 첫째 settings.json — 정적 권한. ‘무엇 할 수 있나, 무엇 할 수 없나’ 정의. 둘째 Hooks — 동적 자동. ‘작업 전후에 무엇을 자동으로 검증·실행’. 셋째 Skills — 도메인 지식. ‘우리 회사의 일하는 방법’ 패키지. 넷째 Slash Commands — 단축. ‘자주 쓰는 명령’ 단축어. 네 가지가 결합되어 ‘하네스 폴스택’을 이룹니다. 1강은 settings.json, 2강은 Hooks, 3강은 Skills + Slash + MCP. 이 순서로 진행하겠습니다.”

STORY

출처 — Anthropic 사내 사례. 'How Anthropic teams use Claude Code' 블로그(URL: claude.com/blog/how-anthropic-teams-use-claude-code) 인용. 핵심 문구를 직접 인용하면 ‘every commit gets Hook-checked’ — 사내 모든 코드 커밋이 Hook을 통한 자동 검증을 거친다는 뜻입니다. Anthropic 사내 모든 개발자가 .claude/ 폴더에 settings·Hooks·Skills를 git으로 공유합니다. 사내 모든 사람이 같은 환경에서 일합니다. ‘직원 실수 = 시스템 차단’ 원칙의 실전 사례예요.

STORY

한국 회사 하네스 도입 사례 (강사 대응용 추정). 카카오·네이버 일부 개발팀이 6개월 이상 Claude Code 운영 후 ‘settings.json + 5~10개 Hooks + 10~20개 Skills’ 사내 표준을 갖췄습니다. 사내 코딩 가이드(이전에 100페이지 PDF로 존재), 보안 정책(별도 200페이지 PDF), 자동 테스트 정책이 모두 ‘.claude/ 폴더 안 settings·Hooks·Skills 파일’로 코드화됐어요. 신규 개발자가 입사 시 .claude/ 폴더를 git clone하면 첫날부터 사내 표준 환경에서 일합니다. 100페이지 매뉴얼 학습 시간 0.

STORY

하네스 도입 ROI (강사 대응용 추정). 도입 전 — 매월 사고 1~2건(.env 노출·rm 사고·민감 데이터 외부 송신 등). 도입 후 — 사고 0건 유지 + 개발 효율 +30%. 1회 작성 + 영구 적용 + 사고 0 + 효율 향상. ROI 측정이 어려울 정도로 큼니다.

NOTE

★ 강사 핵심 배경지식 — 하네스 4가지 도구의 위치: settings.json은 ‘무엇 할 수 있나’ 정적 권한. 한 번 작성하면 변하지 않습니다. Hooks는 ‘무엇 할 때 자동’ 동적 동작. 작업 전후에 함수가 실행됩니다. Skills는 ‘무엇 하는 방법’ 지식. 우리 회사가 영업 보고서를 어떻게 작성하는지, 코드 리뷰를 어떻게 하는지 같은 도메인 지식 패키지입니다. Slash Commands는 ‘자주 쓰는 명령’ 단축. /commit이라고 치면 우리 회사 표준 커밋 메시지 형식으로 자동 생성. 네 가지가 결합되어 폴스택을 이룹니다. 청중이 이 네 가지를 ‘권한·동작·지식·단축’ 네 단어로 기억하면 충분합니다.

NOTE

강사 배경지식 — 하네스의 ‘회사 레벨’: 작은 회사일수록 사용자 레벨(개인) 설정만 있고, 큰 회사일수록 Enterprise 레벨(회사 강제 통제) 설정이 강해집니다. 5명 회사라면 개인 settings 자유롭게 두고 사내 공유는 git README 정도로 충분합니다. 50~200명 회사는 프로젝트 레벨 settings를 git으로 공유. 200명+ 회사는 Enterprise 레벨 강제. 우리 회사 인원 규모에 맞게 단계적으로 진화시키면 됩니다.

NOTE

강사 배경지식 — 하네스 없이 도입했을 때 함정: 첫 Claude Code 사용자들이 자주 빠지는 함정입니다. ‘default settings 그대로 사용’. 함정 결과 — (1) ‘npm install·git push·rm -rf’ 같은 위험 명령이 자동 허용 상태. (2) 사내 정책(예: 특정 파일 보호)이 반영 안 됨. (3) 사고가 나도 누가 무엇을 했는지 추적이 어려움. (4) 사내 코딩 표준이 적용 안 됨. 도입 1~3개월 후 ‘작은 사고’가 발생하고, 그제서야 하네스의 필요성을 깨닫게 됩니다. 그러나 그때는 이미 ‘운영 중인 시스템에 정책 추가’라는 어려운 작업이 됩니다. 처음부터 하네스 갖춰 시작하는 게 정석입니다.

Q

예상 청중 질문 1: ‘우리는 첫 도입인데 하네스 없이 시작해도 되나요?’ → 답변: ‘권장하지 않습니다. 가장 기본적인 settings.json — 위험 명령 차단 4종(.env·secrets·rm·sudo) + 사내 정책 1~2개 — 만 추가해도 충분합니다. 작성 시간 1~2시간. 0에서 시작하는 게 아니라 ‘최소 안전’부터 시작하시는 게 정석입니다. 본 강좌 후반 슬라이드에서 ‘최소 안전 패턴’ settings.json 예시를 보여드립니다.’

Q

예상 청중 질문 2: ‘하네스 작성은 누가 하나요?’ → 답변: ‘사내 시니어 개발자 + IT 부서가 협업합니다. 첫 작성 1~2주, 사내 표준화에 1개월. 이후에는 ‘새 정책 발견 시 추가’ 정도로 유지보수 부담이 작습니다. 사내 인력이 부족하면 외주 컨설팅도 가능 — 약 200~500만 원에 ‘우리 회사 settings.json + 첫 Hooks 5개’ 패키지를 받을 수 있어요. 단 6개월 운영 후엔 사내 IT가 직접 유지하는 게 정석입니다.’

Q

예상 청중 질문 3: ‘5편 3강의 오픈 에이전트(Cline·Aider 등)에도 하네스 가능?’ → 답변: ‘부분적으로 가능합니다. Cline은 자체 설정 시스템이 있고, Aider는 .aider.conf.yml 같은 다른 형식이예요. 단 settings.json + Hooks + Skills의 풀스택은 Claude Code가 가장 풍부합니다. 본 강좌 6편은 Claude Code 기준으로 진행하겠습니다. 오픈 에이전트 하네스는 본 강좌 8편(자체 에이전트)에서 별도로 다룹니다.’

TIP

강사 함정 — 하네스의 본질 깊이를 더 파고들지 마세요. ‘회사 정책 + 코드 + 자동’이라는 세 단어와 ‘인간 신뢰 X 시스템 차단’ 원칙만 청중이 기억하면 충분합니다. 디테일은 본문 후반에서 자연스럽게 나옵니다.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 본질 풀이. 1~2분 ‘하네스 없이 vs 있으면’ 비교(말의 마구 비유 + .env 사고 시나리오). 2~3분 4가지 도구 소개. 3~4분 Anthropic 사내 사례. 4~5분 한국 사례 + 하네스 없이 도입 함정. 5~6분 청중 질문 1~2개. 비유와 사례를 통해 ‘하네스가 왜 필요한지’ 청중 마음에 자리잡게 하는 게 목표입니다.

BRIDGE

“하네스의 본질을 봤다면 — 이제 첫 번째 도구인 `settings.json`의 구조로 들어갑니다.”

SETTINGS STRUCTURE

settings.json 구조 — allow · ask · deny

■ 불변층 — 핵심 3개 배열

- ★ allow — 자동 허용 (예: ‘Bash(git status)’)
- ★ ask — 매번 승인 요청 (예: ‘Edit(**)’)
- ★ deny — 절대 차단 (예: ‘Bash(rm -rf *)’)
- ★ 패턴 문법 — ‘Tool(인자)’ + 와일드카드(**)
- ★ Tools — Read·Edit·Bash·WebFetch·MCP 등 약 10가지
- ★ 우선순위 — deny > ask > allow > 기본

⌚ 6분 · settings 구조

SAY

“settings.json의 구조를 봅니다. 파일 안에는 수많은 키가 있지만, 가장 중요한 건 ‘permissions’라는 키 아래의 세 개 배열 — allow·ask·deny입니다. 이 세 배열이 우리 회사의 모든 권한을 표현합니다.”

SAY

“첫 번째 배열 — allow. ‘자동 허용’이라는 뜻입니다. 여기에 적힌 작업은 매번 사용자 승인을 묻지 않고 바로 실행됩니다. 예를 들어 ‘Bash(git status)’를 allow에 적으면, Claude Code가 git status 명령을 실행할 때 사용자에게 ‘실행할까요?’ 묻지 않습니다. 빠른 작업 효율 + 매번 클릭 부담 제거. 단 — 이 배열에 적힌 작업은 사실상 ‘회사가 안전하다고 판단한 작업’입니다. 신중하게 정해야 합니다.”

SAY

“두 번째 배열 — ask. ‘매번 승인 요청’이라는 뜻이에요. 여기에 적힌 작업은 Claude Code가 실행하기 전에 사용자에게 ‘이 작업을 실행할까요? [예/아니오]’ 묻습니다. 사용자가 ‘예’ 해야 진행. 첫 도입 시 대부분 작업을 ask에 두는 게 안전합니다. 1주 운영 후 ‘이 작업은 매번 ‘예’ 해야 해서 답답하다 + 위험 작다’ 확인되면 allow로 이동합니다.”

SAY

“세 번째 배열 — deny. ‘절대 차단’이에요. 여기에 적힌 작업은 어떤 상황에서도 실행되지 않습니다. 사용자가 매번 승인하려 해도 시스템이 거부합니다. ‘rm -rf *’ 같은 위험 명령, ‘.env 파일 수정’ 같은 보안 위반, ‘사내 DB 직접 변경’ 같은 시스템 위험 작업이 여기에 들어갑니다. 회사 정책의 가장 강한 표현이 deny입니다.”

SAY

“패턴 문법은 단순합니다. ‘Tool(인자)’ 형식이고, 와일드카드로 ‘**’를 씁니다. ‘Bash(git status)’는 ‘git status 명령 하나만’, ‘Bash(git **)’는 ‘git으로 시작하는 모든 명령’, ‘Edit(src/**)’는 ‘src 폴더 안 모든 파일 편집’, ‘Edit(**)’는 ‘모든 파일 편집’입니다. 정규식이 아니라 단순 glob 패턴이라 직원도 보고 이해할 수 있어요.”

SAY

“Tools는 약 10가지가 있어요. Read(파일 읽기)·Edit(파일 수정)·Write(새 파일)·Bash(셸 명령)·WebFetch(URL 다운로드)·WebSearch(웹 검색)·Glob(파일 검색)·Grep(텍스트 검색)·Task(서브에이전트)·Skill(스킬 실행)·MCP 도구. 우리 회사가 어떤 Tools를 어떤 권한으로 줄지 매트릭스를 그려야 합니다.”

SAY

“우선순위가 중요합니다. 같은 명령에 ‘deny’와 ‘allow’가 둘 다 있으면 ‘deny가 이깁니다’. 즉 deny > ask > allow > 기본값. 이 순서는 ‘안전 우선’ 철학의 결과예요. 누군가 실수로 위험 명령을 allow에 넣어도 deny에 같은 명령이 있으면 차단됩니다. 안전망입니다.”

STORY

출처 — Anthropic Claude Code 공식 문서. docs.claude.com/claude-code/settings 페이지에 settings.json 전체 사양이 공개돼 있어요. ‘permissions’ 키 안의 allow·ask·deny 세 배열 구조는 공식 사양 그대로입니다. ‘약 30가지 키’라고 한 건 settings.json 전체에 permissions 외에도 env·hooks·model·statusLine 같은 다양한 키가 있다는 뜻이에요. 본 강좌 6편 1강은 permissions 위주로 진행하고, 다른 키는 후반 슬라이드에서 다룹니다.

STORY

한국 회사 권장 settings.json 예시 (강사 대응용). 첫 도입 1주에 권장하는 안전 패턴을 코드로 보여드리면 — { 'permissions': { 'allow': ['Read(**)', 'Bash(git status)', 'Bash(npm test)'], 'ask': ['Edit(**)', 'Bash(git push)', 'WebFetch(*)'], 'deny': ['Bash(rm -rf *)', 'Bash(sudo *)', 'Edit(.env)', 'Edit(secrets/**)'] } } . 약 10줄. 5분 작성. 이 정도가 '최소 안전' 시작점입니다. 1주 운영 후 사용자 피드백 받으면서 점진적으로 확장하면 됩니다.

STORY

Tools 종류 디테일 (강사 대응용). 각 Tool의 의미를 다시 정리하면 — (1) Read 파일 읽기, 보통 위험 작음 → allow 권장. (2) Edit 파일 수정, 사고 가능성 큼 → ask. (3) Write 새 파일 생성, Edit과 비슷. (4) Bash 셸 명령, 가장 강력 → 정확한 패턴으로 통제. (5) WebFetch URL 다운로드, 외부 통신 → ask 또는 화이트리스트. (6) WebSearch 검색, 일반적으로 안전 → allow. (7) Glob·Grep 파일·텍스트 검색, 안전 → allow. (8) Task 서버에이전트, 자율 동작 → ask. (9) Skill 스킬 실행, 도메인 지식 적용 → allow. (10) MCP 도구, 외부 시스템 → 도구별 다름. 청중이 Tool 이름을 외울 필요는 없지만, 'Tool에는 위험 등급이 다르다'는 인지는 필요합니다.

NOTE

★ 강사 핵심 배경지식 — settings.json 작성의 5단계 순서: 첫 작성 시 청중이 따라할 수 있는 단계를 정리하면 — (1) 위험 명령을 deny에 먼저 작성. rm·sudo·env·secrets 네 가지가 기본 deny. (2) 자주 쓰는 안전 명령을 allow에 작성. git status·npm test·Read(**) 정도. (3) 나머지는 모두 ask. '우리가 안 정한 건 매번 확인'이라는 안전 우선 철학. (4) 1주 운영 후 사용자 피드백 모으기. '이 작업 매번 ask해서 답답하다' 항목을 allow로 이동. (5) 사내 표준화 + git 공유. 1~4주 안에 완성됩니다. 이 5단계가 청중의 머릿속에 '우리 IT가 1주 작업하면 된다'는 시간 감각을 심어줍니다.

NOTE

강사 배경지식 — 와일드카드 패턴의 한계: settings.json의 패턴은 단순 glob입니다. 정규식이 아니에요. 'Bash(git push origin main)' 같은 정확한 명령은 매칭되지만 'Bash(git push origin main --force)' 같은 변형은 별도로 적어야 합니다. 'Bash(git push *)'로 모든 git push를 매칭할 수 있지만, 그러면 'git push --force' 같은 위험 명령도 함께 허용됩니다. 패턴 작성 시 '우리가 허용한 명령에 위험 옵션이 끼어둘 수 있나'를 항상 검토해야 합니다. 사내 시니어 1명이 1주 학습 + 작성 + 검토하면 충분한 수준이에요.

NOTE

강사 배경지식 — settings.json의 3레벨 우선순위: (1) Enterprise 레벨 — Anthropic 클라우드에 등록된 회사 강제 설정. 직원이 우회 불가능. 본격 대기업 도입 시 사용. (2) 사용자 레벨 — ~/.claude/settings.json. 직원 개인 컴퓨터에 있는 개인 설정. 직원 취향. (3) 프로젝트 레벨 — .claude/settings.json. git에 커밋되는 팀 공유 설정. 우선순위는 Enterprise > 사용자 > 프로젝트 > 기본값. 큰 회사일수록 Enterprise 강제, 작은 회사는 프로젝트 git 공유로 충분합니다. 우리 회사 규모에 맞게 단계 선택하면 됩니다.

Q

예상 청중 질문 1: ‘처음부터 deny 매트릭스를 만들어야 하나요?’ → 답변: ‘네, 권장합니다. `rm·sudo·env·secrets` 네 가지가 ‘기본 deny’입니다. 우리 회사가 이걸 안 정해도 사고 위험이 큰 명령이에요. 1시간 안에 작성 가능합니다. 첫 도입 시 가장 먼저 해야 할 작업입니다.’

Q

예상 청중 질문 2: ‘ask가 매번 물어보면 너무 답답하지 않나요?’ → 답변: ‘초기에는 답답할 수 있습니다. 하지만 1주 운영하면서 ‘이 작업은 매번 ask해도 위험 작네, allow로 옮기자’ 식으로 조정합니다. 1개월 후에는 자주 쓰는 안전 작업은 allow, 위험 작업은 ask로 안정됩니다. 단 ‘귀찮다고 모두 allow’ 옮기는 함정은 피하세요. 1주 운영 + 사용자 피드백 + 시니어 검토 후 옮깁니다.’

Q

예상 청중 질문 3: ‘다른 개발자와 settings를 공유하려면 어떻게?’ → 답변: ‘프로젝트 레벨 `settings.json` — `.claude/settings.json` 파일을 프로젝트 폴더 안에 두고 git에 커밋합니다. 다른 팀원이 git pull하면 자동으로 같은 settings를 갖게 돼요. 사내 표준 정책을 1회 작성하면 전 직원이 같은 환경에서 일하게 됩니다. 단 ‘개인 취향 설정’은 사용자 레벨로 분리합니다.’

TIP

강사 함정 — JSON 문법을 자세히 가르치지 마세요. ‘쉼표 위치·따옴표·중괄호’ 같은 디테일은 사내 IT의 일이에요. 청중에게는 ‘allow·ask·deny 세 배열 + 우선순위 `deny>ask>allow`’ 개념만 전달합니다.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 3배열 한눈에. 1~3분 각 배열 1분씩. 3~4분 패턴 문법과 Tools. 4~5분 우선순위 + 5단계 작성 순서. 5~6분 청중 질문 1~2개. 패턴 작성 예시를 칠판이나 슬라이드에 직접 보여주면 청중이 쉽게 이해합니다.

BRIDGE

“*settings.json* 구조를 봤다면 — 다음은 ‘Permission Modes’입니다. 같은 *settings.json*을 가지고도 운영 모드를 바꾸면 권한이 달라지는 메커니즘이에요.”

PERMISSION MODES

Permission Modes — 4가지 운영 모드

■ 불변층 — 상황별 모드 전환

- default 표준 모드 — settings.json 권한 그대로 따름. 안전과 효율의 균형. 평시 운영의 기본값.
- plan 계획 전용 (읽기만) — Edit·Bash·Write 도구 모두 차단. ‘이 코드 분석’ 같은 읽기 작업만 가능. 사고 위험 0. 첫 도입 1주 권장.
- acceptEdits 수정 자동 허용 — Edit에 대한 ask가 자동 allow로 전환. 빠른 작업에 유용. 단 위험 크므로 단기·신중 사용.
- bypassPermissions 모든 권한 자동 — 모든 ask·deny를 무시하고 자동 실행. ★ 절대 위험. ‘격리 환경 (sandbox)’ 안에서만 사용 가능.

🕒 6분 · 4모드

SAY

“Permission Modes는 ‘같은 settings.json을 가지고 운영 모드를 바꾸는 메커니즘’입니다. 4가지 모드가 있어요. 각 모드는 상황에 따라 다르게 사용합니다.”

SAY

“첫 번째 모드 — default. 가장 평범한 모드입니다. settings.json에 적힌 권한을 그대로 따라요. allow는 자동, ask는 매번 승인, deny는 차단. 평시 운영의 기본값이에요. 우리 회사가 첫 도입 후 안정화되면 거의 모든 시간을 default 모드에서 보냅니다.”

SAY

“두 번째 모드 — plan. ‘계획 전용’이라는 뜻이에요. 이 모드에서는 Edit·Bash·Write 같은 ‘쓰기·실행’ 도구가 모두 차단됩니다. Claude Code는 오직 Read·Grep·Glob 같은 ‘읽기’ 작업만 할 수 있어요. ‘이 코드 분석해줘’, ‘이 함수 어떻게 동작하는지 설명해줘’ 같은 작업이 가능. 단 실제 수정이나 명령 실행은 X. 사고 위험이 0입니다. 첫 도입 1주는 plan 모드에서 운영하는 걸 강력 권장합니다. ‘에이전트가 무엇을 할 수 있는지’를 안전하게 관찰하는 단계예요.”

SAY

“세 번째 모드 — acceptEdits. ‘수정 자동 허용’입니다. settings.json에서 Edit이 ask였더라도 이 모드에서는 자동 allow로 동작합니다. 빠른 작업이 필요할 때 유용해요. 예를 들어 ‘이 프로젝트 전체에 lint 적용해줘’ 같은 작업. 매번 ask하면 50번 클릭이 필요한데, acceptEdits 모드에서는 한 번에 끝납니다. 단 — 위험이 큼니다. Edit이 자동 실행되니까 잘못된 수정도 자동으로 들어갑니다. 단기 사용, 신중 사용 원칙을 지켜야 해요. 1~2시간 작업 후 default로 돌아오는 게 정석.”

SAY

“네 번째 모드 — `bypassPermissions`. ★ 가장 위험한 모드입니다. 모든 `ask·deny`를 무시하고 모든 작업을 자동 실행합니다. `settings.json`의 안전망이 사실상 무효화돼요. 어떤 상황에서 사용하느냐 — ‘격리 환경 (sandbox)’ 안에서만 사용 가능합니다. Anthropic Managed Agents의 self-hosted sandbox나 Docker 격리 환경처럼 ‘사고가 나도 사내 메인 시스템에 영향이 없는’ 격리 환경 안에서만. 평소 사내 시스템에서 `bypassPermissions`를 켜면 큰 사고로 이어집니다.”

SAY

“모드 전환은 명령줄 인자나 슬래시 커맨드로 합니다. ‘`claude --permission-mode plan`’으로 plan 모드 시작, 또는 세션 안에서 ‘`/permissions plan`’으로 전환. `default·plan·acceptEdits` 사이 전환은 자유롭게, `bypassPermissions`로의 전환은 사내 정책으로 제한합니다.”

STORY

출처 — Anthropic Claude Code 공식 문서. docs.claude.com/claude-code/permissions 페이지에 Permission Modes 4가지가 정의돼 있어요. 핵심 문구를 직접 인용하면 ‘plan mode is recommended for safe exploration’ — plan 모드는 안전한 탐색에 권장된다는 뜻. 또 ‘`bypassPermissions` should only be used in isolated environments’ — `bypassPermissions`는 격리 환경에서만 사용하라는 명시. 공식 문서가 ‘격리 환경 외 사용 금지’를 명확하게 못 박고 있습니다.

STORY

한국 회사 모드 사용 패턴 (강사 대응용). 일반적인 도입 흐름을 보여드리면 — (1) 신입 개발자가 처음 Claude Code를 사용할 때 1주는 plan 모드. ‘무엇 할 수 있는지’ 관찰만. (2) 1주 후 default 모드로 전환. 본격 작업 시작. `settings.json`의 `ask`가 매번 승인을 요구. (3) 빠른 작업이 필요할 때 30분~1시간 `acceptEdits`로 잠시 전환. 작업 끝나면 default로 복귀. (4) `bypassPermissions`는 Anthropic Managed Agents sandbox 같은 격리 환경에서만. 평소 사내에서는 사용 금지가 사내 정책. 이 단계별 사용이 안전한 첫 도입의 표준 패턴입니다.

STORY

한국 회사 `bypassPermissions` 사고 사례 (강사 대응용 추정). 일부 회사에서 첫 도입자가 ‘귀찮다고 `bypassPermissions` 켜고 평소 작업’을 한 사례가 있어요. 결과 — 1주일 안에 ‘잘못된 `rm` 명령으로 작업 폴더 삭제’ 또는 ‘민감 파일 외부 송신’ 같은 사고. 사후 정책으로 ‘`bypassPermissions`는 격리 환경에서만 + 매번 시니어 승인’이 추가됐습니다. 청중에게 ‘`bypass`는 절대 평소 X’ 메시지를 강조하세요.

NOTE

★ 강사 핵심 배경지식 — 4모드의 ‘사용 흐름’: 한국 회사가 따라가야 할 시간순 사용 패턴을 정리하면 — (1) 첫 1주 — plan 모드. 안전 관찰 + 사용자 학습. (2) 1~4주 — default 모드 + 대부분 작업 ask. 시간이 걸려도 안전이 우선. (3) 1~3개월 — default 모드 + allow 확대. 자주 쓰는 안전 작업이 allow로 이동. ask는 점차 줄어듦. (4) 3개월+ — default 모드 안정 + 빠른 작업 시 acceptEdits 30분 사용. bypassPermissions는 절대 격리 환경 외 X. 이 4단계 흐름이 청중에게 ‘우리 회사도 1주씩 단계 진화’ 시간 감각을 줍니다.

NOTE

강사 배경지식 — bypassPermissions의 ‘격리 환경’ 의미: 격리 환경(sandbox)이 정확히 무엇이나 — (1) Anthropic Managed Agents self-hosted sandbox. Anthropic이 제공하는 격리 옵션으로, AI 모델은 Anthropic 클라우드, 작업 환경은 회사 서버에 격리됩니다. 사고가 나도 격리 환경 안에서만. 4편 3강에서 다른 옵션이에요. (2) Docker 격리 환경. 사내 IT가 Docker 컨테이너 안에 Claude Code를 띄우고, 외부 시스템과 분리. 사고 시 컨테이너만 삭제하면 끝. (3) 클라우드 sandbox 서비스. AWS·GCP 같은 클라우드의 임시 격리 환경. 이 세 가지 외에는 bypassPermissions를 켜지 마세요. 본 강좌 9편 보안 편에서 더 자세히 다룹니다.

NOTE

강사 배경지식 — plan 모드의 ‘교육적 가치’: plan 모드가 ‘첫 1주 안전 관찰’ 외에도 다른 가치가 있어요. 신입 직원이 Claude Code를 처음 학습할 때, plan 모드에서 ‘에이전트가 우리 코드를 어떻게 이해하는지’ 관찰하면 학습 효과가 큼니다. 에이전트가 ‘이 코드는 이런 의도로 보입니다, 이런 수정이 가능해 보입니다’라고 제안하는 걸 사용자가 보고 ‘아 이렇게 자율 작업하는구나’ 감각이 생깁니다. 직접 수정을 안 하니 안전. 본격 작업 전 ‘과외 학습’ 단계로 plan 모드 활용을 권장하세요.

Q

예상 청중 질문 1: ‘첫 도입 시 plan 모드 1주 권장하셨는데, 1주가 너무 길지 않나요?’ → 답변: ‘회사 + 직원에 따라 달라요. IT 강한 회사 + 시니어 직원 — 3~4일이면 충분합니다. 일반 회사 + 신입 + 보안 강박 — 1~2주 권장. plan 모드에서 ‘에이전트가 무엇 할 수 있는지’ 충분히 관찰하는 게 사고 방지의 핵심이에요. 짧게 가면 ‘안본 위험 작업’이 default 모드에서 일어나 사고가 납니다.’

Q

예상 청중 질문 2: ‘acceptEdits는 정말 안전한가요? 자주 사용해도 되나요?’ → 답변: ‘단기·신중 사용 원칙이 있습니다. 30분~1시간 같은 짧은 작업, 그리고 ‘이번 작업은 안전하다고 사용자가 인지’한 상태에서만. 자주 사용 권장 X. 자주 켜면 ‘ask의 안전망이 사실상 무력화’돼요. 정상 운영은 default + ask 중심으로.’

Q

예상 청중 질문 3: ‘bypassPermissions는 무엇이고 어떤 상황에서 쓰는지 다시 설명해주세요.’ → 답변: ‘bypassPermissions는 모든 권한 검증을 무시하고 자동 실행하는 모드입니다. 평소 사내 시스템에서는 절대 켜지 마세요. ‘격리 환경(Anthropic Managed Agents sandbox·Docker 격리·클라우드 sandbox) 안에서 본격 자율 작업’ 시에만 사용 가능합니다. 격리 환경은 사고가 나도 사내 메인 시스템에 영향이 없는 분리된 공간 이에요. 이 옵션은 본 강좌 9편 보안 편에서 더 자세히 다룹니다.’

TIP

강사 함정 — bypassPermissions의 위험을 한 번 더 강조하세요. 청중 중에 ‘귀찮으니 bypass 켜고 쓰면 안 되나’ 생각하는 분이 있을 수 있습니다. ‘격리 환경 외에서 bypass = 사고는 시간 문제’라는 메시지를 명확히 전달합니다.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 4모드 한눈에. 1~4분 각 모드 디테일(default 30초, plan 1분, acceptEdits 30초, bypassPermissions 1분 + 위험 강조). 4~5분 사용 흐름 + bypass 격리 환경. 5~6분 청중 질문 1~2개. bypass의 위험성에 시간을 충분히 할애하세요.

BRIDGE

“4모드를 봤다면 — 다음은 settings.json의 다른 키들입니다. env·hooks·additionalDirectories 같은 키가 어떤 역할을 하는지 봅시다.”

ENV·HOOKS·DIRS

env · hooks · 기타 키

■ 불변층 — settings.json의 추가 키

- ★ env — 환경변수 (API 키·플래그 등)
- ★ hooks — Hooks 정의 (2강 본격 자세히)
- ★ additionalDirectories — 추가 작업 폴더
- ★ model — 사용 모델 (claude-3-5-sonnet 등)
- ★ statusLine — 상태 줄 사내 브랜딩
- ★ 약 30가지 키 — 본 강좌는 핵심 4~5개만

SAY

“settings.json에는 ‘permissions’ 외에도 약 30가지 키가 있습니다. 다 외울 필요는 없어요. 핵심 4~5가지만 알면 됩니다. 본 강좌가 다루는 5가지 — env·hooks·additionalDirectories·model·statusLine — 부터 보겠습니다.”

SAY

“첫째 — env. 환경변수입니다. ‘ANTHROPIC_API_KEY’ 같은 API 키, ‘DISABLE_AUTOUPDATER=1’ 같은 플래그를 여기에 적습니다. 한 번 적으면 Claude Code가 실행될 때마다 자동으로 적용돼요. 회사 표준 환경변수를 settings.json에 적어 git 공유하면 전 직원이 같은 환경에서 일합니다.”

SAY

“둘째 — hooks. 2강에서 본격 자세히 다룰 메커니즘이에요. settings.json의 hooks 키에 ‘어떤 이벤트에 어떤 함수 실행’을 정의합니다. PreToolUse·PostToolUse 같은 이벤트 + Bash·Python 함수. 1강에서는 ‘hooks 키가 있다’만 인지하고, 디테일은 2강에서.”

SAY

“셋째 — additionalDirectories. Claude Code의 기본 작업 폴더는 ‘프로젝트 루트’입니다. 추가로 ‘../shared’ 같은 외부 폴더에서도 작업하고 싶다면 이 키에 등록합니다. 사내 공유 라이브러리·도큐먼트 폴더를 참조할 때 유용해요.”

SAY

“넷째 — model. Claude Code가 어떤 모델을 사용할지 지정합니다. ‘claude-3-5-sonnet-20241022’, ‘claude-opus-4-1’ 같은 모델 ID를 적어요. 작업 종류에 따라 다른 모델을 쓸 수 있어요 — 빠른 작업은 Sonnet, 깊은 분석은 Opus 식으로.”

SAY

“다섯째 — statusLine. Claude Code의 상태 줄 표시를 커스터마이징합니다. 회사 로고·현재 부서·세션 시간·사용 토큰 수 같은 정보를 표시할 수 있어요. 사내 브랜딩과 직원 즉시 인식에 효과적입니다. 4편 1강의 ‘UI 커스터마이징’과 같은 사상이에요.”

STORY

출처 — Anthropic Claude Code 공식 문서. docs.claude.com/claude-code/settings 페이지에 settings.json의 전체 키 목록이 있어요. 약 30가지 키가 정의돼 있고, 그중 가장 자주 쓰이는 게 본 강좌가 다루는 5가지(permissions·env·hooks·model·statusLine·additionalDirectories)입니다. 그 외에는 outputStyle(출력 형식), includeCoAuthoredBy(커밋 메시지에 Claude 표기), defaultMode(기본 Permission Mode), cleanupPeriodDays(임시 파일 정리 주기) 같은 키가 있어요. 사내 IT가 ‘우리 회사에 필요한 키’를 docs에서 찾아 추가합니다.

STORY

한국 회사 statusLine 사례 (강사 대응용). statusLine을 활용하는 한국 회사 사례를 보여드리면 — 회사 로고 (이모지 또는 단축 이름) + 부서명 + 현재 모델 + 세션 시간이 표시됩니다. 직원이 Claude Code를 열 때마다 ‘아 이건 우리 회사 도구구나’ 즉시 인식해요. 작은 커스터마이징이지만 직원 친화감 + 신뢰감을 크게 향상시킵니다. 5분 작업으로 가치 큰 디테일이에요.

STORY

env 키의 ‘회사 표준’ 가치 (강사 대응용). env에 적는 환경변수 중 회사 표준으로 권장하는 것 — (1) DISABLE_AUTOUPDATER=1. Claude Code 자동 업데이트 비활성화. 회사가 통제된 시점에 업데이트 수동 적용. 새 버전의 버그가 갑자기 사내에 전파되는 걸 막아줍니다. (2) ANTHROPIC_BASE_URL=사내 프록시 URL. Anthropic API 호출이 사내 프록시를 통해 외부로 나가도록. 사내 보안 정책 적용과 감사 로그 수집이 가능. (3) 사내 인증 토큰. 본 강좌 9편 보안 편이 자세히 다룹니다.

NOTE

★ 강사 핵심 배경지식 — 핵심 4키 우선순위: 30가지 키 중 첫 도입 시 다루는 핵심 4가지를 정리하면 — (1) permissions — 권한 (오늘 1강의 본문). 첫 작성 1시간. (2) env — 환경변수. DISABLE_AUTOUPDATER + 사내 프록시 + 인증 토큰 3가지. 30분. (3) hooks — 자동 동작 (2강 자세히). 1주~1개월에 5개 작성. (4) statusLine — 사내 브랜딩. 5분. 4가지를 1주 안에 작성하면 ‘기본 settings.json 표준’이 완성됩니다. 나머지 26가지 키는 ‘필요 시 docs 검색 후 추가’ 정도면 충분.

NOTE

강사 배경지식 — env 키의 보안 고려: API 키나 인증 토큰 같은 비밀 정보는 settings.json에 직접 적으면 안 됩니다. settings.json은 git에 커밋되니까 비밀 정보가 외부에 노출됩니다. 대신 (1) .env 파일에 비밀 정보 적기 — 단 .gitignore에 추가해서 git에 안 가도록. (2) settings.json env에는 ‘ANTHROPIC_API_KEY=\${API_KEY_FROM_ENV}’ 같은 변수 참조. (3) 1Password·HashiCorp Vault 같은 secrets manager 사용. 본 강좌 9편 보안 편이 자세히 다룹니다.

NOTE

강사 배경지식 — model 키와 비용: model 키에 어떤 모델을 지정하느냐가 비용에 큰 영향을 줍니다. (1) Claude Opus 가장 비싸지만 가장 강력. 깊은 추론·복잡 코딩. 토큰당 \$15~75. (2) Claude Sonnet 균형. 일반 작업. 토큰당 \$3~15. (3) Claude Haiku 가장 저렴. 단순 작업. 토큰당 \$1~5. ‘기본은 Sonnet, 복잡한 작업은 Opus, 단순 작업은 Haiku’로 분리하면 비용 30% 절감 가능. 본 강좌 11편(관리자 운영)에서 자세히 다룹니다.

Q

예상 청중 질문 1: ‘env에 API 키를 직접 적어도 되나요?’ → 답변: ‘권장 안 합니다. settings.json은 git에 커밋되니까 비밀 정보가 노출됩니다. 대신 .env 파일에 비밀 정보를 두고 .gitignore에 추가하세요. settings.json에는 ‘\${API_KEY}’ 같은 변수 참조만. 본 강좌 9편 보안 편이 자세히 다룹니다.’

Q

예상 청중 질문 2: ‘model 키에 어떤 모델을 지정해야 하나요?’ → 답변: ‘기본은 Claude Sonnet 권장. 균형 잡힌 성능과 비용. ‘이번 작업은 깊은 분석이 필요해’ 같은 경우만 Opus 사용. 단순 검색·요약은 Haiku로 비용 절감. ‘작업별 모델 선택’이 가능합니다.’

Q

예상 청중 질문 3: ‘30가지 키를 다 외워야 하나요?’ → 답변: ‘아닙니다. 본 강좌가 다룬 5가지만 알면 충분합니다. 나머지는 ‘필요 시 docs.claude.com/claude-code/settings 검색 후 추가’ 정도로 대응. 사내 IT가 1년 운영하면서 ‘이 키가 필요하다’ 발견되면 그때 추가합니다.’

TIP

강사 함정 — 30가지 키 디테일을 외우게 하지 마세요. ‘4가지 핵심 + 필요 시 docs’ 메시지만 전달합니다. 청중 부담을 줄여야 합니다.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 5가지 키 한눈에. 1~3분 각 30초씩. 3~4분 보안·비용 고려. 4~5분 청중 질문 1~2개. env의 비밀 정보 함정에 시간을 충분히 할애하세요.

BRIDGE

“키를 봤다면 — 한국 회사 권장 settings.json 5종 패턴으로 넘어갑니다. 단계별로 어떤 settings를 갖춰야 하는지 봅시다.”

KOREA STANDARDS

한국 회사 settings.json 5종 패턴

■ 가변층 — 2026.6 권장 패턴

- 첫 도입 1주 (최소 안전) — deny 4종(rm·sudo·env·secrets) + 모든 작업 ask + allow 비율. 매번 승인 요청 + 안전 우선. 1주 운영 후 다음 단계.
- 1~3개월 안정 (일반 운영) — deny 5종 + 자주 쓰는 안전 작업(Read·git status·npm test)을 allow + 위험 작업(Edit·git push) ask. 효율과 안전 균형.

🕒 6분 · 5종 패턴

SAY

“한국 회사가 단계별로 적용하는 settings.json 5종 패턴을 정리해드리겠습니다. 첫 도입부터 본격 운영까지 단계별로 진화시키는 구조예요.”

SAY

“첫 번째 패턴 — 최소 안전. 첫 도입 1주에 적용합니다. deny 배열에 네 가지 — rm·sudo·env·secrets — 만 적어요. allow 배열은 비웁니다. 나머지 모든 작업은 자동으로 ask 처리. 즉 ‘아무것도 자동 허용 X, 위험 명령은 차단, 나머지는 매번 확인’이라는 가장 안전한 상태예요. 직원이 매번 클릭해야 해서 다소 답답하지만, ‘에이전트가 무엇을 시도하는지’ 모두 확인할 수 있어 학습 + 안전 효과가 큼니다.”

SAY

“두 번째 패턴 — 일반 운영. 1~3개월 안정화 후 적용합니다. deny가 5종으로 늘어나요(외부 fetch 추가). allow에 자주 쓰는 안전 작업이 들어갑니다 — Read(**)·Bash(git status)·Bash(npm test) 정도. ask에는 위험 작업이 — Edit(**)·Bash(git push)·WebFetch(*) — 정도. 효율과 안전의 균형이 잡힌 ‘평시 운영’ 패턴이에요. 한국 회사 60~70%가 이 단계에서 정착합니다.”

SAY

“세 번째 패턴은 ‘본격 자율’ 3~6개월 안정 후. allow가 더 확대되고, Hook과 Skills가 본격 결합됩니다. 6편 2~3강의 본문이에요. 네 번째는 ‘자체 호스팅 + 보안 강화’ — 사내 LLM 사용 + Egress 차단 + 사내 프록시. 다섯 번째는 ‘Enterprise 통제’ — Anthropic이 회사 전체에 강제, 직원 X 우회. 큰 회사 본격 도입.”

SAY

“진화 흐름은 ‘1주 → 1~3개월 → 3~6개월 → 6개월+’입니다. 단계 건너뛰기는 사고를 부릅니다. 1단계에서 ‘답답하다’고 바로 2~3단계로 점프하면 ‘어떤 작업이 위험한지 학습 안 한 상태에서 자동 허용 확대’되어 사고가 납니다. 1주 단위로 단계 진화시키세요.”

STORY

한국 회사 settings.json 5종 패턴의 비율 (강사 대응용 추정). 2026년 6월 기준 한국 회사 settings.json 분포를 추정하면 — 최소 안전 패턴 약 20%(첫 도입 후 1주~1개월), 일반 운영 패턴 약 60%(1~6개월 안정), 본격 자율 패턴 약 10%(6개월+), 자체 호스팅 + 보안 강박 약 5%(보안 회사), Enterprise 통제 약 5%(대기업). 약 80%의 한국 회사가 1·2단계에 있어요. 우리 회사가 어디인지 파악하고 다음 단계로 진화시키는 게 6편의 목표입니다.

STORY

Anthropic 사내 패턴 (강사 대응용). 'How Anthropic teams use Claude Code' 인용 — Anthropic 사내는 ‘Skills + Hooks + MCP 통합 + 부서별 settings’ 본격 운영 단계입니다. 본 강좌의 5단계로 보면 ‘본격 자율 + Enterprise’ 결합. 다만 Anthropic도 처음부터 본격으로 가지 않았어요 — 6개월 단계 진화 후 정착. 한국 회사 ‘부서별 settings 분리’ 패턴을 참고하기 좋습니다.

NOTE

★ 강사 핵심 배경지식 — 5종 패턴의 ‘진화 시점’: 단계 진화의 판단 기준을 명확히 해드리겠습니다 — (1) 최소 안전 → 일반 운영: ‘1주 운영 + 사고 0건 + 사용자 피드백 모음’ 충족 시 전환. 사용자가 ‘이 작업은 매번 ask해도 위험 작네’라는 항목들을 allow로 옮기는 작업. (2) 일반 운영 → 본격 자율: ‘1~3개월 운영 + Hooks 5종 도입 + Skills 5개 도입’ 충족 시. 6편 2~3강 진행 후 가능. (3) 본격 자율 → 자체 호스팅 + 보안: ‘본격 자율 안정 + 보안 부서 요구’ 시. (4) → Enterprise 통제: 대기업 본격 도입 + Claude Enterprise 구독. 단계별 정확한 진화 시점을 청중에게 인지시킵니다.

NOTE

강사 배경지식 — ‘자체 호스팅 + 보안 강박’ 패턴 디테일: 4번 패턴이 무엇을 의미하는지 자세히 보면 — settings.json의 env에 사내 프록시 URL이 적힙니다. Claude Code의 모든 API 호출이 사내 프록시를 통해 외부로 나갑니다. 사내 프록시는 (a) 호출 로그 수집 (b) 데이터 마스킹 (c) 위험 키워드 차단 자동 수행. allow에는 사내 도구만 — 외부 fetch는 deny로 차단. 그리고 model 키에는 ‘사내 호스팅 LLM URL’을 가리키게 해서 사실상 Claude Code의 백엔드를 Ollama·vLLM으로 바꿔버립니다. 본 강좌 9편 보안 편이 자세합니다.

NOTE

강사 배경지식 — Enterprise 통제 패턴 디테일: 5번 패턴은 Anthropic의 Enterprise 구독을 통해 적용됩니다. Anthropic 관리 콘솔에서 ‘회사 전체 적용 settings’를 설정하면, 회사 안 모든 Claude Code 인스턴스가 자동으로 그 settings를 강제 적용받아요. 직원이 자기 ~/.claude/settings.json에서 deny를 우회하려 해도 Enterprise 통제가 이깁니다. 우선순위 Enterprise > 사용자. 대기업 본격 도입 시 ‘회사 정책 강제’가 필요한 케이스에서 의미. 비용은 별도 견적(Anthropic과 협의)이에요.

Q

예상 청중 질문 1: ‘1주 후 자동으로 2단계로 전환되나요?’ → 답변: ‘자동 X. 수동 검토와 결정이 필요해요. 1주 운영 후 시니어 1명이 (a) 사고 발생 여부 확인 (b) 사용자 피드백 수집 (c) ‘자주 ask해서 답답한 작업’ 목록 정리 (d) allow로 이동할지 결정. 결정 후 settings.json 업데이트 + git 커밋. 이게 1단계 → 2단계 전환의 정석입니다.’

Q

예상 청중 질문 2: ‘5번 Enterprise 통제는 우리 회사도 가능한가요?’ → 답변: ‘Anthropic Enterprise 구독이 필요합니다. 비용은 회사 규모·사용량 협의로 결정. 견적 받으려면 sales@anthropic.com 연락. 중소기업은 보통 비현실적 비용 — 그래서 1~3 패턴이 한국 회사 대부분 정답입니다.’

Q

예상 청중 질문 3: ‘다른 회사의 settings.json을 참고할 수 있나요?’ → 답변: ‘가능합니다. GitHub에서 ‘.claude/settings.json’ 검색하면 공개된 사례들이 나와요. Anthropic 공식 예시도 docs에 있어요. 단 ‘우리 회사 정책에 맞게 조정’이 필수입니다. 그대로 복사 X — 우리 회사 문화·보안·도구가 다르니까 그대로면 안 맞아요. 사례를 ‘출발점’으로 삼고 조정하세요.’

TIP

강사 함정 — 5종 패턴의 디테일을 깊이 가지 마세요. ‘단계 진화’ 메시지만 청중에게 남기면 충분합니다. ‘우리 회사 어디인지 파악 + 다음 단계로’.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 5종 한눈에. 1~3분 1·2 패턴 디테일. 3~4분 3~5 패턴 짧게. 4~5분 진화 흐름 + 단계 건너뛰기 함정. 5~6분 청중 질문 1~2개. 1·2 패턴에 시간을 많이 할애하세요 — 80% 한국 회사가 거기 있어요.

“5종 패턴을 봤다면 — settings.json의 우선순위와 git 공유 메커니즘으로 넘어갑니다. 사내 표준이 어떻게 전 직원에게 자동 적용되는지 봅시다.”

PRIORITY·GIT

우선순위 + git 공유

■ 불변층 — 3레벨 결합 + 사내 표준화

- ★ 우선순위 — Enterprise > 사용자 > 프로젝트 > 기본
- ★ 사용자 — ~/.claude/settings.json (개인 취향)
- ★ 프로젝트 — .claude/settings.json (팀 공유 + git 커밋)
- ★ Enterprise — managed (회사 강제, 직원 X 우회)
- ★ git 공유 — 프로젝트 레벨 + CLAUDE.md 결합
- ★ secrets·API 키 — git 절대 X (.gitignore 필수)

🕒 5분 · 우선순위·git

SAY

“settings.json은 한 파일이 아니에요. 세 가지 레벨에 동시에 존재할 수 있고, 그 사이 우선순위가 정해져 있습니다. 이걸 이해하면 ‘사내 표준이 어떻게 직원에게 자동 적용되는지’ 그림이 그려집니다.”

SAY

“우선순위는 Enterprise > 사용자 > 프로젝트 > 기본값 순입니다. 같은 키가 여러 레벨에 있으면 ‘회사 강제’ > ‘개인 설정’ > ‘팀 공유’ > ‘Claude Code 기본값’으로 결정돼요. 가장 강한 게 Enterprise, 가장 약한 게 기본값입니다.”

SAY

“사용자 레벨 — ~/.claude/settings.json. 직원 개인 컴퓨터의 홈 폴더에 있는 파일이에요. ‘내 단축키·취향·자주 쓰는 작업’ 같은 개인적인 설정. 직원마다 다를 수 있어요.”

SAY

“프로젝트 레벨 — .claude/settings.json. 프로젝트 폴더 안에 있는 파일이에요. git에 커밋하면 팀 전체가 공유합니다. 신규 직원이 git pull 받으면 자동으로 같은 settings를 갖게 돼요. ‘이 프로젝트의 표준’ 위치예요.”

SAY

“Enterprise 레벨 — Anthropic 관리 콘솔에서 회사 전체에 강제 통제하는 설정. 직원이 자기 사용자 settings 에 deny를 우회하려 해도 Enterprise가 이깁니다. 회사 보안 정책의 가장 강한 표현이에요. 단 Claude Enterprise 구독이 필요해요.”

SAY

“실전 사용법 — 한국 회사 대부분은 ‘프로젝트 레벨 + git 공유’가 정답입니다. .claude/settings.json + CLAUDE.md + Skills 폴더를 git에 커밋. 신규 직원이 ‘프로젝트 clone’하면 자동으로 사내 표준 환경이에요. Enterprise는 본격 대기업 도입 단계.”

SAY

“그리고 ★ 한 가지 절대 규칙 — secrets·API 키는 settings.json에 직접 적지 마세요. settings.json은 git에 커밋되니까 비밀 정보가 외부에 노출됩니다. .env 파일 별도 + .gitignore에 추가 + 환경변수로 참조하는 패턴을 쓰세요.”

STORY

출처 — Anthropic Claude Code 공식 문서. docs.claude.com/claude-code/settings 페이지에서 ‘Enterprise managed settings override all’ 원칙이 명시돼 있어요. ‘Enterprise 설정은 다른 모든 설정을 덮어쓴다’는 뜻. 회사가 ‘직원이 deny 우회 절대 X’ 보장이 필요하면 Enterprise를 사용합니다. 일반 한국 회사는 프로젝트 + git 공유로 사실상 동등한 효과를 낼 수 있어요(직원이 의도적으로 우회하려면 가능하지만 ‘무심코 우회’는 차단).

STORY

Anthropic 사내 git 공유 패턴 (강사 대응용). 'How Anthropic teams use Claude Code' 인용에 따르면 사내 모든 개발자가 .claude/ 폴더를 git으로 공유합니다. 폴더 안에는 (1) settings.json — 사내 표준 권한. (2) commands/ — 슬래시 커맨드. (3) skills/ — 도메인 지식 패키지. (4) CLAUDE.md — 프로젝트 가이드. 네 가지가 git history에 남아 ‘회사 정책 변경 추적’이 가능해요. ‘정책이 코드 + git history로 관리되는’ 흐름이에요.

STORY

한국 회사 git 공유 패턴 (강사 대응용). 일부 한국 IT 회사가 이미 ‘.claude/ 폴더 git 표준화’를 적용하고 있어요. 신규 개발자 입사 시 ‘프로젝트 clone’ 하면 첫날부터 사내 표준 환경에서 일합니다. 100페이지 매뉴얼 학습 시간 0. ‘회사 정책 = 코드 = git’ 패러다임이에요. ROI 측정이 어려울 정도로 큼니다.

NOTE

★ 강사 핵심 배경지식 — git 공유의 4가지 가치: 사내 표준이 git 공유 시 얻는 가치를 정리하면 — (1) 신규 멤버 즉시 표준. 입사 첫날 프로젝트 clone하면 자동으로 같은 settings·Skills·Hooks. 학습 비용 0. (2) 정책 1회 작성 + 영구 적용. ‘직원에게 매뉴얼 매번 학습’ X. (3) git history로 정책 변경 추적. ‘언제 어떤 정책이 추가됐는지’ 명확. 사고 시 추적 가능. (4) PR 리뷰로 정책 검증. 새 정책 추가 시 PR에서 시니어 검토. 잘못된 정책이 사내 적용되는 걸 막아줌. 4가지가 결합되어 ‘회사 정책 = 코드 + git 흐름’ 패러다임이 완성됩니다.

NOTE

강사 배경지식 — .gitignore 패턴: secrets·키·개인 settings는 git에 가면 안 됩니다. .gitignore 파일에 다음 패턴을 추가하세요 — ‘.env’, ‘.env.local’, ‘.env.*.local’, ‘.claude/settings.local.json’, ‘.claude/cache/’. 첫 도입 시 .gitignore 작성이 필수입니다. 안 하면 비밀 정보가 외부 git 저장소(GitHub 등)에 노출돼요. 본 강좌 9편 보안 편이 자세히 다룹니다.

NOTE

강사 배경지식 — 우선순위 충돌 시 처리: 같은 키가 여러 레벨에 있으면 우선순위 그대로 적용입니다. 예 — 프로젝트 deny에 ‘Bash(rm -rf *)’, 사용자 allow에 ‘Bash(rm -rf *)’가 있다면? 우선순위는 사용자 > 프로젝트니까 사용자가 이깁니다 — 즉 rm -rf 허용. 위험합니다. 이걸 막으려면 Enterprise > 사용자 > 프로젝트인데 Enterprise를 쓰면 됩니다. 또는 ‘프로젝트 deny는 사용자 allow가 덮어쓰지 못한다’ 같은 정책을 사내 표준으로 정해 직원 교육으로 통제합니다. 이게 모호하지 않은 정책이 필요한 이유예요. 본 강좌 9편 보안 편에서 다시.

Q

예상 청중 질문 1: ‘우리 회사도 Enterprise 설정이 가능한가요?’ → 답변: ‘Claude Enterprise 구독 + Anthropic과 협력이 필요합니다. 큰 회사 본격 도입 단계예요. 비용은 회사 규모·사용량에 따라 협의. 중소기업은 보통 프로젝트 git 공유로 사실상 충분한 효과를 냅니다.’

Q

예상 청중 질문 2: ‘직원이 자기 사용자 settings에서 회사 deny를 우회할 수 있나요?’ → 답변: ‘이론적으로 가능합니다. 사용자 > 프로젝트 우선순위 때문이에요. 단 (1) Enterprise 사용 시 X 우회. (2) 사내 정책으로 ‘프로젝트 deny 우회 시 sanctions’ 명시 + 직원 교육. (3) 정기 감사로 직원 settings 검토. 세 가지가 결합되어 우회 가능성을 0에 가깝게 합니다.’

Q

예상 청중 질문 3: ‘1명 직원이 자기 settings에 위험 작업을 추가하면 어떻게?’ → 답변: ‘본 강좌 2강 Hooks가 답입니다. PreToolUse Hook이 ‘settings.json 변경 감지 + 시니어 알림’ 같은 패턴으로 모니터링. 사고 사전 방지. 그리고 분기 1회 정기 감사 — IT가 전 직원 ~/.claude/settings.json을 점검. 위험 추가 시 즉시 시정.’

TIP

강사 함정 — Enterprise의 디테일 깊이 X. ‘3레벨 + 우선순위’ 개념만 청중에게. 본격 Enterprise 도입은 본 강좌 9·11편.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 우선순위 + 3레벨. 1~3분 각 레벨 디테일. 3~4분 git 공유 + 4가지 가치. 4~5분 secrets 함정 + 청중 질문.

BRIDGE

“우선순위와 git 공유를 봤다면 — 다음은 보안 패턴과 사고 방지입니다. 권한 매트릭스로 마무리하기 전에 보안 본질을 한 번 짚습니다.”

SECURITY

보안 패턴 + 사고 방지

■ 불변증 — settings.json 보안 핵심

- ★ secrets·API 키 — git 절대 X (.gitignore + 환경변수)
- ★ 기본 deny 5종 — rm·sudo·env·secrets·외부 fetch
- ★ ask 강제 — Edit(**) + Bash(중요 명령)
- ★ Enterprise 강제 — 회사 정책 직원 X 우회
- ★ 정기 감사 — 분기 1회 settings 점검
- ★ 사고 시 — 로그 분석 + 권한 즉시 강화

🕒 5분 · 보안

SAY

“settings.json 보안 패턴을 정리합니다. 6가지 — 비밀 정보 보호·기본 deny·ask 강제·Enterprise 강제·정기 감사·사고 대응. 본 강좌 9편 보안 편의 미리보기이기도 합니다.”

SAY

“첫째 — secrets·API 키는 git에 절대 X. 앞에서도 강조했지만 다시 — .gitignore에 .env 패턴 추가 + 비밀 정보는 환경변수 또는 1Password·Bitwarden 같은 secrets manager. settings.json env에는 ‘\${API_KEY}’ 같은 변수 참조만. 이게 보안의 가장 기본 첫 단계예요.”

SAY

“둘째 — 기본 deny 5종. rm·sudo·env·secrets·외부 fetch. 첫 settings.json 작성 시 이 5종은 무조건 deny에 포함하세요. 1시간 작업으로 ‘가장 흔한 5가지 사고 시나리오’를 사전 차단합니다.”

SAY

“셋째 — ask 강제. Edit(**)과 중요 Bash 명령은 ask로 둡니다. 자동 허용 X. 사용자가 매번 ‘이 작업을 실행할까요?’ 확인. 이 부분이 답답하다고 곧바로 allow로 옮기지 마세요. 1주 운영 후 ‘위험 작은 작업만’ 신중하게 allow로.”

SAY

“넷째 — Enterprise 강제. 큰 회사 본격 도입 단계. ‘직원이 자기 settings에서 회사 정책 우회 X’ 보장이 필요한 경우 사용. Claude Enterprise 구독 + Anthropic 협의.”

SAY

“다섯째 — 정기 감사. 분기 1회 IT가 사내 settings.json들을 점검. ‘위험 명령이 추가됐나’, ‘우회 사례가 있나’ 검토. 1년 4회 + 1회당 2~4시간 작업이에요.”

SAY

“여섯째 — 사고 시 대응. 로그를 분석해 ‘어떤 작업이 사고로 이어졌는지’ 추적. settings.json에 deny 추가 + Hooks 강화. 같은 사고를 두 번 안 일어나게 합니다. 본 강좌 7편·9편이 자세히.”

STORY

한국 회사 settings 사고 사례 (강사 대응용 추정). 일부 회사에서 발생한 사고 패턴 — 직원이 ‘귀찮다’고 자기 ~/.claude/settings.json에 ‘allow: [Bash(**)]’ 추가. 그 결과 위험 명령이 자동 실행되며 사고. 사후 대응으로 (1) Enterprise 강제 추가. (2) 정기 감사 1회 → 분기 1회로 강화. (3) ‘위험 변경 시 슬랙 알림’ Hook 추가. 본 강좌 9편이 이 사고 패턴들을 자세히 다룹니다.

STORY

secrets manager 도구 (강사 대응용). 비밀 정보 관리에 추천하는 도구 — (1) 1Password Business. 월 \$8/인. 한국 회사 가장 인기. 사내 표준 구축 쉬움. (2) Bitwarden. 무료 옵션 있음. 작은 회사에 적합. (3) AWS Secrets Manager. 이미 AWS 사용 회사. (4) HashiCorp Vault. 본격 엔터프라이즈. 자체 호스팅 가능. 한국 중견기업은 (1) 또는 (4) 가장 많이 사용합니다. 본 강좌 9편이 도구 선택 가이드 제공.

STORY

Anthropic 보안 정책 (강사 대응용). Anthropic 자체가 'claude.ai/api/security'에 공식 보안 정책을 공개하고 있어요. 핵심 — (1) 입력 데이터 모델 학습에 사용 X. (2) 운영 로그 30일 보관 후 삭제. (3) 엔터프라이즈 약관 시 데이터 보호 보장. 한국 회사가 'Anthropic API를 사내 도구로 쓸 때 안전 보장'의 근거예요. 단 100% 보장 X — 추가 안전망이 필요한 이유.

NOTE

★ 강사 핵심 배경지식 — 보안 운영의 5단계 사이클: 일년을 통해 보안을 운영하는 패턴을 정리하면 — (1) 도입 시점 — 기본 deny 5종 작성. 첫 가장 흔한 사고 차단. 1시간. (2) 1개월 후 — Enterprise·Hook·감사 도구 검토. '우리 회사가 어떤 안전망을 추가할지' 결정. 1주. (3) 분기 1회 — 정기 감사 + 정책 업데이트. 새로운 사고 시나리오 발견 시 반영. 1회 2~4시간. (4) 사고 발생 시 — 즉시 강화. 로그 분석 + deny·Hook 추가 + 같은 사고 차단. 1~3일. (5) 연 1회 — 전체 정책 재검토. 기술 트렌드·사내 변화에 맞춰 settings 재구성. 1주. 5단계 사이클이 '안전 운영'의 표준 패턴입니다.

NOTE

강사 배경지식 — 권한 매트릭스 'Tool × 역할' 패턴 예고: 다음 슬라이드에서 자세히 다루는 권한 매트릭스의 핵심 개념을 미리 — Tool 5~10가지 × 사용자 역할 4가지(신입·시니어·시니어+·관리자) 매트릭스에 'auto / ask / deny'를 채워 넣습니다. 사내 표준 권한 매트릭스 1장이예요. 이걸 settings.json으로 코드화하면 '역할 별 자동 권한'이 실현돼요. 본 강좌 6편 1강의 결론에 해당하는 작업입니다.

NOTE

강사 배경지식 — 보안과 효율의 트레이드오프: '너무 강한 보안 = 직원 답답함 + 효율 ↓' 함정이 있습니다. (1) 모든 작업을 ask로 두면 직원이 매번 클릭하느라 시간 낭비. (2) 너무 엄격한 deny면 정상 작업도 차단되어 '우회' 시도 → 보안 약화. (3) 적정 균형은 '위험 큰 작업만 엄격 + 안전 작업은 자동'. 1단계 안전 → 1주 운영 → 2단계 균형 단계 진화가 정석. 단계 건너뛰기 X, 단계 정체 X.

Q

예상 청중 질문 1: '우리 회사 secrets manager가 없는데 어떻게 시작?' → 답변: '1Password Business 월 \$8/인부터 시작 추천. 한국 회사 가장 인기. 또는 사내 Vault 자체 호스팅. 무료로 시작하려면 .env 파일 + .gitignore 기본 패턴부터. 본 강좌 9편이 도구 선택 가이드를 자세히 제공합니다.'

Q

예상 청중 질문 2: 'Enterprise는 구체적으로 무엇을 강제하나요?' → 답변: 'Anthropic 관리 콘솔에서 회사 전체에 적용할 settings를 정의합니다 — 예: deny 항목, env 환경변수, sandbox 설정 등. 이 설정은 직원이 우회 못 합니다. 우선순위 Enterprise > 사용자. 큰 회사 본격 도입 + 보안 강박 케이스에 사용합니다.'

Q

예상 청중 질문 3: ‘사고 발생 시 누가 대응하나요?’ → 답변: ‘사내 보안팀 + IT 부서가 우선. 사전에 ‘사고 대응 플레이북’ 1장 작성 권장. ‘사고 감지 → 즉시 차단 → 영향 평가 → 복구 → 정책 강화’ 5단계. 본 강좌 9·10편이 자세히 다룹니다.’

TIP

강사 함정 — 보안 디테일 깊이 X. ‘5단계 사이클 + 6가지 패턴’ 개념만 청중에게. 디테일은 9편.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 6가지 패턴 한눈에. 1~3분 각 30초씩. 3~4분 5단계 사이클 + 트레이드 오프. 4~5분 청중 질문 1~2개.

BRIDGE

“보안 패턴을 봤다면 — 1강의 결론인 ‘권한 매트릭스’입니다. Tool과 역할의 교차로 사내 표준 권한이 정의돼요.”

MATRIX

권한 매트릭스 — Tool × 역할

■ 불변층 — 사내 표준

- ★ 역할 4가지 — 신입 / 시니어 / 시니어+ / 관리자
- ★ Tools 5가지 — Read·Edit·Bash·WebFetch·MCP
- ★ 신입 — Read 자동 + 기본 Bash 자동, 나머지 ask
- ★ 시니어 — Edit 자동 + Bash 다수 자동
- ★ 시니어+ — bypass 격리 환경 사용 가능
- ★ 관리자 — Enterprise 설정 권한

🕒 4분 · 매트릭스

SAY

“1강의 결론입니다. 권한 매트릭스 — Tool과 역할의 교차로 사내 표준 권한이 정의됩니다. 이 매트릭스 1장이 우리 회사 settings.json의 기반이 돼요.”

SAY

“역할은 4가지로 분류합니다. 신입 — 입사 1년 이하 또는 Claude Code 학습 초기. 시니어 — 1년 이상 경험 또는 본격 사용자. 시니어+ — 시니어 중 ‘추가 권한 신청 + 승인 받은 자’. 관리자 — IT 부서·CIO 등 회사 정책 결정자.”

SAY

“Tools는 5가지 — Read·Edit·Bash·WebFetch·MCP. 각 Tool별 위험 등급이 다릅니다. Read는 안전, Edit는 중간, Bash는 강력, WebFetch는 외부 통신, MCP는 외부 시스템. 위험 등급에 따라 역할별 권한이 차등화돼요.”

SAY

“신입의 권한은 가장 엄격합니다. Read는 자동(안전 + 학습 필수), 기본 Bash(git status 같은) 자동, 나머지 — Edit·고급 Bash·WebFetch·MCP — 모두 ask. 매번 승인 요청. 신입이 학습 중 안전 실수를 막아줘요.”

SAY

“시니어는 권한이 넓어집니다. Read 자동 + Edit 자동 + Bash 다수 자동. WebFetch와 MCP는 일부만 ask. ‘본격 작업 효율’이 목적. 단 위험 명령은 여전히 ask로.”

SAY

“시니어+는 bypassPermissions 격리 환경 사용이 가능합니다. ‘긴 자율 작업’이 필요할 때 격리 환경에서 모든 권한 자동. 일반 시니어는 X. 시니어+ 권한은 ‘시니어 추천 + 관리자 승인’으로 부여.”

SAY

“관리자는 Enterprise 설정 권한입니다. 회사 전체에 적용되는 강제 settings를 정의할 수 있어요. Claude Enterprise 구독 + Anthropic 콘솔 접근 권한. 최고 책임자급.”

STORY

권한 매트릭스 예시 (강사 대응용). 한국 회사 권장 매트릭스를 표로 보여드리면 — | 역할 | Read | Edit | Bash | WebFetch | MCP | | 신입 | auto | ask | ask(일부 auto) | ask | ask | | 시니어 | auto | auto | auto(위험 ask) | ask(화이트리스트 auto) | auto(중요 ask) | | 시니어+ | auto | auto | auto | auto | auto + bypass | | 관리자 | Enterprise 강제 적용 |. 5 Tools × 4 역할 = 20개 셀에 권한이 채워집니다. 1주 작성 작업이에요.

STORY

역할 ‘자동 승격’ 메커니즘 (강사 대응용). 신입 → 시니어 자동 승격이 어떻게 이뤄지는지 — (1) 입사 1년 후 자동 또는 (2) 시니어 추천 + 부서장 승인 후 일찍 승격. (3) 시니어 → 시니어+는 ‘시니어 추천 + 관리자 승인 + 사고 0건 1년’ 조건. 본 강좌 9편이 정책을 자세히. 매트릭스가 단순히 정적이지 않고 직원의 성장과 함께 진화하는 구조예요.

STORY

한국 회사 매트릭스 도입 사례 (강사 대응용 추정). 일부 한국 IT 회사가 이미 ‘Tool × 역할 매트릭스’를 사내 표준화했어요. settings.json + Enterprise + 역할별 자동 부여(LDAP·OAuth 연동). 신규 직원이 입사하면 자동으로 ‘신입’ 역할 권한을 받고, 1년 후 자동 ‘시니어’ 승격. 직원 별 권한 관리 부담이 거의 0이에요. 5명+ 회사 권장 수준.

NOTE

★ 강사 핵심 배경지식 — 매트릭스 실제 구현 방법: 매트릭스를 어떻게 실제 settings.json으로 변환하는지 — (1) 역할별 settings.json 4가지 작성. 신입용·시니어용·시니어+용·관리자용. (2) LDAP·SSO에서 사용자 역할 조회. (3) 역할에 맞는 settings.json 자동 배포. 사내 IT가 자동화 스크립트 작성. (4) 분기 1회 매트릭스 재검토. ‘새 Tool 추가됐나’, ‘권한 조정 필요한가’. 한국 회사 5명+ 권장 수준. 본격 5~10명+ 회사에서는 거의 필수입니다.

NOTE

강사 배경지식 — 매트릭스의 ‘5번째 역할’: 일부 회사는 5번째 역할 ‘외부 협력사’를 추가합니다. 외주 개발자나 컨설턴트가 사내 시스템 일시 사용할 때 권한 매트릭스. 가장 제한적 권한 — Read 일부 + 모든 작업 ask. 본 강좌는 4역할로 진행하지만 ‘외부 협력사 사용 케이스’가 있는 회사는 추가 검토.

NOTE

강사 배경지식 — 매트릭스 ‘시간 진화’: 매트릭스가 도입 후 시간이 지나면서 진화해야 합니다. (1) 1개월 후 — 직원 피드백 수집 + ‘이 권한 부족’ 항목 추가. (2) 3개월 후 — 사고·미사용 권한 분석 + 매트릭스 재조정. (3) 6개월 후 — 본격 안정 + 새로운 Tool 추가 (MCP 새 서버 등). (4) 1년 후 — 전체 재검토 + 회사 정책 변화 반영. 매트릭스가 ‘정적 문서’가 아닌 ‘살아있는 사내 정책’이에요.

Q

예상 청중 질문 1: ‘우리 회사 5~10명. 매트릭스가 너무 무겁지 않나요?’ → 답변: ‘5명도 매트릭스 권장. 단 ‘작은 매트릭스’로 시작. 역할 2가지(신입·시니어) + Tool 3가지(Read·Edit·Bash). 매트릭스 2×3 = 6개 셀만. 작성 1시간. 회사가 커지면 매트릭스도 확장.’

Q

예상 청중 질문 2: 'bypassPermissions는 시니어+만 가능하다는데, 어떤 작업에서 사용하나요?' → 답변: '격리 환경에서의 본격 자율 작업이 대표예요. 예 — '이 큰 기능 다 만들어줘' 같은 6~12시간 자율 작업. 사용자가 매 단계 확인하면 부담 큼. 격리 환경(Docker·Anthropic Managed Agents sandbox)에서 bypass 켜고 자율 진행. 작업 끝나면 격리 환경에서 결과만 가져옴. 사고가 나도 격리 환경 안에서만.'

Q

예상 청중 질문 3: '매트릭스 작성 시간은?' → 답변: '초기 작성 1주(역할 4 × Tool 5 = 20셀). 사내 표준화 1개월. 분기 재검토 2~4시간. 사고 시 즉시 업데이트. 첫 1주가 본격 작업이에요. 사내 시니어 + IT 부서가 협업합니다.'

TIP

강사 함정 — 매트릭스를 '완벽 압박' X. '기본 + 조정 + 진화' 메시지. 처음부터 완벽한 매트릭스 X.

TIP

강사 진행 팁 — 4분 시간 배분: 0~1분 4역할 + 5 Tools. 1~2분 매트릭스 디테일. 2~3분 실제 구현 방법. 3~4분 청중 질문 1~2개. 매트릭스 표를 슬라이드 또는 칠판에 명확히 보여주세요.

BRIDGE

“권한 매트릭스를 봤다면 — 1강의 마지막. 실습으로 우리 회사 settings.json 표준 1페이지를 작성합니다.”

WORKSHOP 1

실습 — 우리 회사 settings.json 표준

■ 적용 (워크시트)

- Q1 · 단계 — 최소 안전(1주) / 일반(1~3개월) / 본격?
- Q2 · 기본 deny 5종 — rm·sudo·env·secrets·외부 fetch 적용?
- Q3 · 자주 allow — git status·npm test·기타 추가?
- Q4 · 역할 매트릭스 — 4역할 × Tool 매트릭스 시작?
- Q5 · git 공유 — .claude/settings.json + CLAUDE.md 결합?
- ★ 다음 액션 — 시니어 1명이 1주 안에 작성 + 사내 공유

SAY

“1강 의 마지막 — 실습입니다. 60분 동안 들은 내용을 한 장에 응축해서 ‘우리 회사 settings.json 표준 1페이지’를 작성합니다. 사내 IT에게 ‘이렇게 작성해 주세요’ 발주서가 되는 결과물이예요.”

DO

조별 실습 진행. 3~4인 1조로 약 5분간 5칸을 채웁니다. Q1~Q5 + 다음 액션. 조 안에서 부서장·IT·일반 직원 역할 분담 자연스럽게. 강사는 조별로 돌며 ‘어디 막혔나’ 확인하고 가이드해주세요.

SAY

“권장 시작 패턴을 다시 — 최소 안전 패턴 + 기본 deny 5종 + Read·git status·npm test 정도만 allow + 역할 2가지(신입·시니어) 시작 + .claude/settings.json git 공유. 1주 작성 작업이예요. 90% 한국 회사가 이 시작 패턴이 정답입니다.”

Q

발표 요청 1: ‘1~2 조 발표해주세요. 1단계는 무엇으로 선택했는지 한 줄로.’ 보통 ‘최소 안전’이 다수. ‘좋습니다. 첫 도입에 가장 안전한 선택입니다’ 강사가 확인.

Q

발표 요청 2: “‘본격 자율’ 선택한 조가 있나요?’ → 보통 1~2조. ‘좋습니다. 이미 6개월+ 운영 중이거나 시니어 충분 + 사내 IT 강한 회사예요. 단 본 강좌 6편 2강(Hooks)·3강(Skills) 진행 후 본격 자율로 가는 게 정석입니다.’ 강사가 단계 진화 메시지 강조.

Q

발표 요청 3: ‘역할 매트릭스 시작했나요?’ → 보통 절반. ‘좋습니다. 시작 안 한 조는 2×3 매트릭스 = 6개 셀로 작게 시작 권장. 회사 커지면 4×5로 확장.’

NOTE

★ 강사 핵심 배경지식 — 결정서 5칸 작성 가이드: 청중이 막힐 수 있는 부분을 정리하면 — (1) Q1 단계: 첫 도입은 ‘최소 안전’ 1주. 단계 진화의 시간 감각 강조. (2) Q2 deny: 5종은 기본. 우리 회사 추가할 deny가 있다면 함께 정리(예: 사내 DB 접근 등). (3) Q3 allow: 자주 쓰는 안전 작업만 신중하게. ‘귀찮다고 모두 allow’ 함정 경고. (4) Q4 매트릭스: 작게 시작(2×3 또는 3×4) 후 확장. (5) Q5 git 공유: 프로젝트 레벨 + CLAUDE.md 결합이 정석. .gitignore도 함께 작성.

NOTE

강사 배경지식 — 다음 주 액션 권장 3가지: 1주 안에 시작할 수 있는 구체 액션 — (1) 시니어 1명이 1주 안에 첫 settings.json 작성. 약 4~8시간 작업. (2) 팀 PR + 시니어 리뷰. 1~2일. (3) git 커밋 + 사내 표준화 + CLAUDE.md 업데이트. 1일. 총 1주 안에 ‘우리 회사 settings.json 표준 v1’이 손에 납니다. 청중에게 ‘이번 주 안 가능한 일정’ 인지시키세요.

NOTE

강사 배경지식 — 실습 ‘완벽 결정 압박’ X: 5분 안에 5칸을 채우는 게 목표. 완벽한 답이 아닙니다. ‘60% 답으로 시작 → 1주 IT 작성 → 운영 후 보완’이라는 1편부터 이어진 메시지를 다시 강조. 청중이 ‘아 우리도 시작 가능 하구나’ 느끼게 하는 게 1강의 결말입니다.

Q

예상 청중 질문 1: ‘우리 회사 결정 못 할 것 같은데?’ → 답변: “최소 안전”이 안전한 기본값. 첫 도입은 모두 최소 안전부터 시작 권장. 모호하면 그것으로 시작 + 1주 운영 후 다음 단계 결정. 결정 못하는 것이 결정 잘못보다 안전합니다.’

Q

예상 청중 질문 2: ‘다음 주 IT가 작성할 시간이 없다면?’ → 답변: ‘외주 옵션 — 약 200~500만 원에 ‘우리 회사 settings.json + 첫 Hooks 5개’ 패키지를 받을 수 있어요. 단 6개월 운영 후엔 사내 IT가 직접 유지하는 게 정석. 사내 학습 가치도 큼.’

Q

예상 청중 질문 3: ‘다른 회사 settings.json을 참고하면 안 되나요?’ → 답변: ‘참고는 OK. GitHub에서 ‘claude/settings.json’ 검색하면 공개 사례 다수. Anthropic 공식 예시도 docs에 있습니다. 단 ‘우리 회사 정책에 맞게 조정’ 필수. 그대로 복사하면 우리 회사 문화·보안과 안 맞아요.’

TIP

강사 함정 — ‘완벽 결정 압박’ X. 5분 안에 6칸 채우는 게 목표. ‘60% 답으로 시작’ 메시지 다시.

TIP

강사 진행 팁 — 8분 시간 배분: 0~5분 개별·조별 작성. 5~7분 1~2조 발표. 7~8분 ‘다음 주 액션 3가지’ 정리 + 마무리. 5분 작성 시간 충분히. 강사는 작성 중 조별로 돌며 막힌 부분 가이드.

BRIDGE

“1강 끝났습니다. 잠깐 쉬고 2강 ‘Hooks’에서 만나요. settings.json이 ‘정적 권한’이었다면 Hooks는 ‘동적 자동’입니다. 작업 전후에 자동 검증·로그·차단을 끼워 넣는 본격 메커니즘이예요.”

INTRO · TITLE

Hooks — 정책 자동 적용

2강 — PreToolUse · PostToolUse · 작성 패턴

🕒 3분 · 시작

SAY

“2강 ‘Hooks’를 시작합니다. 6편의 본격 핵심이예요. 1강 settings.json이 ‘무엇을 할 수 있나, 무엇을 할 수 없나’를 정하는 정적 권한이었다면, Hooks는 ‘무엇을 할 때 자동으로 무엇이 실행되는가’를 정의하는 동적 메커니즘입니다. 회사 정책을 ‘인간 검증’이 아닌 ‘시스템 자동’으로 강제하는 가장 강력한 도구예요.”

SAY

“2강 60분 동안 다섯 가지를 봅니다. 첫째 — Hooks의 본질과 8가지 이벤트. 둘째 — PreToolUse 사전 검증으로 사고를 사전에 차단하는 패턴. 셋째 — PostToolUse 사후 처리로 알림·로그·테스트를 자동화하는 패턴. 넷째 — 추가 이벤트(UserPromptSubmit·SessionStart 등)의 활용. 다섯째 — 한국 회사가 실제 도입한 Hooks 5종 사례와 실습. 결과 — ‘우리 회사 첫 Hook 설계서 1페이지’예요.”

SAY

“2강의 약속 — Hooks는 ‘본격 코딩’입니다. 사내 IT가 Bash나 Python으로 함수를 작성합니다. 단 본 강좌는 ‘초보자 톤’을 유지해요. 청중이 ‘우리 IT가 1주~1개월 안 작성할 수 있구나’ 시간 감각만 가지면 됩니다. 코드 자체는 청중이 외울 필요 없어요. ‘어떤 Hook을 추가할지 결정’이 청중의 역할입니다.”

Q

도입 질문 1: ‘5편에서 Hooks라는 단어를 들어본 분?’ — 보통 다수가 손을 듭니다. 5편 2강에서 Claude Code의 4가지 기능 중 하나로 Hooks를 언급했어요. ‘좋습니다. 5편이 ‘Hooks가 있다’는 개념 소개였다면 6편이 ‘본격 작성’입니다.’ 강사가 연결합니다.

Q

도입 질문 2: ‘우리 회사에 ‘민감 데이터 외부 송신 금지’ 정책 있는 분?’ — 보통 다수. ‘좋습니다. 그 정책을 매뉴얼로만 두면 직원 실수로 위반됩니다. Hooks가 그걸 ‘자동 감지 + 차단’으로 바꿔줘요. 2강 본문의 핵심 메시지입니다.’

NOTE

★ 강사 핵심 배경지식 — Hooks의 본질: Hooks는 ‘회사 정책을 코드화하는 가장 강력한 도구’입니다. settings.json은 ‘일정한 규칙’만 표현할 수 있어요 — Bash(rm -rf *)를 차단하는 식. 하지만 ‘민감 데이터가 포함된 파일 수정 시 차단’ 같은 ‘동적 판단’이 필요한 정책은 settings.json만으로는 불가능합니다. Hooks가 그 ‘동적 판단’을 가능하게 해요. PreToolUse Hook이 ‘이 파일에 민감 키워드가 있나 검사 → 있으면 차단’ 같은 로직을 실행합니다. settings.json + Hooks 결합이 ‘완전한 회사 정책 코드화’예요.

NOTE

강사 배경지식 — 2강 60분 시간 배분 권장: 0~3분 인사·도입 질문. 3~6분 학습 목표. 6~14분 Hooks의 본질과 8가지 이벤트(15번 슬라이드). 14~22분 PreToolUse 디테일(16번). 22~30분 PostToolUse(17번). 30~38분 추가 이벤트(18번). 38~46분 작성 패턴 Bash·Python(19번). 46~52분 한국 회사 사례 5종(20번). 52~58분 실습(21번). 58~60분 마무리(22번). 시간 압박 있지만 ‘각 이벤트의 핵심 + 실용 가치’ 메시지가 분명하면 충분합니다.

TIP

강사 함정 — Hooks 코드를 자세히 가르치지 마세요. ‘8가지 이벤트가 있다 + 각 이벤트의 활용 가치 + 사내 IT가 작성한다’ 메시지만. JSON 입력 형식·종료 코드 같은 디테일은 IT부의 일이에요. 청중이 ‘우리 회사 어떤 Hook이 필요한지’ 결정할 수 있으면 2강의 목표는 달성됩니다.

BRIDGE

“그럼 학습 목표 빠르게 보고 본문 들어갑니다.”

OBJECTIVES

60분 후, 이 3가지를 한다

- ① Hooks 8가지 이벤트의 활용 가치를 안다
- ② PreToolUse·PostToolUse 작성 패턴을 설명한다
- ③ ‘우리 회사 첫 Hook 설계서 1페이지’를 완성한다

SAY

“2강 60분이 끝나면 세 가지를 할 수 있게 됩니다. 첫째 — Hooks의 8가지 이벤트 — PreToolUse·PostToolUse·UserPromptSubmit·SessionStart·SessionEnd·Stop·Notification·PreCompact — 각 이벤트가 ‘무엇에 활용 가치가 있는지’ 설명할 수 있게 됩니다. 외울 필요는 없어요. ‘이런 이벤트가 있고, 이런 정책 자동화에 쓸 수 있구나’ 감각이 핵심.”

SAY

“둘째 — PreToolUse와 PostToolUse 두 핵심 이벤트의 작성 패턴을 알게 됩니다. ‘이런 흐름으로 작성하는구나’ 정도. 직접 코딩은 사내 IT의 일이고, 청중은 ‘우리 회사가 어떤 Hook을 추가할지’ 결정만 합니다.”

SAY

“셋째 — 가장 중요한 결과물 — ‘우리 회사 첫 Hook 설계서 1페이지’를 완성합니다. 5칸의 워크시트에 ‘어떤 Hook을 첫 도입할지’ 답을 채워요. 사내 IT에게 ‘이 Hook을 1주 안에 작성해 주세요’ 발주서가 됩니다.”

SAY

“2강 핵심 메시지를 미리 — ‘첫 Hook 1개를 신중하게 작성 + 1주 운영 + 사고 0 확인 + 점진 확장.’ 처음부터 10개 Hooks를 한꺼번에 작성하는 함정이 있어요. Hook 자체에 버그가 있으면 모든 작업이 영향을 받습니다. 1개씩 작성·검증·운영하는 게 정석입니다.”

NOTE

★ 강사 핵심 배경지식 — 2강의 성공 기준: 청중이 ‘우리 회사 첫 Hook 후보 1개’를 자신 있게 선택할 수 있어야 합니다. 본 강좌가 권장하는 첫 Hook은 ‘.env·secrets 파일 Edit 차단’ Hook — 가장 단순하고 가치 큰 안전망이에요. 청중이 그것 또는 ‘우리 회사 특수 정책 1개’를 결정하면 성공입니다.

NOTE

강사 배경지식 — Hooks vs settings.json의 역할 분담: settings.json은 ‘정적 규칙 + 빠른 적용’입니다. Bash(rm -rf *) 차단처럼 ‘명령 패턴’만 매칭하면 되는 경우. Hooks는 ‘동적 판단 + 복잡 로직’이에요. ‘이 파일에 민감 키워드 있나 검사’처럼 코드 실행이 필요한 경우. 두 가지를 잘 분담하면 회사 정책의 95%를 코드화할 수 있어요. 본 강좌의 2강이 그 ‘5%의 동적 로직 자동화’를 다룹니다.

TIP

강사 진행 팁 — 학습 목표 3분 안에. 길게 가지 마세요. 본문이 더 중요합니다.

“그럼 본문 시작 — 첫 번째로 Hooks의 본질과 8가지 이벤트입니다.”

WHAT IS HOOKS

Hooks — 동작 전후 자동 함수

■ 불변층 — 8가지+ 이벤트

- 수동 검증 + 직원 실수 (Hooks 없이) — 직원이 ‘민감 데이터 송신 시도’ → 사후 발견 → 처벌 + 사고 보고. 시간 비용 큼. 같은 실수가 다른 직원에게서 반복.
- 자동 검증 + 시스템 차단 (Hooks 사용) — PreToolUse Hook이 ‘민감 데이터 감지’ → 차단 → 사용자에게 ‘이런 이유로 차단’ 알림. 사전 방지. 직원 실수 가능성 0.

🕒 6분 · Hooks 풀이

SAY

“Hooks의 본질을 풀이합니다. Hooks는 Claude Code의 동작 흐름 — ‘사용자 입력 → 도구 호출 → 응답’ — 사이사이에 자동 함수를 끼워 넣는 메커니즘이에요. ‘동작 전후에 자동으로 무엇을 실행’할지 정의합니다.”

SAY

“8가지 이벤트가 있어요. 핵심 두 가지 — PreToolUse(도구 실행 직전)와 PostToolUse(도구 실행 직후). 추가 — UserPromptSubmit(사용자 입력 시), SessionStart(세션 시작), SessionEnd(세션 끝), Stop(에이전트 정지), Notification(사용자 알림), PreCompact(컨텍스트 압축 전). 이 8가지가 Claude Code의 모든 ‘흐름의 분기점’이에요.”

SAY

“PreToolUse 활용 예 — ‘민감 데이터 키워드 감지 → Edit 차단’. 직원이 Claude Code에게 ‘.env 파일 수정 해줘’ 요청 → PreToolUse Hook 자동 실행 → ‘이 파일은 회사 보안 정책으로 보호된 파일입니다. Edit 차단’ 메시지 + 작업 중단. 사고 사전 방지의 가장 강력한 패턴이에요.”

SAY

“PostToolUse 활용 예 — ‘git commit 후 자동 슬랙 알림 + CI 트리거’. 직원이 작업 끝나고 git commit을 실행 → PostToolUse Hook 자동 실행 → 슬랙 #ai-changes 채널에 ‘이 작업이 끝났습니다’ 알림 + CI 시스템에 ‘새 커밋을 빌드해 주세요’ 트리거. 사후 자동화의 대표 패턴.”

SAY

“Hooks의 본질 원칙 — ‘인간 검증’이 아닌 ‘코드 검증’. 직원이 매뉴얼을 기억해서 정책을 지키는 게 아니라, 시스템이 자동으로 정책을 적용합니다. 1편의 5단계 성숙도 모델에서 L4·L5 자동화가 이 패턴이에요.”

STORY

출처 — Anthropic Claude Code 공식 문서. docs.claude.com/claude-code/hooks 페이지에 8가지 이벤트가 모두 정의돼 있어요. ‘각 이벤트는 JSON 입력을 받고, 종료 코드(0=OK, 2=차단)와 JSON 응답을 돌려준다’는 사양도 명시돼 있습니다. Anthropic은 Hooks를 ‘policy automation의 가장 강력한 도구’로 위치시켰어요.

STORY

Hooks 동작 흐름 디테일 (강사 대응용). 한 작업이 어떻게 흘러가는지 — (1) 사용자가 Claude Code에 ‘이 파일 수정해줘’ 요청. (2) Claude Code가 PreToolUse Hook이 등록돼 있는지 확인. (3) 등록된 Hook 함수에 JSON 입력 — {tool: ‘Edit’, file_path: ‘...’, ...} — 전달. (4) Hook 함수가 Bash 또는 Python으로 검증 로직 실행. (5) 종료 코드 0 반환 → Claude Code가 Edit 실행. 종료 코드 2 반환 → Claude Code가 차단 + ‘차단 이유’ 메시지를 에이전트에게 전달. (6) Edit 실행 후 PostToolUse Hook이 또 실행되며 결과 처리. 이 흐름이 모든 Hooks의 본질입니다.

STORY

Anthropic 사내 Hooks 사용 사례 (강사 대응용). ‘How Anthropic teams use Claude Code’ 인용 문구 — ‘every commit gets Hook-checked’. 사내 모든 코드 커밋에 PostToolUse Hook이 자동 검증을 수행한다는 뜻이에요. Hook이 (1) 커밋 메시지 형식 검증 (2) 자동 lint 실행 (3) 자동 테스트 호출 (4) 슬랙 알림을 일괄 처리. 사내 ‘직원 실수 0’ 정책의 핵심이 Hooks예요. 한국 회사가 따라가야 할 표준 패턴이기도 합니다.

NOTE

★ 강사 핵심 배경지식 — Hooks의 4대 가치: Hooks가 회사 운영에 가져다주는 가치 4가지를 정리하면 — (1) 사고 방지. 민감 데이터·위험 명령을 사전에 차단. ‘인간 실수 가능성 0’. (2) 자동화. 슬랙 알림·CI 트리거·자동 테스트를 일괄 처리. ‘반복 작업 시간 0’. (3) 정책 적용. 회사 표준이 자동으로 강제. ‘매뉴얼 학습 부담 0’. (4) 감사 로그. 모든 동작이 자동 기록되어 사고 시 추적 가능. 4대 가치가 결합되어 ‘안전한 자율 운영’이 가능해집니다.

NOTE

강사 배경지식 — Hooks 작성 언어: Hook 함수는 Bash 또는 Python으로 작성합니다. 단순 검증은 Bash 5~10줄, 복잡 로직은 Python 50~100줄 정도. settings.json의 hooks 키에 명령으로 등록 — ‘bash /path/to/script.sh’ 또는 ‘python /path/to/check.py’. 사내 IT가 1~2주 학습 후 작성 가능한 수준이에요. 전문 보안 지식 X 일반 개발 지식이면 충분합니다.

NOTE

강사 배경지식 — Hooks의 ‘부담 함정’: Hook 함수 자체가 모든 작업마다 실행됩니다. 만약 Hook이 느리면 → 모든 작업이 느려집니다. ‘100ms 이상 지연 X’가 권장 원칙이에요. 무거운 검증은 (1) 캐싱(같은 파일 결과 재사용) (2) 비동기 처리(PostToolUse 큐) (3) 외부 시스템 호출 최소화로 대응. 본 강좌 11편이 자세히 다룹니다.

NOTE

강사 배경지식 — Hooks ‘무한 루프’ 위험: Hook 함수가 다른 도구를 호출하면 그 도구가 다시 Hook을 트리거할 수 있어요. 예 — PostToolUse Hook이 ‘git log를 슬랙에 전송’하는데 그 git log 자체가 Bash 도구 호출이면 또 PreToolUse Hook이 실행 → 또 git log → 무한 루프. 사내 IT가 Hook 작성 시 ‘이 함수가 어떤 도구를 호출하나’ 명시적으로 검토 + 같은 도구는 Hook 트리거 안 되게 설정 필요. 본 강좌 11편.

Q

예상 청중 질문 1: ‘우리 IT 부서가 Bash·Python으로 Hook 작성 가능한가요?’ → 답변: ‘일반 개발 지식이면 가능합니다. Bash 5~10줄 또는 Python 50~100줄 수준. 사내 IT 시니어가 1~2주 학습 후 첫 Hook 1개 작성. 외주도 가능 — 약 100~300만 원에 ‘첫 Hook 5개 패키지’. 단 사내 학습 후 자체 유지가 정석입니다.’

Q

예상 청중 질문 2: ‘Hook 자체에 버그가 있으면?’ → 답변: ‘큰 문제예요. Hook 버그 = 모든 Claude Code 작업이 영향. 그래서 (1) 충분한 테스트 — dry run 1주(실제 차단 X, 경고만). (2) 작게 시작 — 첫 Hook 1개만 + 1주 운영 + 안정 확인. (3) Hook 자체에 try-except 보호 — Hook 실패 시 작업은 진행되게. 3단계 대응이 필수입니다.’

Q

예상 청중 질문 3: ‘직원이 Hooks를 우회할 수 있나요?’ → 답변: ‘프로젝트 레벨 settings.json에 Hook 등록 + Enterprise 강제면 X 우회. 사용자 레벨만이면 직원이 자기 settings에서 hooks 비활성화 가능 — 단 사내 정책으로 ‘Hook 비활성화 시 sanctions’ 명시 + 정기 감사로 통제. 본 강좌 9편 자세히.’

TIP

강사 함정 — Hook의 ‘위험 측면’도 균형 있게 전달하세요. ‘Hooks가 좋다’만 강조하면 청중이 무리하게 도입하다 사고. ‘좋지만 신중하게, 작게 시작’ 메시지 균형.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 본질 풀이. 1~2분 8가지 이벤트 한눈에. 2~3분 PreToolUse-PostToolUse 활용 예. 3~4분 동작 흐름 + Anthropic 사례. 4~5분 4대 가치 + 부담 함정. 5~6분 청중 질문 1~2개. Anthropic 사내 사례에 시간 충분히.

“Hooks의 본질을 봤다면 — 가장 강력한 이벤트인 PreToolUse를 자세히 봅니다. 사전 검증과 차단이 핵심이
예요.”

PRETOOLUSE

PreToolUse — 사전 검증·차단

■ 불변층 — 안전의 핵심

- ★ 이벤트 — 도구 실행 직전 자동 호출
- ★ 입력 — Tool명 + 인자 (JSON 형식)
- ★ 종료 코드 — 0(OK) / 2(차단) / 기타(에러)
- ★ 활용 1 — 민감 데이터(PII) 감지 + 차단
- ★ 활용 2 — 위험 명령(rm·sudo) 추가 차단
- ★ 활용 3 — 회사 정책(커밋 메시지 형식) 강제

🕒 6분 · PreToolUse

SAY

“PreToolUse Hook을 자세히 봅니다. 사전 검증과 차단이 핵심 이벤트예요. 사고 방지의 가장 강력한 도구입니다.”

SAY

“이벤트가 언제 실행되는지 — 도구 실행 직전입니다. 사용자가 ‘이 파일 수정해줘’ 요청 → Claude Code가 Edit 도구를 실행하려는 순간 → PreToolUse Hook 자동 호출. Hook 함수가 ‘이 작업 안전한가 검증’ → 결과에 따라 작업이 진행되거나 차단됩니다.”

SAY

“입력 데이터는 JSON 형식이에요. ‘tool_name’ + ‘tool_input’ 두 키. tool_input 안에 file_path·old_string·new_string·command 같은 도구 인자가 들어 있어요. Hook 함수가 이 JSON을 받아서 검증을 수행합니다.”

SAY

“종료 코드로 결과를 표현해요. 0 — OK, 작업 진행. 2 — 차단, 작업 중단. 기타(1, 3 등) — 에러, Hook 자체 문제 의미. 또 JSON 응답으로 ‘차단 이유’를 명시할 수 있어요 — ‘이 파일은 보안 보호’ 같은 메시지. 에이전트가 이걸 보고 다른 접근을 시도할 수 있습니다.”

SAY

“활용 1 — 민감 데이터 감지. PII(개인 식별 정보) 정규식으로 ‘이메일·전화·주민번호·신용카드’ 패턴을 감지. 발견 시 차단. 우리 회사 사내 PII 보호의 핵심이에요. 한국 개인정보보호법 준수에도 필요합니다.”

SAY

“활용 2 — 위험 명령 차단. settings.json deny와 보완합니다. settings.json은 단순 패턴 매칭이지만, Hook은 동적 판단이 가능해요. 예 — ‘이 rm 명령의 대상 폴더가 백업이 없는 영구 폴더인가’ 검사 → 백업 없으면 차단. 더 지능적인 차단이 가능합니다.”

SAY

“활용 3 — 회사 정책 강제. 예 — git commit 메시지 형식 검증. ‘feat·fix·docs·refactor’ 같은 접두가 없으면 차단. 사내 표준 커밋 메시지 형식이 자동으로 강제됩니다. 직원이 매뉴얼 기억할 필요 X.”

STORY

PreToolUse 입력 JSON 예시 (강사 대응용). 직원이 ‘.env 파일 수정해줘’ 요청했을 때 Hook이 받는 JSON을 보여드리면 — { ‘tool_name’: ‘Edit’, ‘tool_input’: { ‘file_path’: ‘/home/user/project/.env’, ‘old_string’: ‘API_KEY=xxx’, ‘new_string’: ‘API_KEY=yyy’ } }. Hook 함수가 file_path를 보고 ‘.env’가 포함됐는지 검사 → 포함됐으면 종료 코드 2 + JSON 응답 { “reason”: “환경 파일은 사내 비밀 — Edit 차단” } 반환. 에이전트가 이걸 보고 ‘.env 직접 수정 대신 .env.example 검토’ 같은 다른 접근으로 자동 전환할 수 있어요.

STORY

한국 회사 PreToolUse 사례 (강사 대응용). 일부 한국 IT 회사가 도입한 PreToolUse Hooks를 정리하면 — (1) 주민번호 정규식 매치 차단. KoNLPy + 정규식 결합. 한국어 PII 처리에 효과적. (2) rm·sudo 명령 추가 차단. settings.json deny와 보완. (3) .env·secrets/ 폴더 보호. (4) git commit 메시지 ‘feat/fix/docs/refactor’ 접두 강제. 네 가지가 ‘첫 도입 1개월 안 작성하는 표준 4종’이에요.

STORY

PreToolUse 응답 JSON 디테일 (강사 대응용). Hook이 응답으로 더 풍부한 정보를 반환할 수 있어요. 종료 코드 0(OK) + `{ "continue": true, "systemMessage": "검증 통과" }` JSON. 종료 코드 2(차단) + `{ "reason": "PII 감지", "suggestion": ".env.example 검토 권장" }` JSON. 에이전트가 ‘차단 이유 + 추천 대안’을 이해하고 자동으로 더 안전한 접근을 시도. ‘인간 검증 X 시스템 검증’의 본격 구현이에요.

NOTE

★ 강사 핵심 배경지식 — PreToolUse 작성의 5단계: 첫 PreToolUse Hook을 작성할 때 따라야 할 단계를 정리하면 — (1) 보호 대상 정의. ‘우리는 무엇을 보호하나’ — PII·secrets·민감 파일·위험 명령 중에서 우선순위 결정. (2) 감지 로직 설계. 정규식·키워드 리스트·DB 조회·외부 API 호출 중 어떻게 감지할지. (3) 차단 또는 경고 결정. 첫 도입 1주는 경고만(dry run), 안정 후 차단. (4) 메시지 작성. 에이전트가 ‘차단 이유 + 대안’을 이해할 수 있게 명확하게. (5) 테스트 + 점진 운영. dry run 1주 + 사용자 피드백 + 본격. 총 1~2주 작업이에요.

NOTE

강사 배경지식 — PreToolUse의 ‘성능 함정’: PreToolUse는 매 도구 호출마다 실행돼요. 만약 Hook이 100ms 걸리면 → 한 세션에 100번 도구 호출 → 10초 추가 지연. 사용자 답답함. 권장 — (1) 캐시(같은 파일 결과 재사용) (2) 빠른 정규식만 사용 (3) 외부 API 호출 X 또는 비동기. 본 강좌 11편이 자세히 다룹니다.

NOTE

강사 배경지식 — PII 감지 도구: 한국 PII 감지에 추천하는 도구 — (1) Microsoft Presidio. 다국어 PII 감지 라이브러리. 영어·한국어 일부 지원. 무료 오픈소스. (2) KoNLPy. 한국어 자연어 처리. 정규식과 결합해 ‘한국 이름·주민번호’ 감지에 특화. (3) 자체 정규식. ‘주민번호 6자리-7자리’, ‘010-0000-0000 전화’ 같은 단순 패턴. 빠르고 가벼움. 한국 회사는 ‘자체 정규식 + KoNLPy’ 조합이 가장 인기예요.

NOTE

강사 배경지식 — Hook 사고 시 ‘fail-open vs fail-close’ 선택: Hook 자체가 실패하면 어떻게 처리할까. Fail-open(작업은 진행) — 사용자 답답함 X, 단 사고 가능성. Fail-close(작업 차단) — 안전 우선, 단 Hook 버그 시 모든 작업 중단. 본 강좌 권장 — ‘저위험 검증은 fail-open, 고위험 검증은 fail-close’. 예 — 알림 Hook 실패는 fail-open(알림 못 가도 작업 진행), PII 감지 Hook 실패는 fail-close(PII 못 검증하면 안전을 위해 차단). 본 강좌 9·11편.

Q

예상 청중 질문 1: ‘첫 PreToolUse Hook은 무엇으로 시작?’ → 답변: “.env·secrets/ 파일 Edit 차단”이 가장 단순 + 가치 큰 첫 Hook입니다. Bash 5줄 + 1시간 작성. dry run 1주 + 본격 적용. 사고 가장 흔한 패턴(비밀 정보 노출)을 자동 차단해요.’

Q

예상 청중 질문 2: ‘에이전트가 Hook 차단 이유를 받으면 어떻게 반응?’ → 답변: ‘에이전트가 차단 이유 메시지를 읽고 ‘다른 접근’을 자동 시도합니다. 예 — ‘.env 직접 수정 차단’이면 ‘아 이 파일은 보호되네, 대신 .env.example 검토하거나 사용자에게 secrets manager 사용 권장하자’ 같은 자동 전환. 단순 차단이 아닌 ‘안전 가이드’로 작동해요.’

Q

예상 청중 질문 3: ‘false positive(정상 작업이 잘못 차단) 우려?’ → 답변: ‘있습니다. dry run 1주 + 사용자 피드백 수집 + 조정이 필수예요. 첫 도입은 ‘약간 엄격’으로 시작 → 1~3개월 운영 후 ‘false positive 패턴’ 발견 되면 정규식 조정. 본 강좌 11편 자세히.’

TIP

강사 함정 — JSON 입력 구조의 디테일 깊이 X. ‘JSON 받음, 검증 로직 작성, 종료 코드 반환’ 3단계 메시지만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 이벤트 + 입력. 1~3분 종료 코드 + 3가지 활용. 3~4분 입력 JSON 예시 + 응답 JSON. 4~5분 작성 5단계 + 성능 함정. 5~6분 청중 질문 1~2개.

BRIDGE

“PreToolUse를 봤다면 — 다음은 PostToolUse입니다. 사후 처리와 자동화의 대표 이벤트예요.”

POSTTOOLUSE

PostToolUse — 사후 처리·자동화

■ 불변층 — 자동화 핵심

- ★ 이벤트 — 도구 실행 직후 자동 호출
- ★ 입력 — Tool명 + 인자 + 실행 결과 (JSON)
- ★ 활용 1 — 슬랙 알림 (중요 변경 시)
- ★ 활용 2 — 자동 테스트 (Edit 후 npm test)
- ★ 활용 3 — 감사 로그 (모든 변경 사내 DB 기록)
- ★ 활용 4 — CI 트리거 (git push 후 자동 빌드)

SAY

“PostToolUse Hook을 봅니다. 사후 처리와 자동화의 핵심 이벤트예요. PreToolUse가 ‘사고 방지’였다면 PostToolUse는 ‘업무 자동화’입니다.”

SAY

“이벤트가 언제 실행되는지 — 도구 실행 직후입니다. Claude Code가 Edit 또는 Bash 명령을 실행하고 결과가 나온 순간 → PostToolUse Hook 자동 호출. 결과를 처리하거나, 알림을 보내거나, 다음 작업을 트리거할 수 있어요.”

SAY

“입력 데이터에 PreToolUse와 다른 점은 ‘tool_result’가 추가된다는 거예요. JSON으로 ‘작업이 성공했나, 어떤 결과가 나왔나’ 정보가 함께 전달됩니다. Hook이 이걸 보고 ‘결과에 따라 다른 후처리’를 결정할 수 있어요.”

SAY

“활용 1 — 슬랙 알림. ‘Edit·git commit 후 슬랙 #ai-changes 채널에 자동 알림’. 팀원이 ‘아 누가 어디서 무엇을 수정했네’ 즉시 인지. 사내 협업의 가시성 + 사고 추적성이 크게 향상돼요.”

SAY

“활용 2 — 자동 테스트. ‘src/ 폴더 Edit 후 npm test 자동 실행’. 코드 수정 후 ‘이 변경이 테스트를 깨뜨렸나’ 즉시 확인. 사고를 시간 안에 발견할 수 있어요. CI 시스템의 사내 미니 버전이에요.”

SAY

“활용 3 — 감사 로그. ‘모든 변경을 사내 DB에 기록 — 누가·언제·무엇·결과’. 본 강좌 2편 3강의 ELK Stack 과 결합. 사고 발생 시 추적 가능. 한국 개인정보보호법 준수에도 필요해요.”

SAY

“활용 4 — CI 트리거. ‘git push 후 자동 빌드·배포 트리거’. 4편의 MCP와 결합해서 Jenkins·GitHub Actions 같은 CI 시스템을 호출할 수 있어요. 본격 DevOps 자동화의 시작점이에요.”

STORY

PostToolUse 입력 JSON 예시 (강사 대응용). 직원이 git commit 명령을 실행했을 때 Hook이 받는 JSON을 보여드리면 — { 'tool_name': 'Bash', 'tool_input': { 'command': 'git commit -m "fix: API 에러 처리"' }, 'tool_result': { 'success': true, 'stdout': '[main abc123] fix: API 에러 처리\n 1 file changed', 'stderr': '', 'exit_code': 0 } }. Hook 함수가 이걸 보고 (a) 커밋이 성공했네 (b) 어떤 파일이 바뀌었네 (c) 커밋 메시지가 'fix:' 접두로 시작했네 정보를 추출하고, 슬랙 알림 + CI 트리거 같은 후속 작업을 자동으로 수행합니다.

STORY

한국 회사 PostToolUse 사례 (강사 대응용). 일부 한국 IT 회사가 도입한 PostToolUse Hooks 4종 — (1) git commit 후 슬랙 #ai-changes 알림. 팀 가시성 ↑. (2) src/ 수정 후 자동 ESLint 실행. 코드 품질 자동 유지. (3) 사내 DB 변경 후 자동 백업 스냅샷. 사고 대비. (4) 배포 명령 후 모니터링 시스템 알림. 사후 추적성 ↑. 네 가지가 '첫 도입 1개월 안 작성하는 표준 4종'이에요. PreToolUse 4종 + PostToolUse 4종 = 총 8개 Hooks가 한국 회사 표준 시작점입니다.

STORY

PostToolUse 비동기 패턴 (강사 대응용). PostToolUse Hook이 무거운 작업(예: 슬랙 알림·외부 API 호출·DB 쓰기)을 동기로 실행하면 작업 지연이 큼니다. 권장 패턴 — Hook 자체는 빠르게 종료 + 무거운 작업은 백그라운드 비동기로. 예 — Hook에서 'nohup bash background_task.sh &' 같은 명령으로 백그라운드 실행 + Hook은 즉시 종료. 사용자가 답답함을 느끼지 않으면서도 자동화가 일어납니다. 본 강좌 11편 자세히.

NOTE

★ 강사 핵심 배경지식 — PostToolUse의 '5가지 권장 우선순위': 첫 도입 시 작성 우선순위를 정리하면 — (1) 슬랙 알림(중요 변경만). 가장 가치 큰 첫 PostToolUse. 1~2시간 작성. (2) 자동 테스트(코드 변경 시). 코드 품질 자동. 1일 작성. (3) 감사 로그(모든 변경). 사고 추적. 1주 작성. (4) CI 트리거(git push 후). DevOps 자동. 1~2주. (5) 비동기 큐(무거운 작업). 본격 운영. 1개월. 5가지를 단계별 도입 — 첫 도입 시점은 (1)부터, 6개월 후 (5)까지. 점진 확장.

NOTE

강사 배경지식 — 슬랙 알림의 '소음 함정': 모든 변경에 슬랙 알림을 보내면 → 채널이 시끄러워서 직원이 무시 → 알림 의미 X. 권장 패턴 — (1) 중요 변경만 알림. 'src/ 변경만, test/ 변경 X' 같은 필터. (2) 시간대 제한. '업무 시간 외 알림 X 또는 메일'. (3) 채널 분리. '#ai-changes-prod' vs '#ai-changes-dev'. (4) Digest 모드. '1시간 모아서 1번 알림'. 4가지 조합으로 소음을 줄입니다. 본 강좌 11편.

NOTE

강사 배경지식 — 감사 로그의 ‘위·변조 방지’: 본 강좌 2편 3강에서 다룬 패턴이지만 다시 — (1) 별도 로그 서버. PostToolUse Hook이 ‘사내 로그 서버 API’ 호출 + 로그는 별도 서버에 저장. (2) 로그 서명. 각 로그 항목에 시간·해시 서명 — 위변조 시 감지 가능. (3) 로그 백업. 다른 위치에 복제 — 침입자가 한 곳을 지워도 추적 가능. 본 강좌 9·10편이 자세히 다룹니다.

Q

예상 청중 질문 1: ‘첫 PostToolUse Hook은 무엇으로 시작?’ → 답변: “git commit 후 슬랙 알림”이 가장 단순 + 가치 큰 첫 PostToolUse입니다. Bash 10줄 + 1~2시간 작성. 즉시 팀 가시성이 향상돼요.’

Q

예상 청중 질문 2: ‘PostToolUse가 작업 지연을 일으키지 않나요?’ → 답변: ‘동기 실행이면 지연 큼. 비동기 패턴 — Hook 자체 빠르게 종료 + 무거운 작업 백그라운드. ‘nohup background_task.sh &’ 같은 패턴. 사용자가 답답함 X 자동화도 수행. 본 강좌 11편 자세히.’

Q

예상 청중 질문 3: ‘모든 변경을 DB 로그?’ → 답변: ‘권장. ELK Stack 또는 사내 DB에 자동 기록. 누가·언제·무엇·결과 4가지 필드. 보관 1년+(한국 법규). 사고 추적 + ROI 측정 자료가 됩니다. 본 강좌 9편이 자세히.’

TIP

강사 함정 — Hook 코드의 디테일 깊이 X. ‘5가지 활용 + 4가지 패턴’ 메시지만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 이벤트 + 입력. 1~3분 4가지 활용. 3~4분 입력 JSON 예시 + 비동기 패턴. 4~5분 5가지 우선순위 + 소음 함정. 5~6분 청중 질문 1~2개. 슬랙 알림 ROI에 시간 충분히.

BRIDGE

“PostToolUse를 봤다면 — 추가 이벤트 6가지를 빠르게 봅니다. UserPromptSubmit·SessionStart 같은 특수 이벤트의 활용 가치예요.”

OTHER EVENTS

추가 이벤트 — UserPromptSubmit · Stop · 기타

■ 불변층 — 6가지 추가 이벤트

- ★ UserPromptSubmit — 사용자 입력 시 (정책 안내·검증)
- ★ SessionStart — 세션 시작 시 (환영·안내)
- ★ SessionEnd — 세션 끝 시 (요약·KPI 저장)
- ★ Stop — 에이전트 정지 시 (완료 알림)
- ★ Notification — 알림 시 (슬랙 미러링)
- ★ PreCompact — 컨텍스트 압축 전 (중요 정보 저장)

🕒 5분 · 추가 이벤트

SAY

“PreToolUse·PostToolUse 외 추가 이벤트 6가지를 빠르게 봅니다. 처음 도입 시 다 사용할 필요는 없어요. ‘이런 이벤트가 있고, 이런 활용 가치가 있다’ 정도만 인지하면 됩니다.”

SAY

“첫째 — UserPromptSubmit. 사용자가 Claude Code에 명령을 입력하는 순간 자동 호출되는 이벤트예요. 활용 — ‘민감 키워드 사전 검증’ 또는 ‘회사 정책 사전 안내’. 예 — 사용자가 ‘우리 회사 매출 정보 알려’ 입력 → Hook이 ‘민감 정보 — 임원만 접근 가능’ 메시지 추가 + 권한 안 되면 차단.”

SAY

“둘째 — SessionStart. 새 세션이 시작될 때 자동 호출. 활용 — ‘환영 메시지 + 회사 정책 안내’. 예 — 매 세션 시작 시 ‘오늘 사내 정책 변경: ____’ 같은 안내. 직원이 매번 인지.”

SAY

“셋째 — SessionEnd. 세션이 끝날 때 자동. 활용 — ‘세션 요약 저장 + ROI 데이터 기록’. 사내 BI 시스템에 ‘이 직원이 오늘 무엇을 했나’ 자동 저장 → KPI 추적 자료.”

SAY

“넷째 — Stop. 에이전트가 작업을 정지할 때. 활용 — ‘완료 알림’ 또는 ‘다음 단계 자동 제안’. 긴 자율 작업 끝났을 때 사용자에게 슬랙 알림.”

SAY

“다섯째 — Notification. Claude Code가 사용자에게 알림을 보낼 때. 활용 — ‘슬랙 미러링’. 사용자가 보는 알림을 슬랙 채널에도 자동 복사. 협업에 유용.”

SAY

“여섯째 — PreCompact. 컨텍스트 압축 전에 호출. 긴 세션에서 Claude Code가 ‘이전 대화 요약하고 메모리 절약’ 시도할 때 Hook이 ‘이 중요 정보는 절대 잊지 마’ 같은 우선순위 정보를 저장. 본격 자율 작업에 의미.”

STORY

UserPromptSubmit 활용 디테일 (강사 대응용). 한 회사가 이렇게 사용했어요 — 사용자가 ‘우리 회사 매출 정보 알려’ 입력 → UserPromptSubmit Hook이 자동 감지 → ‘민감 정보 키워드: 매출, 인사, 재무’ 매칭 → 사용자 권한 조회(LDAP) → 권한 있으면 ‘참고: 답에 PII 포함 가능, 외부 공유 시 주의’ 시스템 메시지 추가. 권한 없으면 ‘이 정보는 임원만 접근. 부서장에게 요청 권장’ 메시지로 안내. 단순 차단이 아닌 ‘교육 + 안내’ 자동화예요.

STORY

SessionStart 활용 디테일 (강사 대응용). 매 세션 시작 시 자동 표시되는 ‘오늘의 정보’ 패턴 — (1) 오늘 사용 가능한 토큰 수. (2) 최근 사내 정책 변경. (3) 권장 작업 또는 새 Skill 공지. (4) 사고 대응 안내(긴급 시). 직원이 매일 자연스럽게 정보를 인지. ‘회사 내부 신문’ 같은 역할이에요.

STORY

PreCompact의 ‘긴 자율 작업’ 의미 (강사 대응용). Claude Sonnet 4.5가 30시간 자율 작업이 가능하다고 5편에서 봤어요. 그 30시간 동안 컨텍스트 윈도우(200K 토큰)가 가득 차면 Claude Code가 자동으로 ‘이전 대화 요약하고 압축’을 시도합니다. PreCompact Hook이 그 순간에 ‘이 중요 데이터는 절대 잊지 마, 별도 저장’ 같은 보호 작업을 수행. 본격 30시간 자율 작업에서 필수가 되는 Hook이에요. 본 강좌 8편(자체 에이전트)에서 자세히 다룹니다.

NOTE

★ 강사 핵심 배경지식 — 추가 이벤트의 ‘활용 가치 정리’: 6가지 이벤트의 활용 가치를 한 줄로 — (1) UserPromptSubmit: 입력 검증·정책 안내. (2) SessionStart: 환영·정보 푸시. (3) SessionEnd: 요약·KPI 저장. (4) Stop: 완료 알림. (5) Notification: 슬랙 미러링. (6) PreCompact: 장기 메모리 보호. 핵심 PreToolUse·PostToolUse 외에 ‘세션·입력 흐름’ 자동화에 추가. 회사 전체 정책 사이클을 코드화하는 게 가능해집니다.

NOTE

강사 배경지식 — 추가 이벤트 도입 순서: 추가 이벤트는 핵심 두 가지 도입 + 6개월 운영 후에 추가하는 게 정석이에요. (1) 우선 PreToolUse + PostToolUse 5개씩 운영 안정. (2) 6개월 후 ‘사용자 거부감 X 인지 부족’ 발견 시 SessionStart·UserPromptSubmit 추가. (3) 1년+ ‘본격 자율 작업’ 도입 시 PreCompact 추가. 점진 확장이 정석. 첫 도입에 6가지 다 작성 X.

Q

예상 청중 질문 1: ‘추가 이벤트 다 도입해야 하나요?’ → 답변: ‘아닙니다. PreToolUse·PostToolUse 우선. 추가 이벤트는 ‘6개월 운영 후 필요 발견’ 시 도입. 첫 도입에 다 작성하면 운영 부담만 큼니다.’

Q

예상 청중 질문 2: ‘UserPromptSubmit이 사용자 입력을 검열하는 건가요?’ → 답변: ‘검열보다 ‘안전 안내’ 패턴이 정석. 사용자가 민감 키워드 입력해도 차단이 아니라 ‘이 정보는 민감 — 외부 공유 주의’ 같은 안내. 권한 부족 시에만 차단. 직원 신뢰 + 안전 균형이에요.’

Q

예상 청중 질문 3: ‘PreCompact는 언제 의미 있나요?’ → 답변: ‘긴 자율 작업(30시간 등)이 본격 도입될 때. 그 전엔 우선순위 낮음. 본 강좌 8편(자체 에이전트)에서 본격적으로 다룹니다.’

TIP

강사 함정 — 추가 이벤트의 디테일 깊이 X. ‘6가지 목록 + 활용 가치’ 정도만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 6가지 한눈에. 1~3분 각 30초씩. 3~4분 도입 순서 + UserPromptSubmit 활용. 4~5분 청중 질문 1~2개.

BRIDGE

“이벤트를 다 봤다면 — Hook 작성 패턴으로 넘어갑니다. 실제 코드는 어떻게 쓰는지 봅시다.”

WRITE PATTERNS

Hook 작성 패턴 — Bash·Python

■ 불변층 — 4단계 작성 흐름

- ★ 1단계 — 이벤트 + 활용 결정
- ★ 2단계 — settings.json에 hook 등록
- ★ 3단계 — Bash·Python 함수 작성
- ★ 4단계 — dry run 1주 + 본격 적용
- ★ 패턴 — 단순은 Bash 5~10줄, 복잡은 Python 50~100줄
- ★ 디버깅 — log 출력 + dry run + 단계 적용

🕒 5분 · 작성 패턴

SAY

“Hook 작성의 실제 4단계 흐름을 봅니다. 사내 IT가 따라가야 하는 순서예요. 청중은 ‘이런 4단계로 작성하는구나’ 시간 감각만 가지면 됩니다.”

SAY

“1단계 — 이벤트 + 활용 결정. ‘우리는 무엇을 자동화·차단할까’를 먼저 결정합니다. 본 강좌 전반부에서 다룬 Hook 후보들 — .env 차단·PII 감지·slack 알림·자동 테스트 등 — 중에서 1순위 선택. 시간 1시간.”

SAY

“2단계 — settings.json에 hook 등록. hooks 키 안에 ‘이벤트 이름 + matcher + 명령’을 작성합니다. ‘PreToolUse 이벤트, Edit 도구에만 적용, bash script.sh 실행’ 식. 시간 30분.”

SAY

“3단계 — 함수 작성. 단순 검증은 Bash 5~10줄 짧게, 복잡 로직은 Python 50~100줄 충분히. ‘JSON 입력 받기 → 검증 로직 → 종료 코드 + JSON 응답 반환’ 흐름. 시간 2~4시간.”

SAY

“4단계 — dry run 1주 + 본격 적용. ‘실제 차단은 X 경고만 출력’으로 운영. 사용자 피드백 수집 + false positive 패턴 발견. 1주 후 본격 차단으로 전환. 시간 1주.”

SAY

“패턴 — 단순 검증(파일명·키워드 매칭)은 Bash 5~10줄로 충분합니다. 복잡 로직(PII 정규식·DB 조회·외부 API)은 Python 50~100줄. 사내 IT 능력에 맞춰 선택.”

SAY

“디버깅은 log 출력 + dry run + 단계 적용이 표준이에요. Hook 함수가 ‘무엇을 받았나, 무엇을 결정했나’를 로그에 출력 → IT가 1주 운영 중 로그를 검토 → false positive·false negative 발견 → 함수 조정. 점진 안정.”

STORY

Hook Bash 예시 (강사 대응용). ‘.env 파일 Edit 차단’ Hook을 Bash로 작성하면 — `#!/bin/bash. input=$(cat). file=$(echo "$input" | jq -r .tool_input.file_path). if [[ "$file" == *.env* ]]; then echo "{\"reason\": \".env Edit 차단\"}" >&2; exit 2; fi. exit 0.` 약 5줄. 사내 IT 시니어가 1시간 작성. jq라는 도구로 JSON 파싱. 간단합니다.

STORY

Hook Python 예시 (강사 대응용). 한국 PII 감지 Hook을 Python으로 — `import sys, json, re. data = json.loads(sys.stdin.read()). file = data['tool_input'].get('file_path', ''). new_content = data['tool_input'].get('new_string', ''). pii_patterns = [r'\d{6}-\d{7}', r'010-\d{4}-\d{4}', r'[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+']. for pattern in pii_patterns: if re.search(pattern, new_content): print(json.dumps({'reason': 'PII 감지: ' + pattern}), file=sys.stderr); sys.exit(2). sys.exit(0).` 약 10줄. Python으로 작성하면 정규식 패턴 추가가 쉬워요.

STORY

Hook 라이브러리·도구 (강사 대응용). 사내 IT가 활용할 수 있는 도구 — (1) jq. JSON 파싱 Bash 도구. 표준 설치. (2) Microsoft Presidio. PII 다국어 감지 라이브러리. Python pip install presidio-analyzer. (3) KoNLPy. 한국어 처리 라이브러리. (4) curl. 외부 API 호출. Bash 표준. (5) slack-sdk. 슬랙 API 호출. Python pip install slack-sdk. 5가지로 대부분 Hook 가능.

NOTE

★ 강사 핵심 배경지식 — Hook 작성의 ‘5가지 함정’: 첫 작성 시 빠지기 쉬운 함정 — (1) 무한 루프. Hook이 다른 도구 호출 → 그 도구가 같은 Hook 트리거 → 무한 루프. 같은 Hook의 ‘재호출 방지’ 가드 코드 필수. (2) 성능. Hook 100ms+ 지연 → 사용자 답답. 캐싱·비동기로 대응. (3) false positive. 정상 작업이 차단 → 사용자 불편. 1주 dry run + 조정. (4) false negative. 위험 작업이 통과 → 사고. 정기 감사로 패턴 보강. (5) 디버깅 어려움. 로그 출력 + 분석 도구. 5가지 함정을 인지하고 작성하면 안전한 Hook이 만들어집니다.

NOTE

강사 배경지식 — Hook ‘권장 도구’: 사내 IT가 활용할 수 있는 도구를 정리하면 — Bash 작성 시 jq(JSON 파싱) + grep·sed(텍스트). Python 작성 시 json·re(정규식) + Microsoft Presidio(PII) + KoNLPy(한국어) + requests(외부 API). 외부 통신 필요 시 curl·slack-sdk. 한국 회사 PII 강박이면 Presidio + KoNLPy 결합이 가장 강력. 본 강좌 9편 보안 편.

Q

예상 청중 질문 1: ‘우리 IT가 Bash·Python 모르는데 어떻게 시작?’ → 답변: ‘Bash·Python 기초 1~2주 학습. 무료 온라인 강좌 다수. 또는 외주 — 약 100~300만 원에 ‘우리 회사 첫 Hook 5개 패키지’ 받을 수 있습니다. 외주 후 사내 IT가 인수받아 유지보수.’

Q

예상 청중 질문 2: ‘Hook이 외부 API를 호출해도 되나요?’ → 답변: ‘가능. 단 성능 영향 — 외부 호출 100ms+ 지연. 비동기 패턴 — Hook 자체 빠르게 종료 + 외부 호출 백그라운드. ‘nohup curl ... &’ 같은 패턴. 본 강좌 11편 자세히.’

Q

예상 청중 질문 3: ‘다른 회사 Hook 라이브러리?’ → 답변: ‘GitHub에서 ‘claude-code hooks’ 검색 → 공개된 사례 다수. Anthropic 공식 예시도 docs에 있어요. 사내 표준 패키지 만들어 다른 프로젝트에 공유도 가능합니다.’

TIP

강사 함정 — 코드 디테일 깊이 X. ‘4단계 + 함정 5가지’ 메시지만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 4단계 한눈에. 1~3분 작성 예시(Bash·Python). 3~4분 5가지 함정 + 권장 도구. 4~5분 청중 질문 1~2개.

BRIDGE

“작성을 봤다면 — 한국 회사가 실제 도입한 Hooks 5종 사례를 봅시다. 어떤 Hooks를 1개월 안에 도입하는지 표준 패턴이에요.”

KOREA HOOKS

한국 회사 Hooks 5종 사례

■ 가변층 — 2026.6 표준 패턴

- ★ 1 — 민감 파일(.env·secrets/) Edit 차단 (1시간)
- ★ 2 — 한국 PII(이름·주민번호·전화) 정규식 차단 (1일)
- ★ 3 — git commit 메시지 형식 강제(feat·fix·docs) (1일)
- ★ 4 — 코드 수정 후 자동 ESLint·테스트 (1주)
- ★ 5 — git push 후 슬랙 #ai-changes 알림 (2시간)
- ★ 5종 — 1개월 안 도입 가능, 사고 방지 + 자동화 균형

🕒 5분 · 한국 사례

SAY

“한국 회사가 첫 1개월 안에 도입하는 Hooks 5종 표준 패턴을 정리해드리겠습니다. ‘우리 회사가 따라가면 안전한 사내 표준’이에요. 각 작성 시간도 함께 표시했어요.”

SAY

“첫째 — 민감 파일 Edit 차단. .env·secrets/ 패턴이 들어간 파일 수정을 PreToolUse Hook이 자동 차단합니다. Bash 5줄 + 1시간 작성. 가장 단순 + 가치 큰 첫 Hook이에요. ‘비밀 정보 노출’ 사고를 100% 방지합니다.”

SAY

“둘째 — 한국 PII 정규식 차단. 이름·주민번호·전화 패턴을 PreToolUse Hook이 감지 + 차단. Python 50줄 + KoNLPy·Presidio 결합 + 1일 작성. 한국 개인정보보호법 준수의 핵심 도구예요.”

SAY

“셋째 — git commit 메시지 형식 강제. ‘feat·fix·docs·refactor’ 같은 접두가 없으면 PreToolUse Hook이 차단. Bash 10줄 + 1일 작성. 사내 표준 커밋 메시지가 자동으로 강제돼요. git history가 깔끔해지고 사후 추적이 쉬워집니다.”

SAY

“넷째 — 코드 수정 후 자동 ESLint·테스트. src/ 파일 Edit 후 PostToolUse Hook이 자동으로 npm test나 ESLint 실행. 코드 품질이 자동 유지돼요. Bash 20줄 + 1주 작성. CI 시스템의 사내 미니 버전이에요.”

SAY

“다섯째 — git push 후 슬랙 #ai-changes 알림. PostToolUse Hook + 슬랙 webhook 결합. 팀 가시성이 크게 향상돼요. Bash 10줄 + 2시간 작성. 가장 단순 + 즉시 효과 큰 Hook입니다.”

SAY

“5종을 1개월 안에 도입하면 ‘본격 안전 운영’이 가능합니다. 사고 방지 + 자동화 + 사내 표준화가 일괄 적용돼요. 사내 IT 1명이 1개월 안에 충분히 작성 가능한 분량입니다.”

STORY

한국 회사 Hooks 도입 비용·시간 (강사 대응용 추정). 5종을 작성하는 시간을 정리하면 — 1번 1시간 + 2번 1일(8시간) + 3번 1일(8시간) + 4번 1주(40시간) + 5번 2시간 = 총 59시간 = 약 1.5주 + 운영 안정 1주~1개월. 사내 IT 1명이 1개월 안 가능. 외주 옵션 — 약 200~500만 원에 5종 패키지 + 인수인계. ROI — 사고 방지(년 수백만 원 비용 절감) + 효율 향상(직원 시간 절감) = 1년 회수 충분.

STORY

Hooks 5종의 ROI 정량화 (강사 대응용). 도입 전후 비교 — (1) 비밀 정보 노출 사고: 도입 전 년 1~2건 + 사고당 평균 100~500만 원 비용 → 도입 후 0건. (2) PII 노출 사고: 도입 전 년 1~3건 + 사고당 평균 50~200만 원 → 도입 후 0건. (3) git commit 표준화: 사후 추적 시간 50% 절감. (4) 코드 품질: 버그 발견 시간 70% 단축. (5) 팀 협업: 가시성 + 신뢰. 5가지 결합 ROI 200%+. 본격 측정 가능한 가치예요.

STORY

한국 회사 Hooks 외주 시장 (강사 대응용 추정). 2026.6 기준 한국 ‘Claude Code 컨설팅’ 외주 시장이 성장 중이에요. 일부 보안·DevOps 전문 회사가 ‘Hooks 5종 패키지’를 제공. 가격대 — 단순 5종 약 200만 원, 사내 정책 분석 + 맞춤 Hooks 10종 약 500~1,000만 원. 본격 컨설팅 + 6개월 운영 + 인수인계 약 1,000~3,000만 원. 사내 IT가 부족한 회사 옵션입니다.

NOTE

★ 강사 핵심 배경지식 — 5종 도입 우선순위: 한 달 안 단계별 도입 순서를 권장하면 — (1) 1주 1~2번. 보안 강한 두 Hook 우선. 첫 도입은 안전 우선. (2) 2주 3번. 사내 표준화 추가. (3) 3주 4번. 코드 품질 자동화. (4) 4주 5번. 협업 알림. 5종 1개월 완성. 단 ‘우리 회사 특수 정책’이 있으면 그것 먼저. 우선순위는 회사 상황에 맞게.

NOTE

강사 배경지식 — 5종 이후 ‘추가 후보’: 1개월 5종 + 6개월 운영 후 추가할 수 있는 Hooks 후보 — (1) 비용 한도 — 일일 토큰 한도 초과 시 차단(2강 1번 슬라이드 비용 한도와 결합). (2) 외부 fetch 화이트리스트 — 회사 도메인만 허용. (3) DB 쓰기 사용자 확인 — UPDATE/DELETE 시 매번 확인. (4) 사내 위키 자동 색인 — 변경 사항 자동 검색에 반영. (5) Skills 자동 매칭 — 작업 종류에 맞는 Skill 추천. 5종 추가 후보가 ‘본격 자율 단계’의 표준이에요.

Q

예상 청중 질문 1: ‘5종 다 도입해야 하나요?’ → 답변: ‘권장합니다. 5종 1개월 작업 + 한 번 만들면 영구. 1년 ROI 200%+. 단 우리 회사 IT 1명이 1개월 시간이 없으면 우선순위 1·2·5 세 가지만 먼저(2주). 나중에 3·4 추가.’

Q

예상 청중 질문 2: ‘우리 IT 1명 1개월 부담?’ → 답변: ‘옵션 두 가지 — (1) 외주 200~500만 원 + 사내 인수인계 1개월. (2) 사내 IT 1명 + 1개월 집중 작업. 사내 학습 가치 — 6편 학습이 사내 자산이 됨. 가능하면 사내 권장.’

Q

예상 청중 질문 3: ‘다른 Hook은 어떤 게 있나요?’ → 답변: ‘많습니다. 5종이 첫 도입 표준이고, 6개월 운영 후 ‘사내 발견’에 따라 추가. 비용 한도·외부 fetch·DB 쓰기 사용자 확인·사내 위키 자동 색인·Skills 추천 등 5종 추가 후보가 있어요. 본 강좌 11편 자세히.’

TIP

강사 함정 — ‘완벽 5종’ 압박 X. ‘우선 1·2부터 시작’ 점진 메시지.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 5종 한눈에. 1~3분 각 30초씩 + 작성 시간. 3~4분 ROI 정량화. 4~5분 우선순위 + 청중 질문.

BRIDGE

“5종 사례를 봤다면 — 2강의 마지막. 실습으로 우리 회사 첫 Hook 설계서를 작성합니다.”

WORKSHOP 2

실습 — 우리 회사 첫 Hook 설계서

■ 적용 (워크시트)

- Q1 · 첫 Hook 후보 — 5종 중 무엇?
- Q2 · 이벤트 — PreToolUse 또는 PostToolUse?
- Q3 · 작성 도구 — Bash 또는 Python?
- Q4 · 작성자 — 사내 IT 또는 외주?
- Q5 · 일정 — 1주 / 2주 / 1개월?
- ★ 다음 액션 — IT 1명이 1주 안에 첫 Hook + 테스트

⌚ 8분 · 실습

SAY

“2강의 마지막 — 실습입니다. 첫 Hook 설계서 5칸을 채워서 사내 IT에게 ‘이 Hook을 1주 안에 작성해 주세요’ 발주서를 만듭니다.”

DO

조별 실습 진행. 3~4인 1조로 약 5분간 5칸을 채웁니다. Q1~Q5 + 다음 액션. 강사는 조별로 돌며 가이드.

SAY

“권장 시작 패턴 — 민감 파일 차단(1번) + PreToolUse + Bash + 사내 IT + 1주. 가장 안전 + 간단 + 첫 도입 정석이에요. 90% 한국 회사가 이걸로 시작합니다.”

Q

발표 요청 1: ‘1~2 조 발표해주세요. 첫 Hook 후보는 무엇으로 선택했는지.’ 보통 ‘민감 파일 차단’이 다수. ‘좋습니다. 첫 도입의 정석입니다’ 강사가 확인.

Q

발표 요청 2: “PII 정규식 차단’ 선택한 조 있나요?” → 보통 1~2조. ‘좋습니다. 한국 개인정보보호법 준수가 우선인 회사예요. 단 Python + 1일 작업이라 1번 + 2번 순서로 진행 권장.’

Q

발표 요청 3: ‘외주를 선택한 조가 있나요?’ → 가끔. ‘좋습니다. 사내 IT 시간이 없는 회사 옵션. 단 6개월 후 사내 인수인계는 정석. 사내 학습이 사내 자산이에요.’

NOTE

★ 강사 핵심 배경지식 — 결정서 5칸 작성 가이드: 청중이 막힐 수 있는 부분을 정리하면 — (1) Q1 첫 후보: 5종 중 ‘민감 파일 차단’이 90% 권장. 우리 회사 특수 정책이 있다면 그것 먼저. (2) Q2 이벤트: 차단 목적이면 PreToolUse, 자동화 목적이면 PostToolUse. (3) Q3 도구: 단순 매칭은 Bash, 복잡 로직은 Python. (4) Q4 작성자: 사내 IT가 정석. 외주는 비상시. (5) Q5 일정: 1번은 1주, 2번은 2주, 4번은 1개월. 후보에 따라.

NOTE

강사 배경지식 — 다음 주 액션 권장 3가지: 1주 안 시작할 수 있는 액션 — (1) 첫 Hook 설계서를 사내 IT에 전달. (2) 사내 IT 1명이 1주 안 첫 Hook 작성 + 단위 테스트. (3) 1주 dry run 시작 + 사용자 피드백 수집. 셋 다 2주 안 가능. 청중에게 ‘이번 주 안 가능한 일정’ 인지시키세요.

NOTE

강사 배경지식 — 첫 Hook 후 ‘점진 확장’ 흐름: 1번 Hook 1주 운영 안정 후 → 2번 PII Hook 1~2주 → 3번 commit 메시지 1주 → 4번 자동 테스트 2~3주 → 5번 슬랙 알림 1주. 총 1~2개월 안 5종 완성. ‘한 번에 다 작성 X 점진 확장’이 정석이에요. 단계별 운영 안정성이 우선.

Q

예상 청중 질문 1: ‘우리는 결정 못 할 것 같은데?’ → 답변: “민감 파일 차단’이 안전한 기본값. 첫 도입은 90% 회사가 이걸로 시작. 모호하면 그것으로 결정 + 1주 운영 후 다음 결정. 결정 못하는 것보다 결정 잘못이 안전합니다.’

Q

예상 청중 질문 2: ‘외주는 어떻게 선택?’ → 답변: “사내 IT가 1주 시간 없거나 Bash·Python 학습 부담’ 시 외주 옵션. 약 200만 원에 첫 Hook 5개 패키지. 외주 후 6개월 사내 인수인계가 정석.’

Q

예상 청중 질문 3: ‘일정 1주가 너무 빠르지?’ → 답변: ‘1번 ‘민감 파일 차단’은 1주 충분. 작성 1시간 + dry run 1주. 더 복잡한 Hook(PII 정규식 등)은 2주 이상. 후보에 따라 일정 조정.’

TIP

강사 함정 — ‘완벽 결정 압박’ X. ‘60% 답으로 시작’ 메시지 다시.

TIP

강사 진행 팁 — 8분 시간 배분: 0~5분 개별·조별 작성. 5~7분 1~2조 발표. 7~8분 ‘다음 주 액션 3가지’ + 마무리.

BRIDGE

“2강 끝났습니다. 잠깐 쉬고 3강 ‘Skills + Slash + MCP’에서 만나요. 회사 도메인 지식을 코드 패키지로 묶고, 단축 명령을 만들고, 외부 시스템을 통합하는 본격 결합 단계예요.”

WRAP

2강 정리 — Hooks 핵심

■ 마무리

- Hooks = 동작 전후 자동 함수, 8가지 이벤트
- ★ PreToolUse — 사전 검증·차단(안전 핵심)
- ★ PostToolUse — 사후 자동·알림(자동화 핵심)
- ★ 한국 5종 — 1개월 안 도입 가능
- ★ 첫 Hook 1주 작성 + 1주 dry run + 본격
- ★ 다음 — 3강 Skills·Slash·MCP 결합

🕒 3분 · 정리

SAY

“2강을 정리합니다. Hooks는 Claude Code의 동작 흐름 사이에 자동 함수를 끼워 넣는 메커니즘. 8가지 이벤트가 있고, 핵심은 PreToolUse(안전)와 PostToolUse(자동화) 두 가지예요. 한국 회사는 5종 Hooks를 1개월 안에 도입하면 본격 안전 운영이 가능해집니다.”

SAY

“2강 핵심 메시지 — ‘첫 Hook 1개를 신중하게 작성 + 1주 dry run + 안정 후 본격 + 점진 확장.’ 처음부터 10개 Hooks 한꺼번에 X. 1개씩 안정 → 5개 → 10개 → 사내 표준. 단계별 진화가 정석입니다.”

SAY

“3강 예고 — Skills + Slash Commands + MCP의 결합입니다. settings.json(권한)과 Hooks(자동)가 ‘안전 운영의 기반’이었다면, Skills + Slash + MCP는 ‘도메인 지식 + 단축 + 외부 통합’으로 본격 효율 향상입니다. 우리 회사의 일하는 방법을 코드 패키지로 묶는 단계예요.”

NOTE

★ 강사 핵심 배경지식 — 2강 끝 톤: ‘settings.json + Hooks = 안전 기반’이 갖춰졌으니 다음은 ‘효율 향상’으로 진화한다는 메시지. 청중에게 ‘우리 회사가 6편 1·2강을 1~2개월 안 완성하면 3강을 6개월 내 도입할 준비가 된다’ 시간 감각.

TIP

강사 진행 팁 — 3분 안에. 정리 슬라이드는 메시지 강조가 목적.

BRIDGE

“쉬는 시간 10분. 3강에서 만나요.”

INTRO · TITLE

Skills · Slash · MCP

3강 — 도메인 지식 패키징 + 단축 명령 + 외부 통합

🕒 3분 · 시작

SAY

“3강 ‘Skills + Slash Commands + MCP’를 시작합니다. 6편의 마지막 강이고, 하네스 폴스택을 완성하는 단계예요. settings.json(권한) + Hooks(자동)가 ‘안전 운영의 기반’이었다면, Skills + Slash + MCP는 ‘도메인 지식 + 단축 + 외부 통합’으로 본격 효율 향상입니다.”

SAY

“3강 60분 동안 다섯 가지를 봅니다. 첫째 — Skills의 본질과 폴더 구조. 둘째 — Skills 작성 5단계 패턴. 셋째 — Slash Commands(자주 쓰는 명령 단축). 넷째 — MCP 통합(4편 + 6편 결합). 다섯째 — 4가지를 결합한 ‘하네스 폴스택’과 한국 회사 도입 + 실습. 결과 — ‘우리 회사 하네스 폴스택 1페이지’예요.”

SAY

“3강의 약속 — Skills는 2025년 Anthropic이 추가한 신규 기능입니다. 도입 시점이 비교적 늦었지만 가치가 큼니다. 우리 회사의 ‘일하는 방법’을 코드 패키지로 묶어서 신입도 시니어 수준으로 일하게 만드는 도구예요. 본 강자가 ‘초보자 톤’을 유지해서 디테일은 사내 IT의 일, 청중은 ‘어떤 Skills를 만들지 결정’만 합니다.”

Q

도입 질문 1: ‘Skills를 5편에서 들어본 분?’ — 보통 다수. 5편 2강에서 Claude Code의 4가지 기능 중 하나로 Skills를 언급했어요. ‘좋습니다. 5편이 개념 소개였다면 6편 3강이 본격 작성’이라고 강사가 연결.

Q

도입 질문 2: ‘우리 회사에 ‘표준 양식’이 있는 작업 있나요? 보고서·회의록·계약서·코드 리뷰 같은’ — 보통 다수. ‘좋습니다. 그 표준 양식이 Skills 후보입니다. 3강 본문이 자세히 설명합니다.’

NOTE

★ 강사 핵심 배경지식 — Skills의 위치: settings.json·Hooks가 ‘권한·정책’이라면 Skills는 ‘지식 패키징’입니다. 회사 매뉴얼·표준 양식·코딩 규칙·도메인 전문성이 코드 패키지로 묶여요. 신입 직원이 ‘우리 회사 어떻게 영업 보고서 쓰나요’ 물을 필요 없이 Skills가 자동 활성화. 도메인 지식의 자동 전수. 6편이 다루는 ‘회사 정책 코드화’의 가장 강력한 표현이에요.

NOTE

강사 배경지식 — 3강 60분 시간 배분: 0~3분 인사. 3~6분 학습 목표. 6~14분 Skills 본질(25번). 14~22분 Skills 작성 패턴(26번). 22~30분 Slash Commands(27번). 30~38분 MCP 통합(28번). 38~46분 하네스 폴 스택(29번). 46~52분 실습(30번). 52~58분 6편 정리(31번). 58~60분 작별(32번). 시간 압박 있지만 ‘각 도구의 본질 + 결합 가치’ 메시지가 분명하면 충분합니다.

TIP

강사 함정 — Skills 폴더 구조의 JSON·YAML 디테일 깊이 X. ‘구조 + 작성 패턴 + 가치’ 메시지만. 사내 IT가 디테일 처리합니다.

BRIDGE

“학습 목표 빠르게.”

OBJECTIVES

60분 후, 이 3가지를 한다

- ① Skills + Slash + MCP 결합 구조를 안다
- ② 우리 회사 도메인 Skills 1개를 설계한다
- ③ ‘하네스 폴스택 1페이지’를 완성한다

🕒 3분 · 학습목표

SAY

“3강 60분이 끝나면 세 가지를 할 수 있게 됩니다. 첫째 — Skills + Slash + MCP 세 도구가 어떻게 결합되어 ‘하네스 폴스택’을 이루는지 설명할 수 있게 됩니다. 각 도구가 어떤 역할이고, 어떻게 협력하는지 정리.”

SAY

“둘째 — 우리 회사 도메인 Skills 1개를 설계합니다. ‘영업 보고서 작성’ 또는 ‘회의록 양식’ 같은 후보를 선택하고 SKILL.md를 어떻게 구성할지 결정해요. 본 강좌가 ‘설계’ 단계까지. 실제 작성은 사내 IT + 부서 챔피언이 1주 안 협업.”

SAY

“셋째 — 가장 중요한 결과물 — ‘하네스 폴스택 1페이지’ 워크시트. settings.json + Hooks + Skills + Slash + MCP 5가지를 6개월 일정에 따라 단계별 도입하는 계획서예요. 6편의 결론입니다.”

SAY

“3강 핵심 메시지를 미리 — ‘우리 회사 도메인 지식 1개를 Skills 패키지로 만들어 → 전사 공유 → 신입도 시니어 수준으로 일.’ 매뉴얼 100페이지를 작성·배포·학습 강요 X. Skills 1개 작성 + git 공유. 회사 지식의 본질적 전수 패러다임이에요.”

NOTE

★ 강사 핵심 배경지식 — 3강의 성공 기준: 청중이 ‘우리 회사 도메인 Skills 1개 후보’를 자신 있게 선택하고, ‘하네스 폴스택 6개월 도입 일정 1페이지’를 그릴 수 있어야 합니다. 6편의 결론이자 청중이 회사 가서 즉시 적용 가능한 결과물이에요.

TIP

강사 진행 팁 — 학습 목표 3분 안에. 본문에 더 시간 할애.

BRIDGE

“그럼 본문 시작 — Skills의 본질부터 봅니다.”

SKILLS WHAT

Skills — 도메인 지식 패키징

■ 불변층 — 2025 Anthropic 신규 기능

- ★ Skills — ‘우리 회사 어떻게 일하나’ 코드 패키지
- ★ 구조 — 폴더 + SKILL.md(마크다운) + 코드·예시
- ★ 예 — ‘영업 보고서 작성’, ‘법무 계약서 검토’
- ★ 위치 — .claude/skills/<name>/SKILL.md
- ★ 자동 발견 — Claude Code가 작업 인식 → 자동 호출
- ★ 사내 공유 — git 커밋 + 전사 동기화

🕒 6분 · Skills 풀이

SAY

“Skills의 본질을 풀이합니다. Skills는 Anthropic이 2025년에 추가한 신규 기능이에요. ‘Introducing Agent Skills’라는 공식 발표로 출시됐고, 1년 안에 한국 회사가 도입하는 본격 패턴이 됐어요.”

SAY

“Skills를 한 문장으로 — ‘우리 회사가 어떻게 일하는가’를 코드 패키지로 묶은 것입니다. 영업 보고서 작성·법무 계약서 검토·사내 코드 리뷰 표준·회의록 양식·인사 평가 가이드 같은 ‘우리 회사 도메인 지식’이 폴더 구조로 정리됩니다.”

SAY

“구조는 폴더 + SKILL.md + 보조 파일이에요. ‘.claude/skills/<이름>/SKILL.md’ 파일이 핵심. 마크다운으로 작성된 ‘이 작업의 흐름’ 설명서예요. 보조 파일로 examples 폴더에 ‘과거 결과 3~5건’을 두면 더 풍부합니다.”

SAY

“예를 들어 ‘영업 보고서 작성 스킬’을 만들면 — SKILL.md에 ‘우리 회사 영업 보고서 양식: 1. 제목 2. 요약 3. 매출 분석 4. 다음 분기 계획’ 같은 흐름이 적힙니다. examples 폴더에는 과거에 잘 작성된 영업 보고서 3건. 영업팀 모든 직원이 이 Skill을 git pull로 받아서 사용. 신입도 첫날부터 시니어 수준 보고서를 쓸 수 있어요.”

SAY

“가장 강력한 점은 ‘자동 발견’이에요. Claude Code가 ‘영업 보고서 작성해줘’ 사용자 입력을 받으면 → Skills 목록에서 ‘영업 보고서’ 키워드를 검색 → ‘sales-report’ Skill의 description을 매칭 → 자동으로 활성화. 사용자가 ‘이 Skill을 사용해줘’ 명시할 필요 없어요. 자연어 입력만으로 도메인 지식이 자동 적용됩니다.”

SAY

“사내 공유는 git 커밋 + 전사 동기화입니다. ‘.claude/skills/’ 폴더를 프로젝트에 git으로 두면 전 직원이 git pull로 같은 Skills를 사용. 영업팀이 작성한 Skill을 마케팅팀도 참고. 회사 전체가 같은 도메인 지식을 공유하는 패러다임이에요.”

STORY

출처 — Anthropic 공식 발표. ‘Introducing Agent Skills’ 2025년 발표. URL: claude.com/blog/skills. 핵심 인용 — ‘structured domain knowledge packaging’. 도메인 지식을 구조화된 패키지로 묶는다는 개념이에요. Claude Code와 Claude Desktop 모두 호환. Skills 파일 구조는 ‘공개 표준’으로 출시되어 다른 도구도 호환 가능.

STORY

Skills 폴더 구조 디테일 (강사 대응용). ‘영업 보고서 작성’ Skill 폴더를 자세히 보면 — .claude/skills/sales-report/ 폴더 안에 (1) SKILL.md — 주 설명서. YAML frontmatter에 name·description·tools 명시 + 본문에 ‘작성 흐름’ 마크다운. (2) examples/ — 과거 결과 3~5건. 1.md·2.md·3.md 같은 파일. (3) templates/ — 빈 양식. 영업 보고서 템플릿.md. (4) scripts/ — 보조 코드. ‘매출 데이터 조회 스크립트.py’. 4가지가 결합되어 Skill 1개 완성. 사내 IT + 부서 챔피언 1주 협업으로 작성 가능.

STORY

한국 회사 Skills 사례 (강사 대응용 추정). 일부 한국 IT 회사가 도입한 Skills 종류 — (1) 사내 코드 스타일 가이드. 사내 코딩 규칙(들여쓰기·네이밍·주석)이 Skill로 코드화. (2) API 문서 작성. 사내 표준 API 문서 양식. (3) 회의록 양식. 분야별 회의 양식(개발·영업·전사). (4) 영업 보고서. 영업팀 표준 양식. (5) 코드 리뷰 가이드. 사내 PR 리뷰 체크리스트. 5~10개가 ‘6개월 운영 후 갖춰지는 사내 표준’. 1년+ 운영 회사는 20~50개 Skills로 확장됩니다.

NOTE

★ 강사 핵심 배경지식 — Skills의 ‘5가지 가치’: Skills 도입으로 회사가 얻는 가치를 정리하면 — (1) 1회 작성 + 전사 공유. 영업팀 시니어 1명이 작성한 Skill을 전 영업팀이 사용. (2) 자동 호출. 사용자가 명시할 필요 X. 자연어 입력 → 자동 발견. (3) 도메인 전문성. 신입도 시니어 수준. 학습 비용 ↓. (4) 표준화. 부서별 일관된 결과물. (5) git history. 도메인 지식 변경 추적 가능. 5가지가 결합되어 ‘회사 지식 자산화’가 완성됩니다.

NOTE

강사 배경지식 — Skills vs CLAUDE.md 차이: CLAUDE.md는 프로젝트 전체에 한 개 — ‘이 프로젝트의 일반 가이드’. Skills는 작업별 다수 — ‘이 작업의 특수 지식’. CLAUDE.md에 ‘우리 회사 코딩 표준 + 보안 정책 + 일반 가이드’가 들어가고, Skills에 ‘영업 보고서 작성·법무 계약서 검토·코드 리뷰’ 같은 작업별 디테일이 들어가요. 두 가지를 모두 사용하면 ‘일반 + 특수’ 지식이 잘 분리됩니다.

NOTE

강사 배경지식 — Skills 자동 호출 메커니즘: SKILL.md YAML frontmatter의 description 키가 핵심이에요. 예 — ‘description: 영업 보고서 작성에 사용. 회사 양식·매출 데이터·시즌 트렌드 포함’. Claude Code가 사용자 ‘영업 보고서 작성해줘’ 입력을 받으면 → 모든 Skills의 description을 벡터 임베딩으로 유사도 검색 → 가장 매칭되는 Skill 자동 활성화. 4편 RAG 기술과 유사한 메커니즘이에요. description을 명확하게 쓰는 게 자동 호출 성공의 핵심.

Q

예상 청중 질문 1: ‘우리 회사 첫 Skill 후보는?’ → 답변: ‘자주 반복 + 표준 양식이 있는 작업이 1순위. 일반 회사 ‘회의록 양식’ 또는 ‘이메일 답신 양식’. IT 회사 ‘사내 코드 스타일’ 또는 ‘PR 리뷰 가이드’. 영업팀 있는 회사 ‘영업 보고서’. 부서장과 토론 후 결정. 첫 Skill 1주 작성.’

Q

예상 청중 질문 2: ‘비IT 부서도 Skills 작성 가능한가요?’ → 답변: ‘마크다운 작성은 OK. IT 결합 X도 가능. 영업팀이 ‘영업 보고서 양식 + 과거 예시’ 마크다운으로 작성 → IT가 git 공유 설정. 자동화 코드(스크립트)는 IT 결합. 부서 자체 작성이 가능합니다.’

Q

예상 청중 질문 3: ‘Skills 몇 개부터 시작?’ → 답변: ‘첫 도입 1~3개. 한 부서 1개씩 작성 + 1개월 운영. 6개월 후 5~10개로 확장. 1년+ 운영 시 20~50개. 점진 확장이 정석입니다.’

TIP

강사 함정 — Skills 코드 디테일 깊이 X. ‘구조 + 가치 + 자동 호출’ 메시지만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 풀이. 1~3분 구조 + 예시. 3~4분 자동 호출. 4~5분 사내 공유 + 5가지 가치. 5~6분 청중 질문 1~2개.

BRIDGE

“Skills의 본질을 봤다면 — 실제 작성 패턴을 봅니다. 5단계 흐름으로 정리해드리겠습니다.”

SKILLS WRITE

Skills 작성 패턴 — 5단계

■ 불변증 — 첫 Skill 1주 작성

- ★ 1단계 — 후보 선정 (자주 반복 + 표준 양식)
- ★ 2단계 — SKILL.md 작성 (description·tools·instructions)
- ★ 3단계 — examples/ 폴더 (과거 결과 3~5건)
- ★ 4단계 — 자동 호출 테스트 (입력 → 매칭)
- ★ 5단계 — git 공유 + 피드백 + 개선
- ★ 시간 — 첫 Skill 1주, 이후 2~3일

🕒 6분 · Skills 작성

SAY

“Skills 작성의 실제 5단계 흐름을 봅니다. 첫 Skill을 1주 안에 작성하는 표준 패턴이에요. 사내 IT + 부서 챔피언이 협업합니다.”

SAY

“1단계 — 후보 선정. ‘우리 회사에서 자주 반복되는 + 표준 양식이 있는’ 작업을 선택해요. 영업 보고서 작성·회의록 양식·코드 리뷰 가이드 등 부서별 1순위. 부서장과 토론 후 결정. 시간 1~2시간.”

SAY

“2단계 — SKILL.md 작성. 마크다운 파일이에요. YAML frontmatter에 ‘name(스킬 이름)·description(언제 사용)·tools(필요 도구 — 옵션)·model(권장 모델)’ 명시. 본문에 ‘작업 흐름 1·2·3·4 단계’ 마크다운. 핵심은 description이 명확해야 자동 호출이 정확합니다. 시간 2~4시간.”

SAY

“3단계 — examples 폴더 채우기. 과거에 잘 작성된 결과 3~5건을 examples/1.md·2.md·3.md로 저장. ‘이런 입력에서 이런 결과’ 패턴이 명확해야 Claude Code가 따라하기 좋아요. 부서 챔피언이 ‘우수 사례 3건’을 골라서 추가. 시간 1~2시간.”

SAY

“4단계 — 자동 호출 테스트. 다양한 사용자 입력 — ‘영업 보고서 작성해줘’, ‘우리 회사 분기 매출 정리해줘’, ‘이번 분기 영업 보고 만들어줘’ — 으로 Skill 자동 호출되는지 확인. description 키워드를 보완해서 매칭 정확도 ↑. 시간 1일.”

SAY

“5단계 — git 공유 + 사용자 피드백 + 개선. .claude/skills/ 폴더를 git 커밋 → 전 팀에 공유. 1주 운영 + 사용자 피드백 수집 → SKILL.md 본문 업데이트. 시간 1주.”

SAY

“총 1주(작성 ~2일 + 테스트 1일 + 운영 ~1주)에 첫 Skill 완성. 두 번째 Skill부터는 2~3일이면 가능해요. 1개월에 5~10개 Skills 작성이 현실적입니다.”

STORY

한국 회사 Skills 작성 시간 (강사 대응용 추정). 첫 Skill 1주 = 사내 IT 1명(2일) + 부서 챔피언 1명(1일) + 자동 호출 테스트(1일) + 운영 안정(3일). 두 번째 Skill부터는 사내 IT 1일 + 챔피언 0.5일 + 테스트 0.5일 = 2일 단축. 1개월 5~10개 가능. 외주 — 약 500만 원에 ‘우리 회사 도메인 Skills 10개 패키지’. 단 사내 작성이 사내 도메인 지식 자산화에 정석입니다.

STORY

Anthropic 사내 Skills 사례 (강사 대응용). ‘How Anthropic teams use Claude Code’ 인용에 따르면 Anthropic 사내 부서별 다양한 Skills 사용. Marketing·Legal·Engineering·Sales·Support 5부서 각각 평균 5~10개 Skills. 부서별 도메인 지식이 코드 패키지로 자산화. 신규 입사자가 첫날부터 사내 도메인 적용 작업 가능. 한국 회사가 따라가야 할 표준 패턴이에요.

NOTE

★ 강사 핵심 배경지식 — Skills 작성의 ‘5가지 함정’: 첫 작성 시 빠지기 쉬운 함정 — (1) 너무 일반. description이 ‘보고서 작성’처럼 모호 → 다른 Skill과 충돌·매칭 X. ‘영업 분기 보고서’처럼 구체적이어야 합니다. (2) 너무 특수. description이 ‘우리 회사 분기 매출 KPI 보고서’처럼 너무 좁아 → 다양한 입력에 매칭 X. (3) examples 부족. 1~2개만 두면 Claude가 패턴 학습이 부족 → 결과 품질 ↓. 3~5개 권장. (4) git 공유 X. 개인 폴더에만 두면 전사 공유 X. (5) 피드백 없이 운영. 1주 운영 후 사용자 피드백 수집 + 개선이 필수. 5가지 함정 피하기.

NOTE

강사 배경지식 — description의 ‘좋은 작성법’: Skills 자동 호출의 정확도가 description 작성에 좌우돼요. 좋은 description 예시 — ‘sales-report: 영업 보고서 작성. 분기·반기·연간 매출 분석, KPI 데이터 정리, 시즌 트렌드 분석, 다음 분기 계획. 영업팀 직원이 사용. 회사 표준 양식 적용.’ 약 80~120자. 다양한 키워드(영업 보고서·매출·KPI·분기·영업팀·표준 양식)가 명시되어 사용자의 다양한 표현이 매칭됩니다.

NOTE

강사 배경지식 — Skills의 ‘버전 관리’: Skills가 운영되며 업데이트될 때 git history가 변경 추적의 핵심이에요. SKILL.md 변경 시 git commit 메시지에 ‘sales-report: 분기 KPI 항목 추가’ 같은 명시 → 사후 ‘언제·왜 변경됐나’ 추적 가능. 부서장 PR 리뷰 + 시니어 승인 → main 머지 흐름이 정석. 회사 도메인 지식의 변경이 코드 변경처럼 관리되는 패러다임이에요.

Q

예상 청중 질문 1: ‘우리 회사 첫 Skill을 무엇으로 시작?’ → 답변: ‘부서별 가장 자주 반복 + 표준 양식이 있는 작업. 일반 회사 ‘회의록 양식’ 또는 ‘이메일 답신’. IT 회사 ‘사내 코드 스타일’. 부서장과 토론 후 결정. 첫 Skill 1주 작업.’

Q

예상 청중 질문 2: ‘비IT 부서가 Skills 작성 가능한가요?’ → 답변: ‘마크다운 + 예시 작성은 OK. IT 없이 가능. 영업팀이 ‘영업 보고서 양식 + 우수 사례 3건’ 마크다운으로 작성 → IT가 git 공유 설정만. 자동화 코드(스크립트)가 필요하면 IT 결합.’

Q

예상 청중 질문 3: ‘Skill 사용 여부를 어떻게 확인?’ → 답변: ‘Claude Code 세션 끝에 ‘이번 세션에서 사용된 Skills 목록’ 로그 + 사용자 만족도 설문. 자동 호출 정확도가 낮으면 description 보완. 본 강좌 11편 자세히.’

TIP

강사 함정 — 5단계 작성 디테일 깊이 X. ‘5단계 흐름 + 함정 5가지’ 메시지만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 5단계 한눈에. 1~3분 SKILL.md 구조 + 예시. 3~4분 5가지 함정. 4~5분 시간·외주. 5~6분 청중 질문.

BRIDGE

“Skills 작성을 봤다면 — 다음은 Slash Commands입니다. 자주 쓰는 명령을 단축어로 만드는 도구예요.”

SLASH

Slash Commands — 단축 명령

■ 불변증 — 자주 쓰는 명령 단축

- ★ Slash Commands — ‘/단어’ 형식의 단축 명령
- ★ 위치 — .claude/commands/<name>.md
- ★ 예 — /report(보고서), /review(PR 리뷰), /deploy(배포)
- ★ 구조 — 마크다운 + 명령 흐름
- ★ 차이 — Skills(자동 호출) vs Slash(사용자 명시)
- ★ git 공유 — 프로젝트 레벨로 팀 동기화

🕒 5분 · Slash

SAY

“Slash Commands를 봅니다. 자주 쓰는 명령을 ‘/단어’ 단축어로 만드는 도구예요. Skills와 비슷하지만 ‘자동 호출’이 아닌 ‘사용자 명시’ 방식이에요.”

SAY

“Slash Commands를 한 문장으로 — ‘우리 회사가 자주 쓰는 작업을 /짧은단어로 단축’입니다. 매번 ‘이번 분기 영업 보고서 작성해줘’ 같은 긴 입력 X. ‘/report quarterly sales’ 단축으로 같은 효과.”

SAY

“위치는 ‘.claude/commands/<이름>.md’ 파일. /commit이라는 단축어를 만들려면 ‘.claude/commands/commit.md’ 파일을 작성. Claude Code가 자동으로 등록하고 사용자가 ‘/commit’ 입력 시 호출됩니다.”

SAY

“예 — /commit (사내 표준 커밋 메시지 생성), /review (PR 리뷰), /deploy (배포 흐름), /test (테스트 작성), /report (보고서 작성), /docs (문서화). 사내 표준 작업이 단축어로 정리되면 직원 시간 절감 + 표준화 동시 달성.”

SAY

“구조는 단순합니다. 마크다운 파일에 ‘이 명령이 무엇을 할지’ 설명을 적습니다. ‘/commit을 입력하면 다음 흐름을 따른다: 1. git status 확인 → 2. 변경 사항 분석 → 3. 사내 표준 형식(feat/fix/docs) 적용 → 4. 커밋 메시지 제안’ 같은. 30~50줄 마크다운으로 충분.”

SAY

“Skills와의 차이는 ‘호출 방식’이에요. Skills는 자연어 입력 → 자동 발견 + 자동 호출. Slash는 ‘/단어’ 사용자 명시 호출. 둘 다 가치 있지만 다른 용도예요. Skills는 ‘다양한 입력에 자동 매칭’, Slash는 ‘정확한 단축 명령.’”

SAY

“git 공유는 프로젝트 레벨이에요. .claude/commands/ 폴더를 git 커밋 → 전 팀이 같은 단축어 사용. 사내 표준이 git 공유로 전사 동기화.”

STORY

출처 — Anthropic Claude Code 공식 문서. docs.claude.com/claude-code/slash-commands 페이지에 Slash Commands 사양이 명시돼 있어요. 자동 등록 메커니즘 — Claude Code가 ‘.claude/commands/’ 폴더를 자동 스캔 + 각 .md 파일을 ‘/<파일명>’ 단축어로 등록. 사용자가 ‘/commit’ 입력 시 commit.md의 마크다운 흐름이 실행됩니다.

STORY

한국 회사 Slash 사례 (강사 대응용 추정). 일부 한국 회사가 도입한 Slash Commands 4종 — (1) /commit. 사내 표준 커밋 메시지 형식(feat·fix·docs) 자동 적용 + 변경 사항 요약. (2) /test. 테스트 작성 + 사내 표준 양식 + 자동 실행. (3) /review. PR 리뷰 가이드 + 체크리스트 적용. (4) /docs. 사내 문서 작성 + 표준 양식. 4종이 ‘본격 운영 회사 표준’이에요. 1주 작성 + 즉시 효과.

STORY

Slash vs Skills의 결합 사례 (강사 대응용). 같은 회사가 ‘/report’ Slash + ‘sales-report’ Skills를 둘 다 두면 — (1) 사용자가 ‘/report quarterly sales’ 명시 호출 시 → /report 명령이 ‘영업 보고서 작성 흐름’ 실행 → 내부에서 sales-report Skill 자동 활성화 → 도메인 지식 적용 + 결과 생성. 두 도구의 결합으로 단축 + 도메인 지식이 동시 적용돼요. 본 강좌 6편의 본격 결합 예시입니다.

NOTE

★ 강사 핵심 배경지식 — Slash vs Skills 결정 기준: 두 도구를 어떻게 분담할지 정리하면 — (1) Slash: 사용자가 ‘명시 가능’ + 자주 쓰는 표준 작업. 단축 효과가 핵심. (2) Skills: 사용자가 ‘다양한 표현으로 입력’ + 자동 매칭이 필요한 도메인 작업. 같은 작업을 둘 다 만들 수도 있어요 — Slash는 단축, Skills는 자동 호출. 결합하면 가장 강력합니다.

NOTE

강사 배경지식 — Slash Commands의 ‘인자 전달’: Slash 명령에 인자를 전달할 수 있어요. ‘/report quarterly sales’ 입력 시 → ‘quarterly’와 ‘sales’가 Slash 명령의 인자로 전달. commit.md 안에 ‘\${1}’ ‘\${2}’ 같은 변수로 참조 가능. 단순 단축뿐 아니라 ‘파라미터 받는 함수’ 같은 활용이 가능해요. 사내 IT가 디테일 학습.

Q

예상 청중 질문 1: ‘우리 회사 첫 Slash 후보?’ → 답변: ‘/commit이 가장 단순 + 가치 큰 첫 Slash. 사내 표준 커밋 메시지 형식이 자동 적용. 1시간 작성. 즉시 효과.’

Q

예상 청중 질문 2: ‘Slash + Skills 결합 가능?’ → 답변: ‘가능. /report Slash가 sales-report Skills를 자동 활성화. 단축 명령 + 도메인 지식이 결합된 가장 강력한 패턴. 본격 운영 단계에서 의미.’

Q

예상 청중 질문 3: ‘몇 개 Slash부터 시작?’ → 답변: ‘첫 도입 1~3개. /commit + /test + /review가 표준 시작. 1개월 운영 후 ‘자주 쓰는 패턴’ 발견 시 추가.’

TIP

강사 함정 — Slash 디테일 깊이 X. ‘위치 + 결합 가치’ 메시지만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 풀이. 1~2분 위치 + 예. 2~3분 Skills와 차이. 3~4분 한국 사례 + 결합. 4~5분 청중 질문.

BRIDGE

“Slash를 봤다면 — MCP 통합으로 넘어갑니다. 4편 3강의 MCP가 Claude Code와 어떻게 결합되는지 봅시다.”

MCP INTEGRATION

MCP 통합 — 4편 + 6편 결합

■ 불변층 — 외부 시스템 + 하네스

- ★ MCP = 4편 3강에서 봤음, 외부 시스템 연결 표준
- ★ Claude Code MCP 통합 — settings.json mcpServers 키
- ★ Hooks + MCP — MCP 호출 전후 검증·로그
- ★ Skills + MCP — Skills 안에서 MCP 도구 호출
- ★ Slash + MCP — Slash가 MCP 자동 호출
- ★ 풀스택 — 4편 MCP + 6편 하네스 = 안전 자동 결합

🕒 5분 · MCP 통합

SAY

“MCP 통합을 봅시다. 4편 3강에서 다룬 MCP(Model Context Protocol)가 Claude Code와 6편의 하네스 4가지 도구와 어떻게 결합되는지 정리합니다.”

SAY

“MCP는 ‘AI의 USB-C’로 4편에서 풀이했어요. 사내 ERP·메일·DB·CAD 같은 외부 시스템을 표준 프로토콜로 연결하는 도구. 본 강좌 4편이 MCP의 본질과 한국 시스템 연결 패턴을 다뤘죠.”

SAY

“Claude Code는 MCP를 자동 통합합니다. settings.json의 mcpServers 키에 MCP 서버를 등록하면 — 사내 ERP MCP 서버·Microsoft 365 메일 MCP 서버·사내 DB MCP 서버 등 — Claude Code가 자동으로 각 서버의 도구를 인식하고 사용할 수 있게 돼요.”

SAY

“Hooks + MCP 결합 — MCP 호출 전후 검증과 로그를 자동화합니다. 예 — 사용자가 ‘이번 분기 매출 알려’ 입력 → Claude Code가 사내 ERP MCP 서버의 get_sales 도구 호출 → PreToolUse Hook이 ‘권한 검증 + 사용자 역할 확인’ → 호출 진행 → PostToolUse Hook이 ‘슬랙 알림 + 감사 로그’. 모든 외부 시스템 호출이 안전망 + 감사를 거칩니다.”

SAY

“Skills + MCP 결합 — Skills 안에서 MCP 도구를 사용. 예 — ‘영업 보고서 작성 Skill’이 본문에 ‘1. sales-erp MCP로 매출 조회 → 2. 보고서 작성’ 같은 흐름. Skill이 자동 호출 되면 → 흐름대로 MCP 도구를 자동 사용. 도메인 지식 + 외부 시스템의 자연스러운 결합이에요.”

SAY

“Slash + MCP 결합 — Slash 명령이 MCP를 자동 호출. /sales-report Slash → 사내 ERP MCP 조회 → 보고서 생성. 사용자가 ‘/sales-report Q4’ 한 줄 입력하면 외부 시스템 데이터 + 보고서 작성이 일괄 진행돼요.”

SAY

“풀스택 결합 — 4편 MCP + 6편 하네스 4가지 = ‘안전한 자동 외부 통합’이에요. MCP 단독으로는 위험(자율 동작 사고)이지만 Hooks + 권한 매트릭스 + Skills 흐름이 결합되어 ‘안전 + 자동 + 도메인 지식’ 3박자가 갖춰져요.”

STORY

settings.json MCP 등록 예시 (강사 대응용). mcpServers 키에 MCP 서버를 등록하는 형식 — ‘mcpServers: { sales-erp: { command: ‘node’, args: [‘./mcp-sales.js’] }, gmail: { url: ‘https://gmail-mcp.local’ } }’ 같은 JSON. 사내에서 실행되는 MCP 서버는 command·args로 실행 명령 지정, 원격 MCP 서버는 url로 지정. Claude Code가 자동 연결하고 각 서버의 도구 목록을 가져와요. 사내 IT가 사내 MCP 서버 5~10개를 1~2개월에 등록 + 통합.

STORY

한국 회사 MCP 통합 사례 (강사 대응용 추정). 일부 한국 IT 회사가 도입한 풀스택 — (1) Microsoft 365 메일 + 캘린더 MCP. 6개월 안 도입. (2) 사내 PostgreSQL DB MCP. 사내 위키·고객 DB 조회. (3) 사내 ERP(더존·SAP) MCP. 1년 안 도입. (4) 사내 JIRA·Confluence MCP. 협업 도구 통합. 4가지 + Hooks + Skills + Slash 결합으로 ‘직원이 한 줄 입력에 사내 시스템 통합 작업 완성’ 가능.

NOTE

★ 강사 핵심 배경지식 — 풀스택의 ‘5가지 가치’: 4편 + 6편 결합으로 회사가 얻는 가치를 정리하면 — (1) 외부 시스템 자동. MCP로 ERP·메일·DB 연결. (2) 정책 자동 적용. Hooks로 사고 사전 방지. (3) 도메인 지식 적용. Skills로 ‘우리 회사 어떻게’ 자동. (4) 단축 명령. Slash로 직원 효율. (5) 감사 로그. 모든 동작 자동 기록. 5가지 결합이 ‘완전한 사내 AI 운영’의 본격 단계예요.

NOTE

강사 배경지식 — 풀스택 구축 권장 순서: 6개월~1년에 도입 가능한 순서 — (1) 0~2개월: 4편 MCP 서버 1~2개. (2) 2~4개월: 6편 1·2강 settings + Hooks 5종. (3) 4~6개월: 6편 3강 Skills 5개 + Slash 3개. (4) 6~9개월: MCP 서버 5~10개로 확장. (5) 9~12개월: 풀스택 안정 + 추가 도메인. 점진 확장이 정석이에요.

Q

예상 청중 질문 1: ‘우리 회사 풀스택 시간은?’ → 답변: ‘6~12개월에 완성. 사내 IT 1~2명 + 부서 챔피언 협업. 외주 옵션 — 약 1,000~3,000만 원에 풀스택 구축 + 인수인계 6개월. 사내 인력 부족 시 검토.’

Q

예상 청중 질문 2: ‘우리 작은 회사 풀스택 가능?’ → 답변: ‘작은 회사도 가능. 단 작은 풀스택 — MCP 1~2개 + Hooks 3종 + Skills 3개 + Slash 2개. 비율은 같지만 규모를 회사 크기에 맞게.’

Q

예상 청중 질문 3: ‘MCP 단독 도입 vs 풀스택?’ → 답변: ‘MCP 단독 = 위험. 외부 시스템 호출 사고 가능. 풀스택 결합 = 안전. Hooks 검증 + 권한 매트릭스 + Skills 흐름이 사고를 사전 방지. 본격 도입 시 풀스택 권장.’

TIP

강사 함정 — 통합 코드 디테일 깊이 X. ‘결합 + 가치’ 메시지만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 MCP 복습. 1~3분 4가지 결합. 3~4분 풀스택 가치. 4~5분 청중 질문.

BRIDGE

“MCP 통합을 봤다면 — 6편의 결론인 ‘하네스 풀스택’을 정리합니다. 4가지 도구의 결합 일정과 가치예요.”

FULLSTACK

하네스 풀스택 — 4요소 결합

■ 불변층 — 6편의 결론

- settings.json 정적 권한 — allow·ask·deny + env + model. 사내 표준 1회 작성 + 영구 적용. 첫 1주 작성.
- Hooks 동적 자동 — Pre·Post·Session·기타 8이벤트+. 정책 자동 + 사고 방지. 1개월에 5종 도입.
- Skills 도메인 지식 — ‘우리 회사 어떻게’ 패키지. 자동 호출. 1년에 20개+ 작성. 도메인 자산화.
- Slash + MCP 단축 + 외부 통합 — /단어 단축 + 외부 시스템 연결. 풀스택 완성. 1~3개월 결합 + 안정.

🕒 5분 · 풀스택

SAY

“6편의 결론 ‘하네스 풀스택’입니다. 4가지 도구 — settings.json + Hooks + Skills + Slash + MCP — 가 어떻게 결합되어 ‘회사 정책 + 안전 자동 + 도메인 지식 + 외부 통합’ 풀스택을 이루는지 정리합니다.”

SAY

“첫 번째 도구 — settings.json. 정적 권한 매트릭스. allow·ask·deny + env + model + statusLine 같은 키로 ‘무엇을 할 수 있나’ 정의. 사내 표준 1회 작성 + git 공유 + 영구 적용. 시간 1주.”

SAY

“두 번째 — Hooks. 동적 자동 메커니즘. PreToolUse·PostToolUse 핵심 + 6가지 추가 이벤트. 정책 자동 적용 + 사고 사전 방지. 한국 회사 5종 1개월 도입 표준.”

SAY

“세 번째 — Skills. 도메인 지식 패키징. ‘우리 회사 어떻게 일하나’ 마크다운 + examples + 자동 호출. 1년에 20개+ 작성하면 사내 도메인 자산화 완성.”

SAY

“네 번째 — Slash Commands + MCP. 단축 명령 + 외부 시스템 통합. /단어 단축어 + 사내 ERP·메일·DB 연결. 4편 + 6편 결합으로 풀스택 완성. 1~3개월 결합 + 안정.”

SAY

“권장 도입 순서를 다시 — 1주 settings.json → 2주 첫 Hook → 1개월 Hooks 5종 → 첫 Skill 1주 → Skills 5개 3개월 → MCP 통합 1~3개월 → 폴스택 안정 6개월. 약 6개월~1년에 본격 폴스택 완성. 단계별 진화가 정석이에요.”

SAY

“핵심 메시지를 다시 — ‘처음부터 완벽한 폴스택 X. 단계별 도입 + 단계별 안정성 + 점진 확장.’ 1단계 안정이 갖춰진 후 2단계, 2단계 안정 후 3단계. 운영 안정성이 항상 우선입니다.”

STORY

Anthropic 사내 폴스택 구조 (강사 대응용). ‘How Anthropic teams use Claude Code’ 인용에 따르면 Anthropic 사내가 갖춘 폴스택 — settings.json + Hooks 다수 + Skills(부서별 5~10개) + Slash + MCP 다수 결합. 사내 전 직원이 동일 환경에서 일. 신규 입사자가 첫날부터 사내 표준 폴스택을 받음. ‘회사 정책 = 코드 + git history’ 패러다임이 본격 자리잡았어요.

STORY

한국 회사 폴스택 ROI (강사 대응용 추정). 폴스택 도입 전후 비교 — (1) 사고: 도입 전 년 5~10건 → 도입 후 0~1건. (2) 직원 시간: 평균 30% 절감(사내 표준 + Skills + Slash 단축). (3) 신규 직원 학습: 100페이지 매뉴얼 → 0(폴스택이 자동 적용). (4) 도메인 일관성: 부서별 결과물이 회사 표준. (5) 외부 시스템 통합: 직원이 한 줄 입력에 통합 작업 완성. 5가지 결합 ROI 200%+.

NOTE

★ 강사 핵심 배경지식 — 폴스택의 ‘회사 효율 변화’: 폴스택 도입이 회사 운영에 미치는 변화를 정리하면 — 도입 전: 매뉴얼 100페이지 + 직원 실수 + 수동 작업 + 도메인 다양성 + 외부 시스템 단절. 도입 후: 정책 자동 적용 + 사고 0 + 자동화 + 도메인 표준 + 외부 통합. ‘회사 운영의 패러다임 전환’이라고 부를 수 있는 변화예요.

NOTE

강사 배경지식 — 폴스택 구축의 ‘5가지 함정’: 본격 폴스택 구축 시 빠지기 쉬운 함정 — (1) 처음부터 완벽. 6개월 안에 폴스택 다 작성 X. 단계별. (2) 학습 부족. 사내 IT가 settings·Hooks·Skills 사전 학습 부족하면 사고. 외주 + 인수인계 권장. (3) 부서장 참여 X. Skills는 부서 챔피언 협업이 필수. IT 단독 X. (4) git 공유 X. 사내 표준이 git에 안 가면 전사 동기화 X. (5) 운영 안정 X 확장. 1단계 안정 안 되고 2단계 진행 → 사고. 5가지 함정 피하기.

Q

예상 청중 질문 1: ‘풀스택 완성 시간이 얼마?’ → 답변: ‘본격 풀스택 6~12개월. 작은 회사 3~6개월. 큰 회사 1~2년. 사내 IT 1~2명 + 부서 챔피언 협업. 외주 옵션 약 1,000~3,000만 원 + 인수인계 6개월.’

Q

예상 청중 질문 2: ‘우리 작은 회사 풀스택?’ → 답변: ‘작은 회사도 가능. 단 ‘작은 풀스택’ — settings 1개 + Hooks 3종 + Skills 3개 + Slash 2개 + MCP 1~2개. 비율은 같지만 규모를 작게. 3~6개월 완성.’

Q

예상 청중 질문 3: ‘1년 운영 후 다음은?’ → 답변: ‘1년 운영 후 본격 자율 단계로 진화. 7편(전사 보급) + 8편(자체 에이전트) + 9편(보안) + 11편(관리자 운영). 6편 풀스택이 7편+ 본격 단계의 기반이에요.’

TIP

강사 함정 — 풀스택 ‘완벽 압박’ X. ‘단계별 진화 + 운영 안정’ 메시지.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 4도구 결합. 1~3분 단계별 도입 순서. 3~4분 ROI + 가치. 4~5분 청중 질문 1~2개.

BRIDGE

“풀스택을 봤다면 — 3강의 마지막. 실습으로 우리 회사 풀스택 6개월 도입 일정을 작성합니다.”

WORKSHOP 3

실습 — 하네스 풀스택 결정서

■ 적용 (워크시트)

- Q1 · 1주 — settings.json 표준 작성?
- Q2 · 1개월 — Hooks 5종 도입?
- Q3 · 3개월 — Skills 5개 작성?
- Q4 · 1~3개월 — MCP 통합 시작?
- Q5 · 6개월 — 풀스택 안정?
- ★ 다음 액션 — 이번 주 settings.json 1차 작성 + git 공유

SAY

“3강의 마지막 — 실습입니다. 6개월 하네스 풀스택 도입 일정 1페이지를 작성합니다. 사내 IT + 부서 챔피언 + 의사결정자가 함께 보는 마스터 계획이에요.”

DO

조별 실습 진행. 3~4인 1조로 약 5분간 5칸을 채웁니다. Q1~Q5 + 다음 액션. 강사는 조별로 돌며 가이드.

SAY

“권장 도입 순서 — 1주 settings.json + 1개월 Hooks 5종 + 3개월 Skills 5개 + 1~3개월 MCP 통합 + 6개월 풀스택 안정. 단계별 시간이 명확해요. 90% 한국 회사가 이 일정에 따라가면 풀스택이 완성됩니다.”

Q

발표 요청 1: ‘1~2 조 발표해주세요. 6개월 후 풀스택 가능 여부 + 가장 큰 도전.’ 보통 ‘가능하지만 인력 부족’ 또는 ‘부서장 참여 어려움’ 답변. ‘좋습니다. 두 가지 모두 현실적 도전이에요. 사내 IT 양성 + 부서장 설득이 핵심.’

Q

발표 요청 2: ‘이번 주 settings.json 시작 가능한 조?’ → 보통 절반. ‘좋습니다. 시작이 가장 어려운데 시작만 하면 진행이에요. 시니어 1명이 1주 안 첫 settings.json을 작성하면 그 다음 단계가 자연스럽게 이어집니다.’

NOTE

★ 강사 핵심 배경지식 — 결정서 5칸 작성 가이드: 청중이 막힐 수 있는 부분을 정리하면 — (1) Q1 settings: 시니어 1명이 1주에 가능. 첫 우선 작업. (2) Q2 Hooks 5종: 1개월 단계별. 1번 .env 차단 → 2번 PII → 3번 commit → 4번 자동 테스트 → 5번 슬랙. (3) Q3 Skills 5개: 3개월. 부서별 1개씩. 부서 챔피언 협업. (4) Q4 MCP: 4편 진행 중이거나 완료 후. Microsoft 365 메일이 시작. (5) Q5 풀스택 안정: 6개월. 운영 + 피드백 + 개선.

NOTE

강사 배경지식 — 다음 주 액션 권장 3가지: 1주 안 시작할 수 있는 액션 — (1) 사내 시니어 1명이 첫 settings.json 1주 안 작성. (2) 첫 Hook(.env 차단) 1주 안 작성 + 1주 dry run. (3) git 공유 + 사내 표준 + CLAUDE.md 업데이트. 1주. 셋 다 2주 안 가능.

NOTE

강사 배경지식 — 풀스택 ‘부서장 참여’ 패턴: Skills 도입은 부서장·부서 챔피언 협업이 필수예요. IT 단독으로는 ‘영업 보고서 양식’을 모름. 부서장이 ‘우리 영업팀은 이런 식으로 보고서 쓴다’ 인풋 → IT가 코드화. 1편 ‘챔피언 네트워크’ 패턴이 6편 풀스택의 핵심 동력. 청중에게 ‘IT만의 작업이 아니다’ 인지시키세요.

Q

예상 청중 질문 1: ‘우리 회사 6개월 너무 빠른가?’ → 답변: ‘IT + 사내 시니어 + 부서장 의지가 있으면 6개월 가능. 모자라면 9~12개월 일정으로 조정. ‘6개월 목표 + 9개월 완성’이 안전한 패턴.’

Q

예상 청중 질문 2: ‘우리 IT 1명 부담?’ → 답변: ‘외주 옵션 + 사내 인수인계 정석. 사내 IT 1명이 외주와 함께 6개월 작업 → 자체 유지. 사내 학습 가치 큼.’

Q

예상 청중 질문 3: ‘부서장 참여 강제 어떻게?’ → 답변: ‘1편의 ‘챔피언 네트워크’ 패턴 적용. 부서장 + 부서 챔피언이 ‘Skills 작성 = 부서 가치’ 인지하게. 사장님 지원 + ROI 측정으로 동기 부여.’

TIP

강사 함정 — ‘완벽 결정 압박’ X. ‘60% 답으로 시작 → 운영 후 조정’ 메시지.

TIP

강사 진행 팁 — 8분 시간 배분: 0~5분 개별·조별 작성. 5~7분 1~2조 발표. 7~8분 ‘다음 주 액션 3가지’ + 마무리. 풀스택의 시작점 ‘이번 주 settings.json’이 청중 손에 남도록.

BRIDGE

“실습 끝났습니다. 3강과 6편 마무리로 넘어갑니다.”

PART 6 WRAP

6편 정리 — 하네스 풀스택 마스터

■ 마무리 + 7편 예고

- 1강 — settings.json (권한 + Permission Modes + 5종 패턴)
- 2강 — Hooks (PreToolUse·PostToolUse + 한국 5종)
- 3강 — Skills + Slash + MCP 결합
- ★ 1~6편 합치면 ‘에이전트 안전 운영 16장 마스터’
- ★ 다음 — 7편(사내 에이전트 배포) 전사 보급

🕒 5분 · 마무리

SAY

“6편을 정리합니다. 1강에서 settings.json의 정적 권한, 2강에서 Hooks의 동적 자동, 3강에서 Skills·Slash·MCP의 도메인 지식 + 단축 + 외부 통합을 다뤘어요. 4가지 도구가 결합되어 ‘하네스 풀스택’을 이루는 그림이 청중 손에 남았으면 좋겠습니다.”

SAY

“오늘 손에 남은 것 — settings.json 표준 1페이지(1강), 첫 Hook 설계서 1페이지(2강), 하네스 풀스택 6개월 일정 1페이지(3강). 3장의 워크시트가 사내 IT + 부서장에게 ‘이렇게 진행해 주세요’ 발주서가 됩니다.”

SAY

“1편부터 6편까지 합치면 ‘에이전트 안전 운영 마스터’가 완성됩니다. 1편 의사결정(왜) + 2편 인프라 결정서·장비 구매서·보안 도면(3장) + 3편 모델·추론 셋(3장) + 4편 UI·RAG·MCP(3장) + 5편 에이전트 도구 결정(3장) + 6편 하네스 풀스택(3장) = 총 16장. 사장님께 보고하는 마스터 계획이에요.”

SAY

“추천 시작 셋을 다시 — ‘사내 IT 시니어 1명 + 1주 settings.json + 1개월 Hooks 5종 + 3개월 Skills 5개 + 6개월 풀스택 안정.’ 이번 주 안 시작이 가능합니다. ‘60% 답으로 시작 → 운영하면서 보완’ 메시지가 6편의 결론이에요.”

SAY

“7편 예고 — ‘사내 에이전트 배포’입니다. 6편이 ‘개발팀 + 하네스 풀스택’이었다면 7편이 ‘영업·법무·HR·재무 등 전사 보급’입니다. Cowork 6개월 framework + 부서별 권한 매트릭스 + 카탈로그·챔피언 네트워크. 전사 도입의 본격 패턴이에요. 6편 풀스택 위에 7편 전사가 올라가는 구조입니다.”

NOTE

★ 강사 핵심 배경지식 — 6편 끝 톤: 청중에게 ‘오늘 6편 결과물 3장이 회사에서 즉시 적용 가능한 마스터 계획이다’ 인지시킵니다. 다음 7편이 ‘전사 보급’으로 자연스럽게 이어지면서, 청중이 ‘우리 회사 6편 완성 → 7편 도입 6개월 일정’ 시간 감각을 갖게 됩니다.

NOTE

강사 배경지식 — 11편 강좌 전체에서 6편의 위치: 1~5편이 ‘기술 + 도구 결정’이었다면 6편이 ‘운영 표준화’. 7편이 ‘전사 보급’. 8편이 ‘자체 에이전트 본격’. 9편이 ‘보안’. 10편이 ‘백업·DR’. 11편이 ‘관리자 운영’. 6편이 본 강좌의 ‘본격 운영의 기반’ 위치예요. 6편이 잘 갖춰지면 7~11편이 자연스럽게 이어집니다.

NOTE

강사 배경지식 — 청중 ‘피로 인지’: 6편 3강 = 180분. 청중이 피곤합니다. ‘다 외우려 X. 워크시트 3장 + 이번 주 settings.json 시작’만 손에 남으면 충분’이라는 안심 메시지로 마무리.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 3강 결과물 정리. 1~2분 1~6편 16장 마스터. 2~3분 추천 시작 셋. 3~4분 7편 예고. 4~5분 청중 발표 또는 질문.

BRIDGE

“6편 끝났습니다. 7편 ‘사내 에이전트 배포’에서 만나요. 6편 풀스택 위에 7편 전사 보급이 올라가는 구조예요. 청중 여러분 고생하셨습니다.”

THANKS

6편 끝 — 7편에서

‘사내 에이전트 배포’ 전사 보급

🕒 2분 · 작별

SAY

“6편 마지막 슬라이드입니다. 1·2·3강 180분 동안 함께 해주셔서 감사합니다.”

SAY

“오늘 손에 남은 것을 다시 — settings.json 표준 1페이지 + 첫 Hook 설계서 1페이지 + 하네스 풀스택 6개월 일정 1페이지. 3장이 사내 IT + 부서장에게 ‘이렇게 진행해 주세요’ 발주서가 됩니다.”

SAY

“다음 단계 — 7편 ‘사내 에이전트 배포’. 6편이 ‘개발팀 + 하네스’였다면 7편이 ‘전사 보급’이에요. 영업·법무·HR·재무 등 전 부서에 안전하게 배포하는 6개월 framework + 부서별 매트릭스 + 카탈로그·챔피언이 7편의 본문입니다.”

SAY

“오늘 강의 후기·질문은 visionc.co.kr 사이트 또는 사내 챔피언과 공유 부탁드립니다. 다음 시간에 만나요. 청중 여러분 고생하셨습니다.”

NOTE

★ 강사 핵심 배경지식 — 마무리 톤: 청중 피로 인지 + 칭찬 + 다음 동기 부여. ‘오늘 결과물이 회사에서 즉시 적용 가능’ 메시지로 자신감 부여. 따뜻한 작별이 청중에게 ‘좋은 강의였다’ 인상을 남깁니다.

TIP

강사 진행 팁 — 2분 안에 따뜻한 작별. 청중과 눈맞춤. 손 흔들기.

BRIDGE

“감사합니다.”